JAVASCRIPT · TYPESCRIPT

CHEAT SHEETS

A Complete Reference for Modern JavaScript and TypeScript

Variables & Types · Strings · Operators · Functions & Closures · Arrays & Objects · Classes · Modules & npm · Async / Await · DOM & Events · fetch & Browser APIs · Node.js · TypeScript · Generics & Utility Types

133 REFERENCE SHEETS · 20 SECTIONS

CONTENTS

LANGUAGE CORE

001–004 Introduction & Setup
005–006 Output & Comments
007–015 Variables & Data Types
016–021 Strings
022–029 Operators
030–036 Control Flow
037–040 Error Handling
041–049 Functions

DATA, OOP & RUNTIME

050–062 Data Structures
063–070 OOP
071–075 Files & I/O
076–081 Modules & Packages
082–085 External Libraries & Tooling
086–091 Advanced Topics
092–096 Asynchronous JavaScript

PLATFORM & TYPESCRIPT

097–102 DOM
103–106 Browser APIs
107–110 Node.js Essentials
111–121 TypeScript Fundamentals
122–131 TypeScript Type System
132–133 Quick Reference

JS · TS

Table of Contents

133 sheets · 21 sections

INTRODUCTION & SETUP 001 – 004

001 Introduction & History
002 Runtimes & Project S…
003 Program Structure
004 Running & Debugging

OUTPUT & COMMENTS 005 – 006

005 Console Output
006 Comments & JSDoc

VARIABLES & DATA TYPES 007 – 015

007 Variables & Declarati…
008 Numbers
009 BigInt
010 Booleans & Truthiness
011 null & undefined
012 Symbols
013 Type Conversion & C…
014 Type Checking
015 User Input

STRINGS 016 – 021

016 String Basics
017 Template Literals
018 String Methods — Se…
019 String Methods — Tra…
020 Slicing & Indexing
021 Number & Locale For…

OPERATORS 022 – 029

022 Arithmetic Operators
023 Comparison Operators
024 Logical Operators
025 Nullish Coalescing & …
026 Assignment & Comp…
027 Bitwise Operators
028 Spread & Rest
029 Other Operators & Pr…

CONTROL FLOW 030 – 036

030 if / else if / else
031 Ternary & Conditional…
032 switch
033 for Loops
034 for…of & for…in
035 while & do…while
036 Loop Controls

ERROR HANDLING 037 – 040

037 try / catch / finally
038 Throwing Errors
039 Custom Errors
040 Global Handlers & Err…

FUNCTIONS 041 – 049

041 Declarations & Expre…
042 Parameters & Defaults
043 Rest Parameters & Sp…
044 Variable Scope
045 Closures
046 Arrow Functions
047 this & Binding
048 Higher-Order Functio…
049 Recursion

DATA STRUCTURES 050 – 062

050 Arrays — Basics
051 Array Methods — Mut…
052 Array Methods — Tra…
053 Array Methods — Sea…
054 Objects — Basics
055 Object Static Methods
056 Destructuring
057 Copying & Equality
058 Map & WeakMap
059 Set & WeakSet
060 Sorting & Searching
061 Iterators & Iterables
062 Generators

OOP 063 – 070

063 Prototypes
064 Classes & Objects
065 Constructors
066 Fields, Static & Privat…
067 Getters & Setters
068 Inheritance
069 Polymorphism, Duck …
070 Protocol Methods & …

FILES & I/O 071 – 075

071 JSON
072 Reading Files (Node)
073 Writing & Appending …
074 File Paths (Node)
075 File & Blob APIs (Brow…

MODULES & PACKAGES 076 – 081

076 ES Modules
077 CommonJS & Interop
078 npm & package.json
079 Math & Random
080 Date
081 Temporal

EXTERNAL LIBRARIES & TOOLING 082 – 085

082 Testing with Vitest
083 ESLint & Prettier
084 Vite & Build Tools
085 Runtime Validation wi…

ADVANCED TOPICS 086 – 091

086 Regular Expressions …
087 Regular Expressions …
088 Proxy & Reflect
089 Typed Arrays & Binar…
090 Workers & Parallelism
091 Memory & Resource …

ASYNCHRONOUS JAVASCRIPT 092 – 096

092 The Event Loop & Tim…
093 Promises
094 async / await
095 Promise Combinators
096 Cancellation, Timeout…

DOM 097 – 102

097 Selecting & Traversing
098 Content, Attributes &…
099 Creating & Removing …
100 Events
101 Propagation & Delega…
102 Forms & Input

BROWSER APIS 103 – 106

103 fetch & HTTP
104 Storage & Cookies
105 URL & History
106 Observers & Animati…

NODE.JS ESSENTIALS 107 – 110

107 process & Environme…
108 EventEmitter & Strea…
109 HTTP Server (node:ht…
110 Node Utility Modules

TYPESCRIPT FUNDAMENTALS 111 – 121

111 TypeScript: Introducti…
112 tsconfig.json
113 Basic Types
114 Inference, Widening …
115 Type Aliases & Interfa…
116 Object Types
117 Unions & Intersections
118 Narrowing & Type Gu…
119 Function Types & Ov…
120 Enums & Literal Unions
121 Classes in TypeScript

TYPESCRIPT TYPE SYSTEM 122 – 131

122 Generics
123 keyof, typeof & Index…
124 Utility Types — Objec…
125 Utility Types — Union…
126 Conditional Types
127 Mapped & Template …
128 Declaration Files
129 Assertions & Escape …
130 Decorators
131 Adopting TypeScript

QUICK REFERENCE 132 – 133

132 Quick Reference — Ja…
133 Quick Reference — M…

Introduction & History

OVERVIEW

JavaScript is the language of the web: every browser runs it, and runtimes like **Node.js**, Deno, and Bun run it on servers and desktops. The language itself is standardized as **ECMAScript** by the TC39 committee, which ships a new edition every June (ES2015 was the big modernizing release; ES2025 is current). Engines such as V8 (Chrome, Node) and SpiderMonkey (Firefox) compile it just-in-time.

NOTES

- Created by Brendan Eich at Netscape in 1995, in about ten days
- Not related to Java; the name was a marketing decision
- `.js` follows the nearest package.json `"type"`; `.mjs` is always ESM, `.cjs` always CommonJS
- Dynamically typed, garbage collected, single-threaded with an event loop

HELLO, WORLD — THREE WAYS

```
// 1. Browser: inside an HTML page
<script type="module">
  console.log("Hello, World");   // shows in DevTools console
</script>

// 2. Node.js: save as hello.js, then run
console.log("Hello, World");     // $ node hello.js

// 3. REPL: type `node` with no file
> 2 + 2
4
> "Hello".toUpperCase()
'HELLO'
```

CODE EXPLAINED

- → `<script type="module">` — modern browser entry point; enables `import`
- → `console.log` — standard output in both browser and Node
- → `node file.js` — runs a file; `node` alone opens the REPL
- → The REPL prints the value of each expression automatically

TYPESCRIPT

- **TypeScript** is a typed superset by Microsoft: every valid JS file is valid TS
- Files use `.ts` / `.tsx`; types are erased, so the output is plain JS

KEY ECMASCRIPT EDITIONS

EDITION	HIGHLIGHTS
ES5 (2009)	strict mode, JSON, Array.forEach
ES2015 (ES6)	let/const, classes, arrows, modules, Promise
ES2017	async / await
ES2020	??, ?., BigInt, globalThis
ES2022	class #private, top-level await, .at()
ES2024–25	groupBy, Set methods, iterator helpers

WHERE JAVASCRIPT RUNS

ENVIRONMENT	TYPICAL USE
Browser	UI, DOM, fetch, web apps
Node.js	servers, CLIs, build tools
Deno / Bun	Node alternatives, built-in TS
Electron	desktop apps (VS Code)
React Native	mobile apps
Edge workers	Cloudflare, Vercel functions

⚠ COMMON MISTAKES

- Calling it "Java" or assuming Java knowledge transfers; the type systems and object models are different
- Following pre-2015 tutorials that use `var` and callbacks everywhere
- Forgetting `type="module"` and then wondering why `import` throws

✓ TIPS & BEST PRACTICES

- Check feature support on MDN's compatibility tables before shipping to older browsers
- Use the REPL or DevTools console to test one-liners quickly
- Learn modern syntax first (ES2015+); legacy patterns can wait

QUICK SUMMARY

- `ECMAScript` — the standard; yearly editions
- `V8 / SpiderMonkey` — engines that run it
- `node file.js` — run outside the browser
- `TypeScript` — JS plus a compile-time type layer

Runtimes & Project Setup

OVERVIEW

You can run JavaScript in any browser with no setup, but real projects use a **runtime** (usually Node.js) and **npm** to manage packages. A project is just a folder with a `package.json`. Setting `"type": "module"` there makes every `.js` file an ES module, which is what modern code expects.

NOTES

- Install Node from nodejs.org or through a version manager like `nvm` or `fnm`
- Use an LTS (even-numbered) Node release for real work
- npm ships with Node; `pnpm`, `yarn`, and `bun` are alternatives
- Deno and Bun include a formatter, test runner, and TS support out of the box

NEW PROJECT FROM SCRATCH

```
$ nvm install --lts          # install latest LTS Node
$ node --version             # v24.x.x
$ npm --version              # 11.x.x

$ mkdir my-app && cd my-app
$ npm init -y                # creates package.json

// package.json
{
  "name": "my-app",
  "type": "module",          // .js files are ES modules
  "scripts": { "start": "node index.js" }
}

$ npm start                  # runs node index.js
```

CODE EXPLAINED

- → `nvm install --lts` — installs and switches to the current LTS
- → `npm init -y` — accepts all defaults
- → `"type": "module"` — enables `import` / `export` in `.js` files
- → `scripts` — named commands run with `npm run <name>`

TYPESCRIPT

- `npm i -D typescript` then `npx tsc --init` creates `tsconfig.json`
- Deno, Bun, and recent Node versions run `.ts` directly by stripping types (no type checking)

RUNTIMES COMPARED

RUNTIME	RUN A FILE	NOTES
Node.js	node app.js	largest ecosystem
Deno	deno run app.ts	secure by default, permissions flags
Bun	bun app.ts	fast, npm-compatible
Browser	<script src>	no file system access

SETUP COMMANDS

COMMAND	DOES
`nvm ls`	list installed Node versions
`nvm use 22`	switch version
`npm init -y`	new package.json
`npm i <pkg>`	add a dependency
`npx <tool>`	run a package binary

⚠ COMMON MISTAKES

- Installing Node with the system package manager and getting an outdated version
- Leaving out `"type": "module"`, so `import` fails with "Cannot use import statement outside a module"
- Committing `node_modules/`; add it to `.gitignore`

✓ TIPS & BEST PRACTICES

- Add an `.nvmrc` file with the Node version so teammates match
- Use `node --env-file=.env` instead of the dotenv package on modern Node
- Run `npm doctor` if installs behave strangely

QUICK SUMMARY

- `nvm` — manage Node versions
- `npm init` — create package.json
- `"type": "module"` — modern module mode
- `npx` — run tools without global installs

Program Structure

OVERVIEW

A JavaScript program is a sequence of **statements** executed top to bottom. Files are either classic **scripts** (shared global scope) or **modules** (own scope, strict mode on, can use `import`/`export` and top-level `await`). Semicolons are optional thanks to **automatic semicolon insertion (ASI)**, but a few edge cases make it worth understanding.

NOTES

- There's no `main()` entry point; the file runs from the top
- Modules are always in strict mode; scripts need `"use strict"`
- Blocks `{ }` create scope for `let` / `const`
- Indentation is cosmetic; braces define structure

ANATOMY OF A MODULE

```
import { readFile } from "node:fs/promises";  // imports first

const PATH = "./data.txt";                    // module-scoped constant

function countLines(text) {                   // declaration (hoisted)
  return text.split("\n").length;
}

const text = await readFile(PATH, "utf8");    // top-level await (modules only)
console.log(countLines(text));                // 42

export { countLines };                        // public API

// ASI trap: a line starting with ( or [ joins the previous line
const a = 1
;[1, 2].forEach(n => console.log(n))          // leading ; protects it
```

CODE EXPLAINED

→ `import` — hoisted and resolved before any code runs

→ Top-level `await` — only allowed in modules

→ `export` — makes names available to other modules

→ Leading `;` — guards lines that start with `(`, `[`, or a template literal

TYPESCRIPT

- Same structure; a file with any `import` / `export` is a module, otherwise its declarations become global across files
- Use `export {}` to force a file to be a module

SCRIPT VS MODULE

FEATURE	SCRIPT	MODULE
`Scope`	global	per file
`Strict mode`	opt-in	always
`import / export`	no	yes
`Top-level await`	no	yes
`this at top`	globalThis	undefined

ASI RULES OF THUMB

SITUATION	RESULT
`return\n value`	returns undefined
`line starts with (`	joins previous line
`line starts with [`	joins previous line
`line starts with ` `	tagged template call
`x\n++y`	parsed as x; ++y

⚠ COMMON MISTAKES

- Putting the return value on the line after `return`; ASI inserts a semicolon and returns `undefined`
- Relying on globals between `<script>` tags, then breaking when switching to modules
- Mixing semicolon styles; pick one and enforce it with Prettier

✓ TIPS & BEST PRACTICES

- Default to modules everywhere; scripts are legacy
- If you skip semicolons, let a formatter add the defensive leading `;` for you
- Keep imports at the top and exports at the bottom for readability

QUICK SUMMARY

- `statements` — run top to bottom
- `module` — own scope, strict, import/export
- `ASI` — inserts semicolons, with traps
- `{ }` — block scope for let/const

Running & Debugging

OVERVIEW

Run files with `node`, and use `--watch` to restart on save. For debugging, the `debugger` statement pauses execution whenever DevTools (or Node's inspector) is attached, letting you inspect variables, step through code, and view the call stack. **Source maps** connect bundled or compiled output back to your original files.

NOTES

- `node --inspect file.js` lets Chrome DevTools attach via `chrome://inspect`
- VS Code's JavaScript Debug Terminal attaches automatically
- `console.trace()` prints a stack trace without stopping
- Breakpoints can be conditional: right-click the line number

RUNNING AND PAUSING

```
$ node app.js                  // run once
$ node --watch app.js          // rerun on file change
$ node --inspect-brk app.js    // pause on first line for debugger

function total(items) {
  let sum = 0;
  for (const item of items) {
    debugger;                  // pauses here when DevTools is open
    sum += item.price;
  }
  return sum;
}

console.trace("reached total");  // logs call stack, keeps going
console.assert(sum >= 0, "negative!"); // logs only if false
```

CODE EXPLAINED

- → `--watch` — built into Node 18+; no nodemon needed
- → `--inspect-brk` — waits for a debugger before running
- → `debugger` — no-op unless a debugger is attached
- → `console.assert` — prints only when the condition is false

TYPESCRIPT

- Source maps (`"sourceMap": true`) let breakpoints and stack traces point at `.ts` lines
- Use `tsx` (`npx tsx watch app.ts`) to run and watch TypeScript during development

DEVTOOLS STEPPING

KEY	ACTION
F8	resume
F10	step over
F11	step into
Shift+F11	step out
Ctrl+Shift+P	command menu

USEFUL NODE FLAGS

FLAG	DOES
--watch	restart on change
--inspect	enable debugger
--env-file=.env	load env vars
--test	run node:test files
-e "code"	evaluate a string

⚠ COMMON MISTAKES

- Leaving `debugger` statements in committed code; add an ESLint rule (`no-debugger`)
- Debugging minified bundle output without source maps
- Logging objects and reading them later: browser consoles show the object's *current* state, not the state when logged

✓ TIPS & BEST PRACTICES

- Log a snapshot with `console.log(structuredClone(obj))` or `JSON.stringify`
- Use logpoints in DevTools instead of adding temporary `console.log` lines
- Name the variable in logs: `console.log({ user })` prints `{ user: … }`

QUICK SUMMARY

- `node --watch` — auto-restart
- `debugger` — breakpoint in code
- `--inspect` — attach DevTools to Node
- `source maps` — map output back to source

Console Output

OVERVIEW

The `console` object is the standard output tool in both browsers and Node. Beyond `log`, it has levels (`info`, `warn`, `error`), structured output (`table`, `dir`), grouping, counting, and timing. In Node, `console.error` and `console.warn` write to **stderr**; the rest write to **stdout**.

NOTES

- Multiple arguments are separated by spaces
- Format specifiers work like printf: `%s %d %o %c`
- `%c` applies CSS, browser only
- `process.stdout.write()` prints without a trailing newline (Node)

CONSOLE METHODS WITH OUTPUT

```
console.log("Total:", 42, true);      // Total: 42 true
console.log("%s is %d", "Ana", 30);   // Ana is 30
console.log(`Sum: ${2 + 3}`);         // Sum: 5
console.error("Disk full");           // stderr, red in browsers
console.warn("Deprecated");           // stderr, yellow
console.table([{ id: 1, n: "a" }, { id: 2, n: "b" }]);  // grid
console.dir(obj, { depth: null });    // full nested object (Node)
console.group("Load");                // indents following logs
console.log("step 1");                //   step 1
console.groupEnd();
console.count("hit");                 // hit: 1
console.count("hit");                 // hit: 2
console.time("q"); /* work */ console.timeEnd("q");  // q: 3.2ms
console.log("%cBig", "font-size:20px"); // styled (browser)
```

CODE EXPLAINED

→ `table` — arrays of objects render as rows and columns

→ `dir` with `depth: null` — avoids Node's `[Object]` truncation

→ `group` / `groupEnd` — nest related output

→ `time` / `timeEnd` — labels must match exactly

TYPESCRIPT

- Identical in TypeScript
- `console` types come from `lib.dom` (browser) or `@types/node`

LOG LEVELS

METHOD	STREAM	USE
`log`	stdout	general output
`info`	stdout	informational
`debug`	stdout	verbose (hidden by default)
`warn`	stderr	something odd
`error`	stderr	something failed

FORMAT SPECIFIERS

SPEC	FORMATS AS
`%s`	string
`%d / %i`	integer
`%f`	float
`%o / %O`	object
`%c`	CSS style (browser)

⚠ COMMON MISTAKES

- Using `+` to join objects: `"user: " + obj` prints `user: [object Object]`; pass it as a separate argument
- Leaving noisy logs in production code; use a logger with levels
- Expecting `console.debug` to show in Chrome; it's hidden unless Verbose is enabled

✓ TIPS & BEST PRACTICES

- `console.log({ a, b })` labels values automatically
- Use `console.table` for arrays of records; it's far easier to scan
- Redirect only errors in Node: `node app.js 2> errors.log`

QUICK SUMMARY

- `console.log` — stdout
- `console.error / warn` — stderr
- `console.table` — tabular view
- `console.time` — quick timing

Comments & JSDoc

OVERVIEW

JavaScript has two comment forms: `//` for a single line and `/* */` for blocks. A block that starts with `/` **is a** JSDoc** comment; editors read it to show hover documentation, parameter hints, and even type information. Good comments explain *why*, not *what*.

NOTES

- Block comments cannot be nested
- `/** */` must sit directly above the declaration it documents
- JSDoc tags start with `@`
- `#!/usr/bin/env node` on line 1 is a hashbang, not a comment, but is allowed

COMMENT FORMS AND JSDOC

```
// Single-line comment
const rate = 0.07; // trailing comment

/* Block comment
   spanning lines */

/**
 * Calculates the price including tax.
 * @param {number} price - Base price in dollars
 * @param {number} [rate=0.07] - Tax rate (optional)
 * @returns {number} Price with tax, rounded to cents
 * @example withTax(10) // 10.7
 * @deprecated Use priceWithTax() instead
 */
function withTax(price, rate = 0.07) {
  return Math.round(price * (1 + rate) * 100) / 100;
}
```

CODE EXPLAINED

- → `@param {type} name` — type and description for one parameter
- → `[rate=0.07]` — brackets mark an optional param with a default
- → `@returns` — return type and meaning
- → `@deprecated` — editors strike through the name at call sites

TYPESCRIPT

- Types move into syntax (`price: number`), so JSDoc keeps only descriptions
- Add `// @ts-check` at the top of a `.js` file to type-check it using JSDoc types

COMMON JSDOC TAGS

TAG	PURPOSE
`@param`	parameter type and description
`@returns`	return value
`@type`	type of a variable
`@typedef`	define a reusable type
`@throws`	errors it can throw
`@see / @link`	cross-references

SPECIAL COMMENTS

COMMENT	EFFECT
`// @ts-check`	type-check this JS file
`// @ts-expect-error`	expect an error on next line
`// eslint-disable-next-line`	skip lint for one line
`// TODO:`	flagged by most editors
`/* webpackChunkName */`	bundler hints

⚠ COMMON MISTAKES

- Commenting what the code obviously does (`i++ // increment i`)
- Letting comments go stale after the code changes; wrong comments are worse than none
- Writing `/*` instead of `/**`, so editors ignore the documentation

✓ TIPS & BEST PRACTICES

- Use JSDoc on exported functions; editors show it on hover everywhere they're used
- Write `TODO(name):` with an owner so it can be tracked
- Use `@example` to show a one-line call and its result

QUICK SUMMARY

- `//` — single line
- `/* */` — block
- `/** */` — JSDoc, read by editors
- `@param / @returns` — document functions

Variables & Declarations

let · const · var · hoisting · TDZ · naming conventions · const ≠ immutable

OVERVIEW

Declare variables with `const` by default and `let` when the value must be reassigned. Avoid `var`: it is function-scoped and hoisted with an `undefined` value, which causes subtle bugs. `let` and `const` are block-scoped and sit in a **temporal dead zone (TDZ)** until their declaration runs, so using them early throws.

NOTES

- `const` prevents reassignment, not mutation of the object it holds
- Names: letters, digits, `_`, `$`; can't start with a digit
- Convention: `camelCase` for variables, `UPPER_SNAKE` for true constants
- Undeclared assignment creates a global in sloppy mode; throws in strict mode

LET, CONST, VAR

```
const name = "Ana";          // can't reassign
let count = 0;               // can reassign
count += 1;                  // 1

const user = { age: 30 };
user.age = 31;               // OK: mutating, not reassigning
// user = {};                // TypeError: Assignment to constant

console.log(x);              // undefined (var is hoisted)
var x = 5;

// console.log(y);           // ReferenceError: TDZ
let y = 5;

if (true) { var v = 1; let b = 2; }
console.log(v);              // 1  (var leaks out of the block)
// console.log(b);           // ReferenceError
let a = 1, z = 2;            // multiple declarations
```

CODE EXPLAINED

- → `const` — binding is fixed; contents can change
- → `var` hoisting — declaration moves up, value doesn't
- → TDZ — `let` / `const` exist but can't be touched before their line
- → `var` ignores block scope; `let` respects it

TYPESCRIPT

- Annotate with `let count: number = 0`, but inference usually handles it
- `const` infers a literal type (`"Ana"`); `let` widens to `string`

LET VS CONST VS VAR

	CONST	LET	VAR
Scope	block	block	function
Reassign	no	yes	yes
Hoisted	TDZ	TDZ	as undefined
Redeclare	no	no	yes
Global prop	no	no	yes

NAMING CONVENTIONS

STYLE	USED FOR
camelCase	variables, functions
PascalCase	classes, components
UPPER_SNAKE	fixed config constants
_leading	"private" by convention
$el	DOM / jQuery refs

⚠ COMMON MISTAKES

- Assuming `const` makes objects immutable; use `Object.freeze` for that
- Using `var` in loops with callbacks, so every callback sees the final value
- Reusing one `let` for unrelated values; declare a new `const` instead

✓ TIPS & BEST PRACTICES

- Start with `const`; change to `let` only when the linter or a reassignment requires it
- Enable ESLint's `prefer-const` and `no-var`
- Declare variables as close to first use as possible

QUICK SUMMARY

- `const` — default choice
- `let` — when reassigning
- `var` — legacy, avoid
- `TDZ` — no access before declaration

Numbers

OVERVIEW

JavaScript has one number type, `Number`, a 64-bit IEEE 754 float used for both integers and decimals. Integers are exact up to `Number.MAX_SAFE_INTEGER` ($2^{53}-1$); beyond that, use `BigInt`. Floating-point rounding means `0.1 + 0.2 !== 0.3`, and invalid math produces `NaN` rather than throwing.

NOTES

- `NaN` is the only value not equal to itself
- Division by zero gives `Infinity`, not an error
- `-0` exists; `Object.is(-0, 0)` is `false`
- Numeric separators `_` are just for readability

LITERALS AND SPECIAL VALUES

```
const big = 1_000_000;            // 1000000
const hex = 0xff;                 // 255
const bin = 0b1010;               // 10
const oct = 0o17;                 // 15
const sci = 2.5e3;                // 2500

0.1 + 0.2;                        // 0.30000000000000004
0.1 + 0.2 === 0.3;                // false
Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON;  // true

1 / 0;                            // Infinity
"abc" * 2;                        // NaN
Number.isNaN(NaN);                // true
Number.isInteger(5.0);            // true
Number.isSafeInteger(2 ** 53);    // false
Number.MAX_SAFE_INTEGER;          // 9007199254740991
```

CODE EXPLAINED

→ Prefixes `0x`, `0b`, `0o` — hex, binary, octal

→ `Number.EPSILON` — tolerance for float comparison

→ `Number.isNaN` — reliable; global `isNaN` coerces first

→ Past 2^{53}, consecutive integers can't be represented

TYPESCRIPT

- One `number` type for all of these; there's no `int` or `float`
- Literal types like `type Dice = 1 | 2 | 3 | 4 | 5 | 6` restrict values

NUMBER CONSTANTS

CONSTANT	VALUE
`MAX_SAFE_INTEGER`	$2^{53} - 1$
`MIN_SAFE_INTEGER`	$-(2^{53} - 1)$
`MAX_VALUE`	≈ 1.8e308
`MIN_VALUE`	≈ 5e-324 (smallest > 0)
`EPSILON`	≈ 2.2e-16
`POSITIVE_INFINITY`	Infinity

NUMBER CHECKS

METHOD	EXAMPLE → RESULT
`Number.isNaN`	isNaN("x") → false
`Number.isFinite`	isFinite("5") → false
`Number.isInteger`	isInteger(5.5) → false
`Number.isSafeInteger`	isSafeInteger(2**53) → false
`isNaN (global)`	isNaN("x") → true (coerces)

⚠ COMMON MISTAKES

- Comparing floats with `===`; compare the difference against a tolerance
- Storing money as floats; use integer cents or a decimal library
- Checking `x === NaN`, which is always false; use `Number.isNaN(x)`

✓ TIPS & BEST PRACTICES

- Round for display with `toFixed`, but keep full precision in calculations
- Use `BigInt` for IDs or counters that can exceed 2^{53}
- Write large literals with `_` separators: `1_000_000`

QUICK SUMMARY

- `Number` — one 64-bit float type
- `NaN` — invalid result, never equals itself
- `Infinity` — overflow or ÷ 0
- `MAX_SAFE_INTEGER` — exact-integer limit

BigInt

OVERVIEW

`BigInt` represents integers of any size, exactly. Create one with an `n` suffix (`10n`) or `BigInt()`. BigInts support the usual arithmetic, but you **can't mix** them with Numbers in one expression, division truncates toward zero, and `Math` functions don't accept them.

NOTES

- `typeof 10n` is `"bigint"`
- `10n == 10` is `true`; `10n === 10` is `false`
- `JSON.stringify` throws on BigInt by default
- No decimals: `5n / 2n` is `2n`

CREATING AND USING BIGINT

```
const big = 9007199254740993n;      // exact, beyond MAX_SAFE_INTEGER
const fromNum = BigInt(42);         // 42n
const fromStr = BigInt("12345678901234567890"); // works

2n ** 64n;                          // 18446744073709551616n
7n / 2n;                            // 3n  (truncates)
-7n / 2n;                           // -3n
7n % 2n;                            // 1n

// 1n + 1;                          // TypeError: Cannot mix BigInt
1n + BigInt(1);                     // 2n
Number(10n) + 1;                    // 11 (may lose precision)

10n > 5;                            // true (comparison is OK)
typeof 10n;                         // "bigint"
JSON.stringify({ id: 1n.toString() }); // '{"id":"1"}'
```

CODE EXPLAINED

- → `n` suffix — BigInt literal
- → `BigInt("...")` — parse huge integers from strings safely
- → Mixing types throws; convert one side explicitly
- → Comparisons across types are allowed

TYPESCRIPT

- Use the `bigint` type: `let id: bigint = 1n`
- BigInt literals need `"target": "ES2020"` or later in tsconfig

OPERATOR SUPPORT

OPERATOR	BIGINT?
`+ - * ** %`	yes
`/`	yes, truncates
`unary +`	no (TypeError)
`>>>`	no
`< > ==`	yes, with Number too

CONVERSIONS

EXPRESSION	RESULT
`BigInt(1.5)`	RangeError
`BigInt("0x1f")`	31n
`Number(2n**60n)`	1152921504606847000
`String(5n)`	"5"
`Boolean(0n)`	false

⚠ COMMON MISTAKES

- Passing a BigInt to `Math.max` or `Math.round`; they only accept Numbers
- Converting big values back with `Number()` and silently losing digits
- Sending BigInts to `JSON.stringify` without a replacer; it throws

✓ TIPS & BEST PRACTICES

- Store database IDs from 64-bit columns as BigInt or strings, never Number
- Use a JSON replacer: `(k, v) => typeof v === "bigint" ? v.toString() : v`
- Stick to Number for ordinary math; BigInt is slower

QUICK SUMMARY

- `10n` — BigInt literal
- `BigInt(x)` — convert
- no mixing — convert one side
- `/` truncates — integers only

Booleans & Truthiness

OVERVIEW

Every value in JavaScript is either **truthy** or **falsy** when used as a condition. There are exactly eight falsy values; everything else, including empty arrays, empty objects, and the string `"0"`, is truthy. Convert explicitly with `Boolean(x)` or the double-bang `!!x`.

NOTES

- Booleans are `true` and `false`, lowercase
- `if (x)` tests truthiness, not `x === true`
- `new Boolean(false)` is an object, so it's truthy
- `document.all` is the one weird falsy object (legacy)

TRUTHINESS IN PRACTICE

```
Boolean(0);          // false
Boolean("");         // false
Boolean("0");        // true  (non-empty string)
Boolean([]);         // true  (empty array is an object)
Boolean({});         // true
!!"hello";           // true
!!null;              // false

const items = [];
if (items) { }                // always runs: [] is truthy
if (items.length) { }         // correct emptiness check

const qty = 0;
const shown = qty || 1;       // 1  (0 treated as missing)
const kept = qty ?? 1;        // 0  (only null/undefined replaced)
```

CODE EXPLAINED

→ `!!x` — shorthand for `Boolean(x)`

→ Check arrays with `.length`, objects with `Object.keys(o).length`

→ `||` replaces every falsy value, including `0` and `""`

→ `??` replaces only `null` and `undefined`

TYPESCRIPT

- Use `boolean` (lowercase), never the `Boolean` wrapper type
- typescript-eslint's `strict-boolean-expressions` can ban relying on truthiness

ALL 8 FALSY VALUES

VALUE	TYPE
`false`	boolean
`0 / -0`	number
`0n`	bigint
`""`	string
`null`	null
`undefined`	undefined
`NaN`	number

SURPRISINGLY TRUTHY

VALUE	WHY
`"0"`	non-empty string
`"false"`	non-empty string
`[]`	object
`{}`	object
`-1`	non-zero number
`Infinity`	non-zero number

⚠ COMMON MISTAKES

- `if (arr)` to check for an empty array; it's always true
- `count || default` when `0` is a valid count; use `??`
- Comparing with `== true`; `"1" == true` is true but `"2" == true` is false

✓ TIPS & BEST PRACTICES

- Use `??` for defaults unless you truly want to replace `""` and `0`
- Name boolean variables as questions: `isReady`, `hasItems`
- Prefer explicit checks (`x !== undefined`) when the falsy cases matter

QUICK SUMMARY

- 8 falsy values — everything else is truthy
- `Boolean(x)` / `!!x` — convert
- `[]` and `{}` — truthy
- `??` vs `||` — null-only vs any falsy

null & undefined

OVERVIEW

JavaScript has two "nothing" values. `undefined` means a value was never assigned: missing properties, missing arguments, and functions with no `return`. `null` is an intentional "no value" that you assign yourself. They are loosely equal (`null == undefined`) but strictly different.

NOTES

- `typeof null` is `"object"`, a historical bug
- `typeof undefined` is `"undefined"`
- JSON has `null` but no `undefined`; those keys are dropped
- `void 0` always evaluates to `undefined`

WHERE EACH APPEARS

```
let a;                      // undefined (declared, not assigned)
const o = { x: 1 };
o.y;                        // undefined (missing property)
function f(p) { return p; }
f();                        // undefined (missing argument)
(() => {})();               // undefined (no return)

let user = null;            // intentional "no user yet"

null == undefined;          // true
null === undefined;         // false
typeof null;                // "object"

if (value == null) { }      // catches both null and undefined
value ?? "default";         // default only for null/undefined
user?.name;                 // undefined instead of TypeError
JSON.stringify({ a: undefined, b: null }); // '{"b":null}'
```

CODE EXPLAINED

- → `== null` — the one widely accepted use of loose equality
- → `??` — default only for nullish values
- → `?.` — stops and returns `undefined` on null/undefined
- → JSON drops `undefined` keys but keeps `null`

TYPESCRIPT

- `strictNullChecks` makes `null` / `undefined` separate types you must handle
- `x?: T` (may be missing) differs from `x: T | undefined` (must be present)

NULL VS UNDEFINED

	UNDEFINED	NULL
Meaning	not assigned	empty on purpose
typeof	"undefined"	"object"
Number()	NaN	0
JSON	dropped	kept
Default params	trigger default	don't

CHECKING

CHECK	CATCHES
`x === undefined`	undefined only
`x === null`	null only
`x == null`	both
`x ?? d`	both
`!x`	all falsy (too broad)

⚠ COMMON MISTAKES

- Using `typeof x === "object"` to detect objects; it's also true for `null`
- Passing `null` and expecting the default parameter to kick in; only `undefined` triggers it
- Assigning `undefined` on purpose; use `null` so intent is clear

✓ TIPS & BEST PRACTICES

- Pick one convention: `null` for "intentionally empty", `undefined` for "not set"
- Combine `?.` and `??`: `user?.name ?? "Guest"`
- Use `x == null` when either value means "missing"

QUICK SUMMARY

- `undefined` — never assigned
- `null` — deliberately empty
- `== null` — matches both
- `?.` / `??` — safe access and defaults

Symbols

OVERVIEW

A `Symbol` is a unique, immutable primitive, mostly used as an object key that can't collide with any other key. Every `Symbol()` call creates a new value, even with the same description. **Well-known symbols** like `Symbol.iterator` let you hook into language features such as `for...of`.

NOTES

- Symbol keys are skipped by `for...in`, `Object.keys`, and `JSON.stringify`
- `Symbol.for(key)` returns a shared, global symbol
- Symbols can't be implicitly converted to strings
- Not truly private: `Object.getOwnPropertySymbols` reveals them

CREATING AND USING SYMBOLS

```
const id = Symbol("id");
const id2 = Symbol("id");
id === id2;                          // false (always unique)
id.description;                      // "id"

const user = { name: "Ana", [id]: 123 };
user[id];                            // 123
Object.keys(user);                   // ["name"]  (symbol hidden)
JSON.stringify(user);                // '{"name":"Ana"}'
Object.getOwnPropertySymbols(user); // [Symbol(id)]

Symbol.for("app") === Symbol.for("app"); // true (global registry)
Symbol.keyFor(Symbol.for("app"));        // "app"

// `${id}`;                          // TypeError
String(id);                          // "Symbol(id)"
```

CODE EXPLAINED

- → `[id]: 123` — computed key syntax puts a symbol on an object
- → Symbol keys don't show in normal enumeration
- → `Symbol.for` — same key gives the same symbol across files
- → Use `String(sym)` or `.description` to print

TYPESCRIPT

- Type is `symbol`; a `const` declared with `Symbol()` gets `unique symbol`
- `unique symbol` types can be used as typed object keys

WELL-KNOWN SYMBOLS

SYMBOL	HOOKS INTO
`Symbol.iterator`	for…of, spread
`Symbol.asyncIterator`	for await…of
`Symbol.toPrimitive`	type conversion
`Symbol.hasInstance`	instanceof
`Symbol.toStringTag`	Object.prototype.toString
`Symbol.dispose`	using declarations

SYMBOL API

CALL	RETURNS
`Symbol(desc)`	new unique symbol
`Symbol.for(key)`	registry symbol
`Symbol.keyFor(s)`	registry key or undefined
`s.description`	description string
`typeof s`	"symbol"

⚠ COMMON MISTAKES

- Calling `new Symbol()`; it's a TypeError, symbols aren't constructed
- Treating symbol keys as truly private; reflection APIs can read them
- Expecting `Symbol("x") === Symbol("x")`; use `Symbol.for` for shared ones

✓ TIPS & BEST PRACTICES

- Use symbols for metadata keys on objects you don't own, to avoid collisions
- Use `#private` class fields for real privacy instead of symbols
- Implement `Symbol.iterator` to make your own objects work with `for...of` (sheet 061)

QUICK SUMMARY

- `Symbol()` — unique key
- `Symbol.for()` — shared key
- hidden from keys/JSON
- well-known symbols — language hooks

Sheet 013

Type Conversion & Coercion

String() · Number() · parseInt · parseFloat · unary + · implicit coercion · Boolean()

OVERVIEW

JavaScript converts types **implicitly** in many operators (+, ==, `if`), which is the source of most "wat" moments. Convert **explicitly** instead: `String()`, `Number()`, and `Boolean()` for general use, and `parseInt`/`parseFloat` when you need to read a number from the start of a string.

NOTES

- `+` with any string operand concatenates
- `-`, `*`, `/` always convert to Number
- `Number("")` is `0`, a common surprise
- `parseInt` stops at the first non-digit; `Number` rejects the whole string

EXPLICIT VS IMPLICIT CONVERSION

```
String(42);              // "42"
(42).toString(2);        // "101010" (base 2)
Number("42");            // 42
Number("42px");          // NaN
Number("");              // 0
Number(null);            // 0
Number(undefined);       // NaN
parseInt("42px", 10);    // 42
parseFloat("3.14abc");   // 3.14
+"7";                    // 7 (unary plus)

"5" + 3;                 // "53"  (concatenation)
"5" - 3;                 // 2     (numeric)
"5" * "2";               // 10
[] + {};                 // "[object Object]"
true + 1;                // 2
```

CODE EXPLAINED

- → `toString(radix)` — number to string in any base
- → `parseInt(s, 10)` — always pass the radix
- → Unary `+` — shortest conversion, but less readable
- → `+` prefers strings; other math operators prefer numbers

TYPESCRIPT

- `as number` only tells the compiler; it never converts at runtime
- TS flags many implicit coercions (`"5" - 3` is a type error)

TO NUMBER

INPUT	NUMBER()	PARSEINT()
`"12"`	12	12
`"12.9"`	12.9	12
`"12px"`	NaN	12
`""`	0	NaN
`" 7 "`	7	7
`true`	1	NaN

TO STRING

INPUT	STRING()
`123`	"123"
`null`	"null"
`[1, 2]`	"1,2"
`{}`	"[object Object]"
`Symbol("a")`	"Symbol(a)"

⚠ COMMON MISTAKES

- Reading form input and adding: `input.value + 1` gives `"51"`, not `6`
- Using `parseInt` without a radix on user input
- Using `parseInt(0.0000005)`, which returns `5` because the number stringifies as `5e-7`

✓ TIPS & BEST PRACTICES

- Validate after converting: `const n = Number(s); if (Number.isNaN(n)) …`
- Use `Number()` for strict parsing, `parseFloat` for "number with units"
- Use `===` everywhere to avoid comparison coercion

QUICK SUMMARY

- `String() / Number() / Boolean()` — explicit
- `parseInt(s, 10)` — leading integer
- `+` — concatenates with strings
- `Number("")` — 0, watch out

Type Checking

OVERVIEW

`typeof` returns a string naming a value's primitive type, but it can't tell objects apart (arrays, dates, and `null` all report `"object"`). Use `instanceof` for class instances, `Array.isArray` for arrays, and the `Number.is*` helpers for numeric checks. For an exact internal type tag, `Object.prototype.toString.call(x)` works on anything.

NOTES

- `typeof` never throws, even on undeclared variables
- `instanceof` checks the prototype chain
- `instanceof` fails across iframes/realms; `Array.isArray` doesn't
- Functions report `"function"`, even classes

TYPE CHECKS WITH RESULTS

```
typeof "hi";                 // "string"
typeof 42;                   // "number"
typeof 42n;                  // "bigint"
typeof true;                 // "boolean"
typeof undefined;            // "undefined"
typeof Symbol();             // "symbol"
typeof null;                 // "object"   (!)
typeof [];                   // "object"
typeof (() => {});           // "function"

Array.isArray([]);           // true
new Date() instanceof Date;  // true
[] instanceof Object;        // true
Number.isNaN(NaN);           // true
Number.isInteger(4.0);       // true
Object.prototype.toString.call([]);   // "[object Array]"
Object.prototype.toString.call(null); // "[object Null]"
```

CODE EXPLAINED

- → `typeof` — good for primitives and functions
- → `Array.isArray` — the only reliable array test
- → `instanceof` — true for any class in the chain
- → `toString.call` — precise tag for any value

TYPESCRIPT

- `typeof`, `instanceof`, `in`, and `Array.isArray` all **narrow** the type inside the `if`
- TS also has a type-level `typeof` that captures a variable's type (sheet 123)

TYPEOF RESULTS

VALUE	TYPEOF
`""`	"string"
`NaN`	"number"
`null`	"object"
`[] / {}`	"object"
`class A {}`	"function"
`undeclaredVar`	"undefined"

WHICH CHECK TO USE

TO DETECT	USE
`array`	Array.isArray(x)
`null`	x === null
`plain object`	x?.constructor === Object
`class instance`	x instanceof Cls
`real number`	Number.isFinite(x)

⚠ COMMON MISTAKES

- `typeof x === "object"` without excluding `null`
- `typeof arr === "array"`; that string is never returned
- Using global `isNaN("abc")` (true) when you meant `Number.isNaN` (false)

✓ TIPS & BEST PRACTICES

- Write a small `isObject = x => x !== null && typeof x === "object"` helper
- Prefer `Number.isFinite` to check "is a usable number"
- Use duck typing (`typeof x.then === "function"`) for protocol checks

QUICK SUMMARY

- `typeof` — primitives and functions
- `instanceof` — class instances
- `Array.isArray` — arrays
- `toString.call` — exact tag

User Input

OVERVIEW

Input comes from different places depending on the runtime. In the browser it's usually a form field or `prompt()`. In Node it's command-line arguments (`process.argv`) or interactive input through the `readline/promises` module. Input always arrives as a **string**, so convert and validate it before use.

NOTES

- `prompt()` blocks the page and returns `null` on Cancel
- `process.argv[0]` is the node binary, `[1]` the script
- Close the readline interface or the process won't exit
- `util.parseArgs` handles `--flags` without a library (sheet 110)

READING INPUT IN NODE

```
import * as readline from "node:readline/promises";
import { stdin as input, stdout as output } from "node:process";

const rl = readline.createInterface({ input, output });
const answer = await rl.question("Your age? ");   // "30"
rl.close();

const age = Number(answer.trim());
if (!Number.isInteger(age) || age < 0) {
  console.error("Please enter a whole number");
  process.exit(1);
}
console.log(`Next year you'll be ${age + 1}`);

// $ node greet.js Ana 3
const [, , name, times] = process.argv;  // "Ana", "3"
```

CODE EXPLAINED

- → `readline/promises` — `question()` returns a Promise
- → `.trim()` — remove the trailing newline and spaces
- → Validate with `Number.isInteger` before using the value
- → Destructure `process.argv`, skipping the first two entries

TYPESCRIPT

- Every input is typed `string`; parse it, then the result narrows to `number`
- `process.argv` and `readline` types come from `@types/node`

INPUT SOURCES

SOURCE	API	RETURNS
Browser dialog	prompt(msg)	string | null
Form field	input.value	string
CLI args	process.argv	string[]
Interactive	rl.question()	Promise<string>
Piped stdin	for await (line of rl)	string lines

VALIDATION PATTERNS

GOAL	CHECK
non-empty	s.trim() !== ""
integer	Number.isInteger(Number(s))
in range	n >= 1 && n <= 10
one of	["y","n"].includes(s)

⚠ COMMON MISTAKES

- Doing math on input without converting: `"2" + 1` is `"21"`
- Forgetting `rl.close()`, so the script hangs after the last question
- Ignoring `null` from `prompt()` when the user cancels

✓ TIPS & BEST PRACTICES

- Normalize input first: `.trim().toLowerCase()`
- Loop `rl.question` until the answer validates instead of exiting
- Read piped input line by line with `for await (const line of rl)`

QUICK SUMMARY

- `process.argv` — CLI arguments
- `readline/promises` — interactive prompts
- always a string — convert first
- validate — before using the value

String Basics

" · '' · \\ · length · immutability · escapes · concatenation · surrogate pairs

OVERVIEW

Strings are **immutable** sequences of UTF-16 code units. You can write them with single quotes, double quotes, or backticks (template literals, sheet 017). Every "modifying" method returns a new string. `length` counts UTF-16 units, so some emoji and rare characters count as 2.

NOTES

- Single and double quotes behave identically
- Strings are compared by value: `"a" === "a"`
- Index access `s[0]` is read-only; assignments are silently ignored (or throw in strict mode)
- `String.length` is a property, not a method

LITERALS, ESCAPES, LENGTH

```
const a = 'single';
const b = "double";
const c = `template ${a}`;          // "template single"

"It's".length;                      // 4
'Say "hi"';                         // quotes inside quotes
"Line 1\nLine 2";                   // newline escape
"Tab\there";                        // tab
"\u00e9";                           // "é"
"\u{1F600}";                        // "😀"

"😀".length;                        // 2  (surrogate pair)
[..."😀"].length;                   // 1  (code points)

let s = "cat";
s[0] = "b";                         // ignored; strings are immutable
s = "b" + s.slice(1);               // "bat"
"a" + "b" + 1 + 2;                  // "ab12"
```

CODE EXPLAINED

- → `\u{…}` — code point escape for any Unicode character
- → `[...str]` — spread iterates code points, not UTF-16 units
- → Build a new string to "change" one
- → `+` goes left to right: once a string appears, everything concatenates

TYPESCRIPT

- Type `string`; literal types like `"GET" | "POST"` restrict to exact values
- Assigning to `s[0]` is a compile error (index signature is readonly)

ESCAPE SEQUENCES

ESCAPE	MEANING
`\n`	newline
`\t`	tab
`\\`	backslash
`\' \"`	quote
`\uXXXX`	UTF-16 unit
`\u{X…}`	code point

BASIC OPERATIONS

EXPRESSION	RESULT
`"ab".length`	2
`"ab" + "cd"`	"abcd"
`"ab".concat("c")`	"abc"
`"a" < "b"`	true
`"B" < "a"`	true (code order)
`"ab" * 2`	NaN

⚠ COMMON MISTAKES

- Using `length` to count user-visible characters; use `[...s].length` or `Intl.Segmenter`
- Trying to change one character in place
- Concatenating in a loop to build huge strings; collect parts in an array and `join`

✓ TIPS & BEST PRACTICES

- Pick one quote style and let Prettier enforce it
- Use template literals whenever you embed values
- Use `localeCompare` for human sorting, not `<`

QUICK SUMMARY

- `''` `""` `` ` `` — three literal forms
- immutable — methods return new strings
- `length` — UTF-16 units
- `\u{…}` — any Unicode character

Template Literals

OVERVIEW

Template literals use backticks and support `${expression}` interpolation, real line breaks, and **tagged templates**, where a function receives the string parts and values separately. Any expression works inside `${}`: math, function calls, ternaries, even other template literals.

NOTES

- Values inside `${}` are converted with `String()`
- Whitespace and newlines inside backticks are kept exactly
- `String.raw` keeps backslashes as written
- Tags power libraries like `styled-components` and `lit-html`

INTERPOLATION AND TAGS

```
const name = "Ana", items = 3, price = 4.5;

`Hi ${name}!`;                          // "Hi Ana!"
`Total: $${(items * price).toFixed(2)}`; // "Total: $13.50"
`${items > 1 ? "items" : "item"}`;      // "items"
`Upper: ${name.toUpperCase()}`;         // "Upper: ANA"

const html = `
<ul>
  ${["a", "b"].map(x => `<li>${x}</li>`).join("")}
</ul>`;                                  // nested templates

String.raw`C:\new\table`;               // "C:\new\table" (raw)

function tag(strings, ...values) {
  return strings.raw.join("|") + " :: " + values.join(",");
}
tag`a${1}b${2}c`;                        // "a|b|c :: 1,2"
```

CODE EXPLAINED

→ `$${…}` — a literal `$` followed by an interpolation

→ `map(...).join("")` — common pattern for lists in HTML

→ Tag receives `strings` (length = values + 1) and the values

→ `strings.raw` — the uninterpreted text

TYPESCRIPT

- Template literal **types** use the same syntax at type level: type Id = `user-${number}`
- Tag functions can be typed: `(s: TemplateStringsArray, ...v: unknown[])`

TEMPLATE VS CONCATENATION

CONCATENATION	TEMPLATE
`"Hi " + name`	`` `Hi ${name}` ``
`a + " " + b`	`` `${a} ${b}` ``
`"l1\n" + "l2"`	`` `l1↵l2` (real newline) ``
`"$" + n.toFixed(2)`	`` `$${n.toFixed(2)}` ``

INTERPOLATED VALUES

VALUE	BECOMES
`null`	"null"
`undefined`	"undefined"
`[1, 2]`	"1,2"
`{a: 1}`	"[object Object]"
`Symbol()`	TypeError

⚠ COMMON MISTAKES

- Using quotes instead of backticks, so `${x}` prints literally
- Injecting user input into HTML with templates; that's an XSS hole (use `textContent` or escape)
- Accidental indentation inside multi-line templates

✓ TIPS & BEST PRACTICES

- Use `JSON.stringify(obj)` inside `${}` to print objects
- Keep complex logic out of `${}`; compute it in a variable first
- Use `String.raw` for Windows paths and regex sources

QUICK SUMMARY

- `${x}` — interpolation
- multi-line — newlines kept
- tagged templates — custom processing
- `String.raw` — no escape handling

String Methods — Search

OVERVIEW

Search methods tell you **whether** a substring exists and **where**. `includes`, `startsWith`, and `endsWith` return booleans; `indexOf` and `lastIndexOf` return a position or −1. For patterns, `search` returns a position and `match` returns match details (regex details on sheet 087). All are case-sensitive.

NOTES

- Positions are zero-based
- `-1` means not found, and it's truthy, so don't use `if (s.indexOf(x))`
- `startsWith` / `endsWith` / `includes` throw if given a regex
- Lowercase both sides for case-insensitive checks

SEARCH METHODS WITH RESULTS

```
const s = "The quick brown fox";

s.includes("quick");        // true
s.includes("Quick");        // false (case-sensitive)
s.startsWith("The");        // true
s.startsWith("quick", 4);   // true  (from index 4)
s.endsWith("fox");          // true
s.endsWith("quick", 9);     // true  (as if length were 9)

s.indexOf("o");             // 12
s.indexOf("o", 13);         // 17   (search from 13)
s.lastIndexOf("o");         // 17
s.indexOf("cat");           // -1

s.search(/[A-Z]/);          // 0
s.match(/qu(\w+)/);         // ["quick", "ick", index: 4, ...]
s.toLowerCase().includes("the quick"); // true
```

CODE EXPLAINED

- → Second argument to `indexOf` / `includes` — start position
- → `endsWith(x, n)` — treats the string as only `n` chars long
- → `search` — like `indexOf` but takes a regex
- → `match` — returns capture groups, or `null`

TYPESCRIPT

- `indexOf` returns `number`; `match` returns `RegExpMatchArray | null`, so check for null first
- Otherwise identical in TypeScript

SEARCH METHODS

METHOD	RETURNS	REGEX?
`includes(sub, from)`	boolean	no
`startsWith(sub, pos)`	boolean	no
`endsWith(sub, len)`	boolean	no
`indexOf(sub, from)`	index | -1	no
`lastIndexOf(sub, from)`	index | -1	no
`search(re)`	index | -1	yes
`match(re)`	array | null	yes

COMMON CHECKS

GOAL	CODE
contains	`s.includes(x)`
case-insensitive	`s.toLowerCase().includes(x)`
count occurrences	`s.split(x).length - 1`
all positions	`[...s.matchAll(/o/g)]`
file type	`name.endsWith(".png")`

⚠ COMMON MISTAKES

- `if (s.indexOf(x))` fails when the match is at position 0; use `includes` or `!== -1`
- Calling `.length` on the result of `match` without checking for `null`
- Passing a regex to `includes`; it throws a TypeError

✓ TIPS & BEST PRACTICES

- Use `includes` for yes/no questions; `indexOf` only when you need the position
- Use `localeCompare(x, undefined, { sensitivity: "base" }) === 0` for accent-insensitive equality
- Use `matchAll` to iterate every regex match with groups

QUICK SUMMARY

- `includes` — contains?
- `startsWith / endsWith` — prefix / suffix
- `indexOf` — position or -1
- `match / search` — regex searches

String Methods — Transform

OVERVIEW

Transform methods return a **new** string and leave the original unchanged. The workhorses are `replace`/`replaceAll`, `split` and its partner `Array.join`, `trim` for whitespace, and case conversion. `padStart`/`padEnd` are handy for aligned output and zero-padded numbers.

NOTES

- `replace` with a string replaces only the first match
- `replaceAll` with a regex requires the `g` flag
- `split("")` splits into UTF-16 units; use `[...s]` for characters
- `normalize()` fixes accented characters that look equal but aren't

TRANSFORM METHODS WITH RESULTS

```
const s = "  Hello, World  ";

s.trim();                           // "Hello, World"
s.trimStart();                      // "Hello, World  "
"a-b-c".replace("-", "+");          // "a+b-c"  (first only)
"a-b-c".replaceAll("-", "+");       // "a+b+c"
"a1b2".replace(/\d/g, "#");         // "a#b#"
"john".replace(/^./, c => c.toUpperCase()); // "John"

"a,b,c".split(",");                 // ["a", "b", "c"]
"a,b,c".split(",", 2);              // ["a", "b"]
["a", "b"].join(" & ");             // "a & b"

"Hi".toUpperCase();                 // "HI"
"5".padStart(3, "0");               // "005"
"ab".padEnd(5, ".");                // "ab..."
"ha".repeat(3);                     // "hahaha"
"é" === "e\u0301".normalize();      // true
```

CODE EXPLAINED

→ Replacement function — receives the match and returns the new text

→ `split(sep, limit)` — limit caps the number of pieces

→ `padStart(len, fill)` — pads until the total length is `len`

→ `normalize()` — converts to a canonical Unicode form (NFC)

TYPESCRIPT

- All return `string`; `split` returns `string[]`
- Identical behavior in TypeScript

TRANSFORM METHODS

METHOD	EXAMPLE → RESULT
`replace(a, b)`	"aa".replace("a","b") → "ba"
`replaceAll(a, b)`	"aa".replaceAll("a","b") → "bb"
`split(sep)`	"a b".split(" ") → ["a","b"]
`trim()`	" x ".trim() → "x"
`toLowerCase()`	"AB" → "ab"
`padStart(n, c)`	"7".padStart(2,"0") → "07"
`repeat(n)`	"-".repeat(3) → "---"

RECIPES

GOAL	CODE
capitalize	`s[0].toUpperCase() + s.slice(1)`
reverse	`[...s].reverse().join("")`
collapse spaces	`s.replace(/\s+/g, " ")`
slugify	`s.toLowerCase().replaceAll(" ", "-")`
words	`s.trim().split(/\s+/)`

⚠ COMMON MISTAKES

- Expecting `replace("x", "y")` to replace every `x`; use `replaceAll`
- Calling `s.trim()` without assigning the result; strings don't change in place
- Using `toUpperCase` for locale-specific text (Turkish i); use `toLocaleUpperCase("tr")`

✓ TIPS & BEST PRACTICES

- Chain methods: `s.trim().toLowerCase().split(/\s+/)`
- Use `$1`, `$2` in replacement strings to insert capture groups
- Use `Intl.Segmenter` to split into user-perceived characters or words

QUICK SUMMARY

- `replace / replaceAll` — substitute
- `split / join` — string ↔ array
- `trim` — strip whitespace
- `padStart` — fixed-width output

Slicing & Indexing

OVERVIEW

Extract parts of a string with `slice(start, end)`, which takes negative indexes from the end. `at(i)` reads one character and also accepts negatives. `substring` is an older alternative that swaps reversed arguments and treats negatives as 0. The end index is always **exclusive**.

NOTES

- `s[i]` returns `undefined` out of range; `charAt` returns `""`
- `at(-1)` is the last character
- `substr` is deprecated; don't use it
- All indexing is by UTF-16 unit, not by visible character

SLICING AND INDEXING

```
const s = "JavaScript";
//        0123456789

s[0];                 // "J"
s.at(-1);             // "t"
s.charAt(4);          // "S"
s[20];                // undefined
s.charAt(20);         // ""

s.slice(0, 4);        // "Java"
s.slice(4);           // "Script"
s.slice(-6);          // "Script"
s.slice(-6, -3);      // "Scr"
s.substring(4, 0);    // "Java" (args swapped)
s.slice(4, 0);        // ""     (no swap)

s.codePointAt(0);     // 74
for (const ch of "hé😀") console.log(ch);  // h, é, 😀
```

CODE EXPLAINED

- → `slice(start, end)` — end not included
- → Negative values count back from the end
- → `substring` swaps args when start > end; `slice` returns `""`
- → `for...of` iterates full code points, including emoji

TYPESCRIPT

- `s[i]` is typed `string`, even out of range; enable `noUncheckedIndexedAccess` to get `string | undefined`
- `at()` already returns `string | undefined`

SLICE PATTERNS (S = "ABCDEF")

CODE	RESULT
`s.slice(0, 3)`	"abc" (first 3)
`s.slice(-3)`	"def" (last 3)
`s.slice(1, -1)`	"bcde" (trim ends)
`s.slice(2)`	"cdef" (drop 2)
`s.slice(0, -2)`	"abcd" (drop last 2)
`s.slice()`	"abcdef" (copy)

SLICE VS SUBSTRING VS AT

	SLICE	SUBSTRING	AT
Negatives	from end	→ 0	from end
start > end	""	swaps	n/a
Returns	string	string	char | undefined

⚠ COMMON MISTAKES

- Expecting the end index to be included; `slice(0, 3)` returns 3 characters
- Using `s[s.length - 1]` everywhere instead of the clearer `s.at(-1)`
- Slicing strings with emoji by index and cutting a surrogate pair in half

✓ TIPS & BEST PRACTICES

- Default to `slice`; you rarely need `substring`
- Use `Array.from(s)` or `[...s]` before slicing user text with emoji
- Think of indexes as positions *between* characters to reason about ranges

QUICK SUMMARY

- `slice(a, b)` — substring, end exclusive
- negative index — from the end
- `at(i)` — one character
- `for...of` — iterate characters safely

Number & Locale Formatting

OVERVIEW

For quick formatting, `toFixed` sets decimal places and `toPrecision` sets significant digits. For anything shown to users, use `toLocaleString` or `Intl.NumberFormat`, which handle thousands separators, currency, percentages, units, and compact notation for any locale.

NOTES

- `toFixed` returns a string, not a number
- `toFixed` uses binary floats: `(1.005).toFixed(2)` is `"1.00"`
- Create one `Intl.NumberFormat` and reuse it; it's faster than `toLocaleString` in loops
- Locale defaults to the user's browser or system locale

FORMATTING NUMBERS

```
(3.14159).toFixed(2);              // "3.14"
(1234.5).toFixed(0);               // "1235"
(0.000123).toPrecision(2);         // "0.00012"
(1234567.891).toLocaleString();    // "1,234,567.891" (en-US)
(1234567.891).toLocaleString("de-DE"); // "1.234.567,891"

const usd = new Intl.NumberFormat("en-US",
  { style: "currency", currency: "USD" });
usd.format(1234.5);                // "$1,234.50"

new Intl.NumberFormat("en", { style: "percent" }).format(0.256);  // "26%"
new Intl.NumberFormat("en", { notation: "compact" }).format(15300); // "15K"
new Intl.NumberFormat("en", { style: "unit", unit: "kilometer-per-hour" })
  .format(50);                     // "50 km/h"
new Intl.NumberFormat("en", { maximumFractionDigits: 1 }).format(2.47); // "2.5"
```

CODE EXPLAINED

- → `style: "currency"` — needs a `currency` code
- → `notation: "compact"` — 15K, 1.2M
- → `unit` style — localized unit names
- → `minimumFractionDigits` / `maximumFractionDigits` — control decimals

TYPESCRIPT

- Option objects are typed, so a typo like `style: "curency"` is a compile error
- Returns `string`; convert back with `Number()` only if you must

INTL.NUMBERFORMAT OPTIONS

OPTION	VALUES
`style`	decimal · currency · percent · unit
`currency`	"USD" · "EUR" · "JPY"
`notation`	standard · compact · scientific
`minimumFractionDigits`	0–100
`maximumFractionDigits`	0–100
`useGrouping`	true · false
`signDisplay`	auto · always · never

QUICK RESULTS

CODE	OUTPUT
`(5).toFixed(2)`	"5.00"
`(1e6).toLocaleString("en-IN")`	"10,00,000"
`(0.5).toLocaleString("fr")`	"0,5"
`(1234).toLocaleString("ja")`	"1,234"

⚠ COMMON MISTAKES

- Doing math on the result of `toFixed`; it's a string, so `+` concatenates
- Building currency strings by hand (`"$" + n`), which breaks for other locales and negatives
- Assuming `toFixed` rounds half-up; floating-point representation can round down

✓ TIPS & BEST PRACTICES

- Keep numbers as numbers until the moment you display them
- Use `Intl.NumberFormat(...).formatToParts()` to style pieces (e.g., a smaller currency symbol)
- Pass the user's locale explicitly on servers, where the default may be wrong

QUICK SUMMARY

- `toFixed(n)` — n decimals, string
- `toLocaleString()` — locale-aware
- `Intl.NumberFormat` — currency, percent, units
- `notation: "compact"` — 15K style

Arithmetic Operators

OVERVIEW

Arithmetic operators work on Numbers (and BigInts). + is special: if either side is a string it concatenates instead of adding. % is a **remainder**, not modulo, so its result takes the sign of the left operand. ** is exponentiation and is right-associative.

NOTES

- `++x` increments then returns; `x++` returns then increments
- `-x 2` **is a SyntaxError; write** `(-x) 2`
- Precedence: `**` › `* / %` › `+ -`
- Division always returns a float: `7 / 2` is `3.5`

ARITHMETIC WITH RESULTS

```
7 + 2;         // 9
7 - 2;         // 5
7 * 2;         // 14
7 / 2;         // 3.5
7 % 2;         // 1
-7 % 2;        // -1   (sign of left side)
2 ** 10;       // 1024
2 ** 3 ** 2;   // 512  (right-assoc: 2 ** 9)

Math.trunc(7 / 2);          // 3  (integer division)
((-7 % 3) + 3) % 3;         // 2  (true modulo)

let i = 5;
i++;           // returns 5, i is now 6
++i;           // returns 7, i is now 7
"3" + 4;       // "34"
"3" * 4;       // 12
2 + 3 + "1";   // "51"
```

CODE EXPLAINED

- → `Math.trunc(a / b)` — integer division toward zero
- → `((a % n) + n) % n` — modulo that's never negative
- → Prefix vs postfix — only matters when you use the returned value
- → `+` evaluates left to right, switching to concatenation at the first string

TYPESCRIPT

- Arithmetic on strings or objects is a compile error (`"3" * 4` won't compile)
- `+` with a string is still allowed and typed as `string`

OPERATORS

OP	NAME	EXAMPLE	RESULT
+	addition	5 + 2	7
-	subtraction	5 - 2	3
*	multiplication	5 * 2	10
/	division	5 / 2	2.5
%	remainder	5 % 2	1
**	exponent	5 ** 2	25
++ / --	inc / dec	x++	x + 1

UNARY + AND -

EXPRESSION	RESULT
+"42"	42
+""	0
+true	1
-"5"	-5
+[]	0

⚠ COMMON MISTAKES

- Using `%` as modulo with negative numbers and getting negative results
- Adding numbers read from inputs without converting them first
- Writing `i++` inside complex expressions, which makes the order hard to follow

✓ TIPS & BEST PRACTICES

- Use `+= 1` instead of `++` if your lint style prefers clarity
- Use `Math.round(x * 100) / 100` for 2-decimal rounding in calculations
- Add parentheses whenever precedence isn't obvious

QUICK SUMMARY

- `+ - * /` — basic math, `/` is float
- `%` — remainder, sign of left side
- `**` — exponent, right-associative
- `+` with a string — concatenation

Comparison Operators

OVERVIEW

Use **strict equality** === and !==: they compare value and type with no conversion. Loose equality == converts types first and has rules few people remember. Relational operators (<, >) compare strings character by character by code unit, and objects are equal only if they're the same reference.

NOTES

- `NaN` isn't equal to anything, including itself
- `Object.is` is like `===` but treats `NaN` as equal and `+0` / `-0` as different
- Arrays and objects compare by reference, never by contents
- `"10" < "9"` is `true` (string comparison)

COMPARISONS WITH RESULTS

```
5 === 5;              // true
5 === "5";            // false
5 == "5";             // true   (coerces)
0 == "";              // true
null == undefined;    // true
null == 0;            // false
NaN === NaN;          // false
Object.is(NaN, NaN); // true
Object.is(0, -0);     // false

[1] === [1];          // false  (different references)
const a = [1]; a === a;  // true

"apple" < "banana";   // true
"Z" < "a";            // true   (uppercase sorts first)
"10" < "9";           // true   (string compare)
10 < 9;               // false
"10" < 9;             // false  (string converted to number)
```

CODE EXPLAINED

- → `===` — no type conversion, the safe default
- → `==` — converts operands; `null == undefined` is the one useful case
- → Two strings compare lexically; a string vs number compares numerically
- → Compare arrays by contents with a loop or `JSON.stringify` for simple data

TYPESCRIPT

- Comparing types with no overlap (`x === "a"` when `x: number`) is a compile error
- Comparisons narrow: after `if (x === null) return`, `x` is non-null

OPERATORS

OP	MEANING	EXAMPLE
`===`	strict equal	1 === 1 → true
`!==`	strict not equal	1 !== "1" → true
`==`	loose equal	1 == "1" → true
`!=`	loose not equal	1 != "1" → false
`< > <= >=`	relational	2 >= 2 → true

LOOSE EQUALITY SURPRISES

EXPRESSION	RESULT
`"" == 0`	true
`"0" == false`	true
`[] == false`	true
`[] == ![]`	true
`null == false`	false
`NaN == NaN`	false

⚠ COMMON MISTAKES

- Using `==` and getting coerced matches like `"" == 0`
- Comparing objects or arrays with `===` and expecting content equality
- Sorting or comparing numeric strings without converting them

✓ TIPS & BEST PRACTICES

- Turn on ESLint's `eqeqeq` rule (allowing `== null` if you like)
- Use `Number.isNaN` instead of comparing to `NaN`
- Use `a.localeCompare(b)` for natural, locale-aware string ordering

QUICK SUMMARY

- `===` — strict, use by default
- `==` — coerces, avoid
- objects — compared by reference
- `Object.is` — handles NaN and -0

Logical Operators

OVERVIEW

&&, ||, and ! combine conditions, but && and || don't return booleans. They return one of their **operands**: || returns the first truthy value (or the last), && returns the first falsy value (or the last). They also **short-circuit**, so the right side runs only when needed.

NOTES

- `!` always returns a boolean
- Precedence: `!` › `&&` › `||`
- Mixing `??` with `&&` / `||` requires parentheses
- Short-circuiting makes `a && a.b` a safe guard (though `?.` is cleaner)

OPERAND VALUES AND SHORT-CIRCUITING

```
true && "yes";          // "yes"
0 && "yes";             // 0      (first falsy)
"a" && "b" && "c";      // "c"    (all truthy → last)

"" || "default";        // "default"
"hi" || "default";      // "hi"   (first truthy)
null || 0 || "";        // ""     (none truthy → last)

!true;                  // false
!!"text";               // true

isLoggedIn && showDashboard();   // calls only if logged in
const port = env.PORT || 3000;   // fallback (but 0 → 3000)
const ok = (a > 0 && b > 0) || override;

// a ?? b || c;                  // SyntaxError
(a ?? b) || c;                   // OK with parentheses
```

CODE EXPLAINED

- → `&&` — stops at the first falsy operand
- → `||` — stops at the first truthy operand
- → `cond && fn()` — run a function conditionally
- → Use `??` instead of `||` when `0` or `""` are valid values

TYPESCRIPT

- `&&` and `||` narrow types: inside `if (user && user.name)`, both are known to exist
- The result type is a union, e.g. `0 | "yes"`

TRUTH TABLE

A	B	A && B	A || B
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

WHAT GETS RETURNED

EXPRESSION	RESULT
`"x" && 0`	0
`0 || null`	null
`undefined && x`	undefined
`[] || "fallback"`	[]
`!""`	true

⚠ COMMON MISTAKES

- Expecting `a || b` to return `true` / `false`; it returns one of the values
- Using `||` for defaults when `0`, `false`, or `""` are legitimate
- Writing long `&&` chains as statements; an `if` is often clearer

✓ TIPS & BEST PRACTICES

- Use `!!x` or `Boolean(x)` when you need an actual boolean
- Put the cheapest or most likely-to-fail check first in `&&` chains
- Use parentheses when mixing `&&` and `||` even if precedence is right

QUICK SUMMARY

- `&&` — first falsy or last value
- `||` — first truthy or last value
- `!` — always boolean
- short-circuit — right side may not run

Nullish Coalescing & Optional Chaining

OVERVIEW

Nullish coalescing `??` returns its right side only when the left is `null` or `undefined`, unlike `||`, which also replaces `0`, `""`, and `false`. **Optional chaining** `?.` stops evaluation and returns `undefined` when the value before it is nullish, instead of throwing a TypeError.

NOTES

- `?.` works for properties `a?.b`, brackets `a?.[k]`, and calls `a.f?.()`
- Short-circuit covers the whole chain: `a?.b.c` is safe if `a` is nullish
- You can't assign through `?.`: `a?.b = 1` is a SyntaxError
- `??` can't mix with `&&` / `||` without parentheses

?? AND ?. IN ACTION

```
0 ?? 10;                // 0      (0 is not nullish)
0 || 10;                // 10
"" ?? "n/a";            // ""
null ?? "n/a";          // "n/a"
undefined ?? false;     // false

const user = { profile: null, tags: ["a"] };
user.profile?.name;           // undefined (no TypeError)
user.settings?.theme;         // undefined
user.tags?.[0];               // "a"
user.greet?.();               // undefined (method missing)
user.profile?.name ?? "Guest"; // "Guest"

const limit = opts.limit ?? 25;      // keeps 0 if passed
const len = list?.length ?? 0;
```

CODE EXPLAINED

- → `??` — default only for missing values
- → `?.[]` — optional bracket access
- → `?.()` — call only if the function exists
- → Combine: `a?.b ?? fallback` is the common pattern

TYPESCRIPT

- The **non-null assertion** `x!` tells TS a value isn't null without a runtime check; prefer `?.` or a real check
- TS knows the result of `a?.b` includes `undefined`

?? VS ||

LEFT VALUE	?? "D"	|| "D"
`null`	"d"	"d"
`undefined`	"d"	"d"
`0`	0	"d"
`""`	""	"d"
`false`	false	"d"
`NaN`	NaN	"d"

OPTIONAL CHAINING FORMS

FORM	USE
`obj?.prop`	property
`obj?.[key]`	computed key
`arr?.[0]`	array element
`fn?.()`	call if defined
`obj.method?.()`	optional method

⚠ COMMON MISTAKES

- Sprinkling `?.` everywhere, which hides bugs where a value should always exist
- Using `||` for numeric settings so a deliberate `0` gets overridden
- Writing `a?.b = 1`; optional chaining can't be assigned to

✓ TIPS & BEST PRACTICES

- Use `?.` only where nullish is a valid, expected state
- Use `??=` to set a default in place: `opts.retries ??= 3`
- Read `a ?? b` as "a, unless it's missing"

QUICK SUMMARY

- `??` — default for null/undefined only
- `?.` — safe property access
- `?.()` — safe call
- `a?.b ?? d` — access with fallback

Sheet 026

Assignment & Compound Assignment

= · += -= *= **= · ??= · ||= · &&= · chained assignment

OVERVIEW

= assigns and also **returns** the assigned value, which is why chains like `a = b = 0` work. Compound operators (+=, *=, …) combine an operation with assignment. The **logical assignment** operators &&=, ||=, and ??= assign only when the condition holds, and they short-circuit.

NOTES

- Assignment is right-associative: `a = b = 5` assigns `b` first
- `x ??= y` only evaluates `y` if `x` is nullish
- Compound assignment on `const` throws a TypeError
- Destructuring assignment unpacks values (sheet 056)

ASSIGNMENT OPERATORS

```
let x = 10;
x += 5;         // 15
x -= 3;         // 12
x *= 2;         // 24
x /= 4;         // 6
x %= 4;         // 2
x **= 3;        // 8

let s = "Hi";
s += "!";       // "Hi!"

let a, b;
a = b = 0;      // both 0

const opts = { retries: 0, name: "" };
opts.retries ??= 3;    // stays 0 (not nullish)
opts.name ||= "anon";  // "anon" ("" is falsy)
opts.timeout ??= 500;  // 500 (was undefined)
opts.debug &&= false;  // stays undefined (falsy, no assign)

[a, b] = [b, a];       // swap
```

CODE EXPLAINED

- → `x op= y` — same as `x = x op y`
- → `??=` — assign only if null/undefined
- → `||=` — assign if falsy
- → `&&=` — assign only if truthy

TYPESCRIPT

- Identical in TypeScript; compound assignments are type-checked (`s -= 1` on a string is an error)
- After `x ??= 5`, TS knows `x` is not nullish

COMPOUND OPERATORS

OP	EQUIVALENT
`x += y`	x = x + y
`x -= y`	x = x - y
`x *= y`	x = x * y
`x /= y`	x = x / y
`x %= y`	x = x % y
`x **= y`	x = x ** y
`x <<= y`	x = x << y

LOGICAL ASSIGNMENT

OP	ASSIGNS WHEN	EQUIVALENT
`x ??= y`	x is nullish	x ?? (x = y)
`x ||= y`	x is falsy	x || (x = y)
`x &&= y`	x is truthy	x && (x = y)

⚠ COMMON MISTAKES

- Writing `if (x = 5)` instead of `===`; it assigns and is always truthy
- Using `||=` for numbers where `0` is valid; use `??=`
- Chaining `let a = b = 0`, which creates `b` as an implicit global in sloppy mode

✓ TIPS & BEST PRACTICES

- Use `??=` to lazily initialize: `cache[key] ??= compute(key)`
- Destructuring swap `[a, b] = [b, a]` avoids a temp variable
- Enable ESLint `no-cond-assign` to catch accidental `=` in conditions

QUICK SUMMARY

- `=` — assigns and returns the value
- `+= -= *= …` — operate and assign
- `??=` — default if missing
- `||= / &&=` — conditional assignment

Bitwise Operators

OVERVIEW

Bitwise operators work on the binary representation of numbers. JavaScript converts operands to **32-bit signed integers** first, applies the operation, and converts back. They're used for flags and masks, color manipulation, hashing, and quick integer truncation.

NOTES

- `>>` keeps the sign; `>>>` fills with zeros (unsigned)
- `~x` equals `-(x + 1)`
- Values above $2^{31}-1$ wrap around
- BigInt supports all except `>>>`

BITS IN ACTION

```
//      5 = 0101     3 = 0011
5 & 3;      // 1  (0001)  AND
5 | 3;      // 7  (0111)  OR
5 ^ 3;      // 6  (0110)  XOR
~5;         // -6         NOT
5 << 1;     // 10 (1010)  shift left = ×2
5 >> 1;     // 2  (0010)  shift right = ÷2
-8 >> 1;    // -4         sign kept
-8 >>> 28;  // 15         zero-filled

const READ = 1, WRITE = 2, EXEC = 4;    // flags
let perm = READ | WRITE;                // 3
(perm & WRITE) !== 0;                   // true  (has WRITE)
perm &= ~WRITE;                         // 1  (remove WRITE)
perm ^= EXEC;                           // 5  (toggle EXEC)

(0xff8800 >> 8) & 0xff;                 // 136 (green channel)
(13).toString(2).padStart(8, "0");      // "00001101"
```

CODE EXPLAINED

→ Flags — each power of 2 is one bit

→ `perm & FLAG` — test a bit

→ `perm &= ~FLAG` — clear a bit; `^=` toggles

→ `toString(2)` — view the bits

TYPESCRIPT

- Identical in TypeScript
- Numeric `enum` flags (`enum Perm { Read = 1, Write = 2 }`) pair well with bitwise ops

OPERATORS

OP	NAME	EXAMPLE	RESULT
`&`	AND	6 & 3	2
`|`	OR	6 | 3	7
`^`	XOR	6 ^ 3	5
`~`	NOT	~6	-7
`<<`	left shift	6 << 2	24
`>>`	signed right	-6 >> 1	-3
`>>>`	unsigned right	-1 >>> 0	4294967295

IDIOMS

CODE	DOES
`x | 0`	truncate to 32-bit int
`n & 1`	1 if odd
`n & (n - 1)`	0 if power of 2
`1 << k`	2 to the k
`x >>> 0`	to unsigned 32-bit

⚠ COMMON MISTAKES

- Using `x | 0` on numbers above 2^{31}; they overflow to negative
- Confusing `&` with `&&` in conditions; `&` evaluates both sides and returns a number
- Forgetting parentheses: `perm & WRITE !== 0` parses as `perm & (WRITE !== 0)`

✓ TIPS & BEST PRACTICES

- Use `Math.trunc` instead of `| 0` for readability unless performance matters
- Define flags with shifts for clarity: `1 << 0`, `1 << 1`, `1 << 2`
- Use BigInt for masks wider than 32 bits

QUICK SUMMARY

- `& | ^ ~` — bit logic
- `<< >>` — shift, ×2 / ÷2
- `>>>` — unsigned shift
- 32-bit — operands are converted first

Spread & Rest

… in arrays · objects · calls · rest params · rest in destructuring

OVERVIEW

The `...` syntax has two jobs. As **spread**, it expands an iterable or object into individual elements: in array literals, object literals, and function calls. As **rest**, it gathers remaining items into an array or object: in parameters and destructuring patterns. Same syntax; position decides which.

NOTES

- Spread makes **shallow** copies; nested objects are shared
- Object spread copies own enumerable properties; later keys win
- Rest must be the last element in a pattern
- Strings, Sets, and Maps are iterable, so they spread into arrays

SPREAD AND REST

```
const a = [1, 2], b = [3, 4];
[...a, ...b];                    // [1, 2, 3, 4]
[..."hi"];                       // ["h", "i"]
[...new Set([1, 1, 2])];         // [1, 2]
Math.max(...a);                  // 2

const base = { x: 1, y: 2 };
({ ...base, y: 9 });             // { x: 1, y: 9 }
({ y: 9, ...base });             // { y: 2, x: 1 } (base wins)

function sum(...nums) {          // rest parameter
  return nums.reduce((t, n) => t + n, 0);
}
sum(1, 2, 3);                    // 6

const [first, ...others] = [1, 2, 3];  // 1, [2, 3]
const { id, ...rest } = { id: 7, n: "a", t: 1 }; // 7, { n, t }
```

CODE EXPLAINED

- → Spread in `[]` — copy or concatenate arrays
- → Spread in `{}` — copy or merge objects; order sets priority
- → Rest in params — collects extra arguments as a real array
- → Rest in destructuring — "everything else"

TYPESCRIPT

- Spreading objects merges their types: `{ ...a, ...b }` is typed as the combination
- Rest parameters can be typed as tuples: `(...args: [string, number])`

SPREAD VS REST

CONTEXT	ROLE	EXAMPLE
array literal	spread	`[...a, 4]`
object literal	spread	`{ ...o, k: 1 }`
function call	spread	`f(...args)`
parameter list	rest	`function f(...xs)`
array pattern	rest	`const [h, ...t] = arr`
object pattern	rest	`const { a, ...r } = o`

RECIPES

GOAL	CODE
copy array	`[...arr]`
dedupe	`[...new Set(arr)]`
merge objects	`{ ...defaults, ...opts }`
omit a key	`const { pw, ...safe } = user`
add item immutably	`[...list, item]`

⚠ COMMON MISTAKES

- Treating a spread copy as a deep copy; nested objects are still shared
- Putting defaults after options: `{ ...opts, ...defaults }` overwrites the user's values
- Spreading huge arrays into function calls; engines limit argument counts

✓ TIPS & BEST PRACTICES

- Use `structuredClone` when you need a deep copy (sheet 057)
- Use rest destructuring to remove a property without mutating
- Prefer rest parameters over the old `arguments` object

QUICK SUMMARY

- spread — expand items
- rest — collect items
- shallow — nested values shared
- order matters — later keys win

Other Operators & Precedence

OVERVIEW

A few operators don't fit elsewhere: `typeof` and `instanceof` check types, `in` checks whether a property exists, `delete` removes a property, `void` evaluates an expression and returns `undefined`, and the comma operator evaluates several expressions and returns the last. The precedence table explains why some expressions parse in surprising ways.

NOTES

- `in` checks the prototype chain; `Object.hasOwn` checks own props only
- `delete` returns `true` even if the property didn't exist
- `delete` on an array leaves a hole; use `splice`
- The comma operator is mostly seen in `for` loop headers

MISCELLANEOUS OPERATORS

```
const car = { make: "Ford", year: 2020 };

"make" in car;                 // true
"toString" in car;             // true  (inherited)
Object.hasOwn(car, "toString"); // false

delete car.year;               // true; car is { make: "Ford" }
const arr = [1, 2, 3];
delete arr[1];                 // [1, empty, 3]  (hole!)

void 0;                        // undefined
typeof car;                    // "object"
car instanceof Object;         // true

for (let i = 0, j = 10; i < j; i++, j--) {}  // comma in for
const r = (1, 2, 3);           // 3
!("x" in car);                 // parentheses needed
```

CODE EXPLAINED

- → `in` — key exists anywhere in the chain
- → `delete` — removes a property; can't delete variables
- → `void` — forces `undefined`
- → Comma — evaluates left to right, keeps the last value

TYPESCRIPT

- `in` narrows union types: `if ("swim" in animal)`
- `delete` is only allowed on optional properties under `strict`

PRECEDENCE (HIGH → LOW)

LEVEL	OPERATORS
access / call	`. ?. [] () new`
unary	`! ~ + - typeof void delete await`
arithmetic	`** › * / % › + -`
relational	`< > <= >= in instanceof`
equality	`== != === !==`
logical	`&& › || › ??`
assign / arrow	`= += ??= => ?:`
comma	`,`

OPERATORS

OPERATOR	RETURNS
`typeof x`	type name string
`x instanceof C`	boolean
`key in obj`	boolean
`delete obj.k`	boolean
`void expr`	undefined
`a, b`	b

⚠ COMMON MISTAKES

- Writing `!key in obj`, which tests `false in obj`; use `!(key in obj)`
- Using `delete` on arrays and leaving holes
- Using `in` when inherited properties should not count; use `Object.hasOwn`

✓ TIPS & BEST PRACTICES

- Prefer `Object.hasOwn(o, k)` over `o.hasOwnProperty(k)`
- To remove a key immutably, use rest destructuring instead of `delete`
- When in doubt about precedence, add parentheses

QUICK SUMMARY

- `in` — property exists (incl. inherited)
- `delete` — remove a property
- `void` — undefined
- precedence — parenthesize when unsure

if / else if / else

OVERVIEW

`if` runs a block when its condition is **truthy**; `else if` chains further checks; `else` catches everything left. Conditions don't need to be booleans, which is convenient and occasionally dangerous. **Guard clauses** (early `returns`) keep the main logic un-nested.

NOTES

- Braces are optional for single statements, but always use them
- `let` / `const` inside a block are scoped to that block
- There's no `elif`; it's `else if`
- Only the first matching branch runs

BRANCHING AND GUARD CLAUSES

```
const score = 82;

if (score >= 90) {
  grade = "A";
} else if (score >= 80) {
  grade = "B";                // this branch runs
} else {
  grade = "C";
}

// Guard clauses: handle bad cases first, then the main path
function ship(order) {
  if (!order) return "no order";
  if (order.items.length === 0) return "empty";
  if (!order.paid) return "awaiting payment";
  return `shipping ${order.items.length} items`;
}

if (user) { /* truthy: object exists */ }
if (count === 0) { /* explicit beats truthy for numbers */ }
```

CODE EXPLAINED

- → Conditions are checked top to bottom; first truthy wins
- → Guard clauses — return early so the happy path isn't nested
- → Truthy checks are fine for objects; be explicit for numbers and strings
- → Each branch has its own block scope

TYPESCRIPT

- Each branch **narrows** types: after `if (typeof x === "string")`, x is a string inside
- After a guard `if (!user) return`, TS knows `user` is defined below

CONDITION PATTERNS

PATTERN	MEANING
`if (x)`	x is truthy
`if (x != null)`	x is not null/undefined
`if (arr.length)`	array not empty
`if (str.trim())`	string has content
`if (Object.keys(o).length)`	object has keys
`if (a && b)`	both truthy

STYLE RULES

DO	AVOID
`always use braces`	`if (x) doThing()`
`guard early`	`deep nesting`
`=== comparisons`	`== comparisons`
`name complex tests`	`long inline conditions`

⚠ COMMON MISTAKES

- Writing `if (x = 5)`; it assigns and is always true
- Using `if (count)` when `0` is a valid count
- Nesting four levels of `if`; invert the conditions and return early

✓ TIPS & BEST PRACTICES

- Extract long conditions into a named boolean: `const canShip = paid && inStock;`
- Use a lookup object instead of long `else if` chains that map values
- Keep the most common case first for readability

QUICK SUMMARY

- `if / else if / else` — branch on truthiness
- guard clauses — early return
- block scope — per branch
- explicit checks for 0 and ""

Ternary & Conditional Expressions

? : · nested ternaries · && short-circuit · ?? defaults · expression vs statement

OVERVIEW

The **ternary operator** `cond ? a : b` is an *expression*, so it produces a value you can assign, return, or embed in a template. Use it for simple either/or choices. Short-circuit operators `&&`, `||`, and `??` are the other common conditional expressions.

NOTES

- Ternaries can be nested, but readability drops fast
- Format nested ternaries one condition per line
- `cond && value` returns `false` /falsy when cond fails
- In JSX, `&&` and ternaries are the standard conditional rendering tools

CONDITIONAL EXPRESSIONS

```
const age = 20;
const label = age >= 18 ? "adult" : "minor";       // "adult"
const msg = `You have ${n} item${n === 1 ? "" : "s"}`;

const size =
  w > 1200 ? "xl" :
  w > 800  ? "lg" :
  w > 500  ? "md" : "sm";                          // flat chain

return isAdmin ? renderAdmin() : renderUser();

const name = user?.name ?? "Guest";                // null fallback
const title = draft.title || "Untitled";           // falsy fallback
const badge = count > 0 && `(${count})`;           // false or string

// Statement form: prefer if/else for side effects
isValid ? save() : showError();   // works, but if/else is clearer
```

CODE EXPLAINED

→ `? :` — the whole expression evaluates to one branch

→ Flat chained ternary — readable when each line is one rule

→ `&&` returns `false` when the condition fails, so watch what you render

→ Use `if/else` when branches *do* things rather than produce values

TYPESCRIPT

- The result type is the union of both branches: `cond ? 1 : "a"` is `1 | "a"`
- Ternaries also exist at type level as conditional types (sheet 126)

WHICH TO USE

SITUATION	USE
pick one of two values	a ? b : c
default for missing	x ?? d
default for falsy	x || d
value only if true	c && v
run side effects	if / else
map many values	object lookup

LOOKUP INSTEAD OF CHAINS

STYLE	CODE
chain	`s === "a" ? 1 : s === "b" ? 2 : 0`
lookup	`({ a: 1, b: 2 })[s] ?? 0`
Map	`new Map([["a", 1]]).get(s) ?? 0`

⚠ COMMON MISTAKES

- Deeply nested ternaries on one line; nobody can read them
- `count && <Badge/>` renders `0` when count is 0; use `count > 0 &&`
- Using a ternary purely for side effects

✓ TIPS & BEST PRACTICES

- If a ternary needs a comment, rewrite it as `if/else`
- Let Prettier format nested ternaries so each condition gets a line
- Replace value-mapping chains with an object or `Map` lookup

QUICK SUMMARY

- `c ? a : b` — expression, returns a value
- nested — one rule per line
- `&& / || / ??` — short-circuit choices
- side effects — use if/else

switch

OVERVIEW

`switch` compares one value against several `case` labels using **strict equality** (===). Execution **falls through** to the next case unless you `break` (or `return`). `default` handles unmatched values. It's best for matching one variable against many fixed values.

NOTES

- Cases can be any expression, compared with `===`
- Stacking empty cases groups them: `case "a": case "b":`
- Wrap a case in `{ }` to declare `let` / `const` inside it
- `default` can go anywhere but usually comes last

SWITCH, FALL-THROUGH, SWITCH(TRUE)

```
function dayType(day) {
  switch (day) {
    case "Sat":
    case "Sun":                        // grouped cases
      return "weekend";
    case "Fri": { const msg = "almost"; return msg; } // block
    default:
      return "weekday";
  }
}

switch (status) {
  case 200: console.log("OK"); break;
  case 404: console.log("Missing"); // no break: falls through!
  case 500: console.log("Error");   // also runs for 404
}

switch (true) {                        // ranges
  case temp < 0:  label = "freezing"; break;
  case temp < 20: label = "cool"; break;
  default:        label = "warm";
}
```

CODE EXPLAINED

→ `return` exits the function, so no `break` needed

→ Missing `break` — execution continues into the next case

→ Braces give a case its own scope

→ `switch (true)` — first case expression that is `true` wins

TYPESCRIPT

- Get **exhaustiveness checks** by assigning to `never` in `default`: `const _x: never = value;`
- Cases narrow discriminated unions: `case "circle":` knows the shape is a circle

PARTS

KEYWORD	ROLE
`switch (x)`	value to match
`case v:`	runs if x === v
`break`	exit the switch
`default:`	no case matched
`{ }`	scope inside a case

SWITCH VS LOOKUP OBJECT

SWITCH	OBJECT
`any logic per case`	`value mapping only`
`fall-through possible`	`no fall-through`
`=== comparison`	`string keys`
`statements`	`expression`

⚠ COMMON MISTAKES

- Forgetting `break` and running the next case too
- Declaring `const` in two cases without braces: "Identifier has already been declared"
- Expecting `case "1"` to match the number `1`; matching is strict

✓ TIPS & BEST PRACTICES

- Add `// falls through` comments when fall-through is intentional (ESLint `no-fallthrough`)
- Return from each case inside functions to avoid `break` entirely
- Use an object lookup when every case just returns a value

QUICK SUMMARY

- `switch` — match one value, `===`
- `break` — stops fall-through
- `default` — unmatched values
- `switch (true)` — range checks

for Loops

OVERVIEW

The classic `for` loop has three parts: an **initializer**, a **condition** checked before each pass, and an **update** run after each pass. Use it when you need the index, a custom step, or reverse order. For simply visiting every element, `for...of` (sheet 034) is cleaner.

NOTES

- All three parts are optional: `for (;;)` loops forever
- Declare the counter with `let` so each iteration gets its own copy
- `arr.length` is re-read every pass, so modifying the array changes the loop
- Loop backward when removing items by index

FOR LOOP VARIANTS

```
for (let i = 0; i < 5; i++) {
  console.log(i);                 // 0 1 2 3 4
}

const arr = ["a", "b", "c"];
for (let i = 0; i < arr.length; i++) {
  console.log(i, arr[i]);         // 0 "a", 1 "b", 2 "c"
}

for (let i = arr.length - 1; i >= 0; i--) {}   // reverse
for (let i = 0; i <= 100; i += 10) {}          // step by 10
for (let i = 0, j = 9; i < j; i++, j--) {}     // two counters

for (let r = 0; r < 3; r++) {                  // nested: grid
  for (let c = 0; c < 3; c++) {
    grid[r][c] = r * 3 + c;
  }
}

for (var k = 0; k < 3; k++) setTimeout(() => console.log(k)); // 3 3 3
for (let k = 0; k < 3; k++) setTimeout(() => console.log(k)); // 0 1 2
```

CODE EXPLAINED

→ `i < arr.length` — stops before the out-of-range index

→ Reverse loops — start at `length - 1`, condition `>= 0`

→ Comma lets you update two counters

→ `let` creates a fresh binding per iteration; `var` shares one

TYPESCRIPT

- The counter is inferred as `number`; no annotation needed
- With `noUncheckedIndexedAccess`, `arr[i]` is `T | undefined` even inside the loop

COMMON FORMS

HEADER	ITERATES
`(let i = 0; i < n; i++)`	0 … n-1
`(let i = 1; i <= n; i++)`	1 … n
`(let i = n - 1; i >= 0; i--)`	n-1 … 0
`(let i = 0; i < n; i += 2)`	even indexes
`(;;)`	forever (needs break)

WHICH LOOP?

NEED	LOOP
`index + value`	for or entries()
`just values`	for…of
`object keys`	for…in / Object.keys
`unknown count`	while
`transform array`	map / filter

⚠ COMMON MISTAKES

- Off-by-one: `i <= arr.length` reads one past the end (`undefined`)
- Using `var` for the counter with async callbacks inside
- Removing items with `splice` while looping forward, which skips elements

✓ TIPS & BEST PRACTICES

- Cache length in hot loops only if profiling says so; engines optimize it already
- Reach for `for...of` or array methods first; use `for` when you need index control
- Name nested counters meaningfully (`row`, `col`) instead of `i`, `j`

QUICK SUMMARY

- `for (init; cond; update)` — classic loop
- `let i` — fresh per iteration
- reverse — `i--` from `length - 1`
- use for index or step control

for...of & for...in

OVERVIEW

`for...of` loops over the **values** of any iterable: arrays, strings, Maps, Sets, NodeLists, generators. `for...in` loops over the **enumerable string keys** of an object, including inherited ones. Use `for...of` for arrays and `Object.entries()` + `for...of` for objects.

NOTES

- `for...in` on arrays gives string indexes and can include extra properties
- Plain objects aren't iterable; `for...of` on one throws
- `break`, `continue`, and `return` work in both (unlike `forEach`)
- `for await...of` iterates async iterables (sheet 094)

FOR...OF VS FOR...IN

```
const colors = ["red", "green"];
for (const c of colors) console.log(c);          // red, green
for (const [i, c] of colors.entries()) {}         // 0 red, 1 green
for (const ch of "hi") console.log(ch);           // h, i

const prices = { apple: 1.2, pear: 0.8 };
for (const key in prices) {
  console.log(key, prices[key]);                  // apple 1.2, pear 0.8
}
for (const [fruit, price] of Object.entries(prices)) {
  console.log(`${fruit}: $${price}`);
}

const m = new Map([["a", 1], ["b", 2]]);
for (const [k, v] of m) console.log(k, v);        // a 1, b 2
for (const v of new Set([1, 1, 2])) {}            // 1, 2

for (const i in colors) typeof i;                 // "string"!
// for (const x of prices) {}                     // TypeError: not iterable
```

CODE EXPLAINED

- → `entries()` — index and value together
- → `Object.entries` — `[key, value]` pairs of an object
- → Map iteration yields `[key, value]` pairs in insertion order
- → `for...in` keys are always strings

TYPESCRIPT

- In `for...in`, the key is typed `string`, not `keyof T`, so indexing needs a cast or `Object.entries`
- `for...of` infers the element type from the iterable

FOR...OF VS FOR...IN

	FOR...OF	FOR...IN
Iterates	values	keys
Works on	iterables	any object
Arrays	yes, use this	avoid
Inherited props	no	yes
Key type	n/a	string

ITERATING OBJECTS

NEED	CODE
keys	`Object.keys(o)`
values	`Object.values(o)`
pairs	`Object.entries(o)`
own keys only	`Object.keys / hasOwn`

⚠ COMMON MISTAKES

- Using `for...in` on arrays; you get string indexes and possibly extra keys
- Using `for...of` on a plain object; it throws "not iterable"
- Declaring the loop variable with `let` when you never reassign it; use `const`

✓ TIPS & BEST PRACTICES

- Default to `for (const x of arr)`
- Destructure in the loop head: `for (const { id, name } of users)`
- Use `Object.entries` instead of `for...in` to avoid inherited keys

QUICK SUMMARY

- `for...of` — values of iterables
- `for...in` — object keys (strings)
- `entries()` — index + value
- `Object.entries` — loop over objects

while & do...while

OVERVIEW

`while` repeats as long as its condition is truthy and checks **before** each pass, so the body may never run. `do...while` checks **after** the body, so it always runs at least once. Use them when the number of iterations isn't known in advance: reading input, retrying, polling, or walking a linked structure.

NOTES

- Make sure something in the body moves toward ending the loop
- `while (true)` with an internal `break` is a valid pattern
- `do...while` needs a semicolon after the condition
- In async code, `await` inside `while` runs iterations sequentially

WHILE AND DO...WHILE

```
let n = 5;
while (n > 0) {
  console.log(n);           // 5 4 3 2 1
  n--;
}

let node = list.head;       // walk a linked list
while (node) {
  console.log(node.value);
  node = node.next;
}

let guess;
do {
  guess = Math.ceil(Math.random() * 6);
} while (guess !== 6);      // always rolls at least once

let attempts = 0;
while (true) {              // retry with exit condition
  if (await tryConnect()) break;
  if (++attempts >= 3) throw new Error("gave up");
}
```

CODE EXPLAINED

- → Condition checked first — `while` may run zero times
- → `while (node)` — runs until the reference is `null`
- → `do...while` — body first, then the check
- → `while (true)` + `break` — clear when exits are in the middle

TYPESCRIPT

- Narrowing works in loops: `while (node)` treats `node` as non-null inside
- Otherwise identical in TypeScript

WHILE VS DO...WHILE

	WHILE	DO...WHILE
Check	before body	after body
Min runs	0	1
Typical use	unknown count	prompt until valid
Syntax end	}	} while (c);

PATTERNS

PATTERN	SHAPE
countdown	while (n > 0) n--
sentinel	while ((x = next()) !== null)
traverse	while (node) node = node.next
retry	while (true) { … break }
validate input	do { … } while (!ok)

⚠ COMMON MISTAKES

- Forgetting to update the condition variable, creating an infinite loop
- Busy-waiting with `while` for time to pass; use `setTimeout` or `await`
- Leaving off the semicolon after `do...while (cond)`

✓ TIPS & BEST PRACTICES

- Add a max-iterations guard to loops driven by external data
- Prefer `for` when the count is known; `while` when it isn't
- Put the exit condition near the top of a `while (true)` body when possible

QUICK SUMMARY

- `while` — check first, may run 0 times
- `do...while` — runs at least once
- `while (true)` — exit with break
- always progress toward the exit

Loop Controls

OVERVIEW

`break` exits the innermost loop (or `switch`) immediately. `continue` skips to the next iteration. **Labels** let `break` or `continue` target an outer loop. Array methods like `forEach` can't be broken out of; use `for...of`, `some`, `every`, or `find` when you need to stop early.

NOTES

- `return` inside a loop exits the whole function
- Labels are identifiers followed by `:`
- `continue` in a `for` loop still runs the update expression
- There's no `pass`; an empty block `{}` does nothing

BREAK, CONTINUE, LABELS

```
for (const n of [1, 2, 3, 4, 5]) {
  if (n === 2) continue;        // skip 2
  if (n === 4) break;           // stop at 4
  console.log(n);               // 1 3
}

outer: for (const row of grid) {
  for (const cell of row) {
    if (cell === target) {
      console.log("found");
      break outer;              // exits both loops
    }
  }
}

[1, 2, 3].forEach(n => {
  if (n === 2) return;          // only skips this callback
});

const firstBig = nums.find(n => n > 100);    // stops at first match
const hasNeg = nums.some(n => n < 0);        // stops when true
```

CODE EXPLAINED

- → `continue` — skip the rest of this pass
- → `break` — leave the loop entirely
- → `break label` — leave a named outer loop
- → `return` in `forEach` acts like `continue`, not `break`

TYPESCRIPT

- TS flags unreachable code after `break` / `return` (controlled by `allowUnreachableCode`)
- Unused labels are flagged by `allowUnusedLabels: false`

CONTROLS

STATEMENT	EFFECT
`break`	exit innermost loop/switch
`continue`	next iteration
`break label`	exit labeled loop
`continue label`	next pass of labeled loop
`return`	exit the function

EARLY-EXIT ARRAY METHODS

METHOD	STOPS WHEN
`some`	callback returns true
`every`	callback returns false
`find / findIndex`	first match
`forEach`	never (use for…of)

⚠ COMMON MISTAKES

- Trying to `break` out of `forEach`; it's a SyntaxError
- Using `return` in `forEach` expecting it to stop the loop
- Overusing labels; extracting a function with `return` is usually clearer

✓ TIPS & BEST PRACTICES

- Replace search loops with `find`, `some`, or `includes`
- Use `continue` for guard-style skips at the top of the loop body
- Move nested search loops into a function and `return` when found

QUICK SUMMARY

- `break` — exit loop
- `continue` — skip to next
- labels — target outer loops
- `forEach` — can't break

try / catch / finally

OVERVIEW

`try` runs code that might throw; `catch` receives the thrown value; `finally` always runs afterward, whether or not an error happened, even if `try` returns. Errors are objects with `name`, `message`, and `stack`. Catch only what you can handle, and let the rest propagate.

NOTES

- The catch binding is optional: `catch { … }`
- `try/catch` only catches synchronous errors and awaited rejections
- A `return` in `finally` overrides earlier returns and swallows errors
- Any value can be thrown, but always throw `Error` objects

TRY / CATCH / FINALLY

```
function parseConfig(text) {
  try {
    return JSON.parse(text).port;
  } catch (err) {
    if (err instanceof SyntaxError) {
      console.error("Bad JSON:", err.message);
      return 3000;                      // recover
    }
    throw err;                          // not ours: rethrow
  } finally {
    console.log("parse attempted");     // always runs
  }
}

try { null.prop; }
catch (e) {
  e.name;       // "TypeError"
  e.message;    // "Cannot read properties of null…"
}

try { risky(); } catch { /* ignore on purpose */ }
```

CODE EXPLAINED

→ Check the error type before handling it

→ Rethrow errors you don't recognize

→ `finally` — cleanup that must always happen

→ `catch { }` without a variable when you don't need the error

TYPESCRIPT

- With `strict` (`useUnknownInCatchVariables`), `err` is `unknown`; narrow with `instanceof Error` before using `.message`
- There are no typed catch clauses; you can't write `catch (e: SyntaxError)`

BUILT-IN ERROR TYPES

TYPE	RAISED WHEN
`Error`	generic base class
`TypeError`	wrong type, null access, calling a non-function
`ReferenceError`	undeclared variable, TDZ
`SyntaxError`	bad code or JSON.parse input
`RangeError`	invalid length, too much recursion
`AggregateError`	several errors, e.g. Promise.any

ERROR PROPERTIES

PROPERTY	CONTAINS
`name`	type name
`message`	description
`stack`	trace (non-standard but universal)
`cause`	underlying error

⚠ COMMON MISTAKES

- Empty catch blocks that hide real bugs
- Wrapping async calls in try/catch without `await`; the rejection escapes
- Returning from `finally`, which silently discards thrown errors

✓ TIPS & BEST PRACTICES

- Catch at the level that can do something useful (retry, fallback, user message)
- Log `err` itself, not just `err.message`, to keep the stack
- Check types with `instanceof`, not by comparing `message` strings

QUICK SUMMARY

- `try` — risky code
- `catch (e)` — handle or rethrow
- `finally` — always runs
- `instanceof` — choose what to handle

Throwing Errors

OVERVIEW

`throw` stops the current function and unwinds the call stack until a `catch` handles it. Throw `Error` instances (or subclasses) so you get a stack trace. Use the `cause` option to **wrap** a low-level error in a higher-level one without losing the original.

NOTES

- `throw` is a statement; it can't be used as an expression (yet)
- Throwing a string loses the stack trace
- `AggregateError` bundles several errors in `.errors`
- Uncaught errors crash Node processes and log in browsers

THROWING, RETHROWING, WRAPPING

```
function withdraw(balance, amount) {
  if (typeof amount !== "number") {
    throw new TypeError("amount must be a number");
  }
  if (amount > balance) {
    throw new RangeError(`Insufficient funds: ${balance}`);
  }
  return balance - amount;
}

async function loadUser(id) {
  try {
    return await db.get(id);
  } catch (err) {
    throw new Error(`Failed to load user ${id}`, { cause: err });
  }
}

try { await loadUser(7); }
catch (e) { console.log(e.message, e.cause.message); }

throw new AggregateError([e1, e2], "2 checks failed");
const fail = msg => { throw new Error(msg); };  // throw in an expression
```

CODE EXPLAINED

→ Pick the most specific built-in type that fits

→ `{ cause: err }` — keeps the original error attached

→ `e.cause` — read the underlying error

→ Wrap `throw` in an arrow to use it where an expression is needed

TYPESCRIPT

- Function signatures don't declare what they throw; there are no checked exceptions
- A function that always throws can return `never`

WHEN TO THROW

SITUATION	THROW
bad argument type	TypeError
value out of range	RangeError
invalid state	Error (or custom)
wrapping lower error	new Error(msg, { cause })
many failures	AggregateError

DO / DON'T

DO	DON'T
`throw new Error("x")`	`throw "x"`
keep the cause	replace and lose it
clear messages with values	"Something went wrong"
rethrow unknown errors	swallow everything

⚠ COMMON MISTAKES

- Throwing strings or plain objects, which have no stack trace
- Catching, logging, and rethrowing at every level, producing duplicate logs
- Using exceptions for normal control flow like "not found"

✓ TIPS & BEST PRACTICES

- Include the offending value in the message: `Invalid port: ${port}`
- Wrap third-party errors at module boundaries using `cause`
- Validate inputs at the top of public functions and throw early

QUICK SUMMARY

- `throw new Error(msg)` — always Error objects
- `TypeError / RangeError` — specific types
- `{ cause }` — wrap without losing info
- `AggregateError` — several failures

Custom Errors

OVERVIEW

Create **custom errors** by extending `Error`. Set `this.name` so logs and stack traces show your type, and add fields that callers need, like an HTTP status or a field name. Callers then branch with `instanceof`. Create one base class per library or app so all your errors can be caught together.

NOTES

- Always call `super(message, options)` first in the constructor
- Setting `name` via a class field is the cleanest pattern
- `instanceof` works across a class hierarchy
- Error objects serialize poorly with `JSON.stringify`; add a `toJSON` if needed

A SMALL ERROR HIERARCHY

```
class AppError extends Error {
  name = "AppError";              // inherited constructor keeps cause
}
class NotFoundError extends AppError {
  name = "NotFoundError";
  status = 404;
}
class ValidationError extends AppError {
  name = "ValidationError";
  status = 400;
  constructor(field, message, options) {
    super(`${field}: ${message}`, options);
    this.field = field;
  }
}

const e = new ValidationError("email", "is required");
e instanceof AppError;            // true
e.field;                          // "email"
`${e}`;                           // "ValidationError: email: is required"
```

CODE EXPLAINED

→ `name` field — shows in stack traces and `toString()`

→ Base `AppError` — catch every app error with one `instanceof`

→ Extra fields like `status` and `field` carry structured info

→ Pass `options` to `super` to keep `cause` support

TYPESCRIPT

- Declare extra fields with types: `readonly status = 404;`
- When targeting ES5, add `Object.setPrototypeOf(this, new.target.prototype)` or `instanceof` breaks

USEFUL CUSTOM FIELDS

FIELD	PURPOSE
`status`	HTTP status to return
`code`	stable machine-readable ID
`field`	which input failed
`retryable`	safe to retry?
`details`	extra context object

CUSTOM ERROR OR NOT?

SITUATION	CHOICE
`callers branch on it`	custom class
`needs extra data`	custom class
`one-off failure`	plain Error
`wrong argument type`	TypeError

⚠ COMMON MISTAKES

- Forgetting to set `name`, so every error logs as "Error"
- Accessing `this` before calling `super()`
- Creating a custom class for every possible failure; keep the hierarchy small

✓ TIPS & BEST PRACTICES

- Add a stable `code` string; messages change, codes shouldn't
- Map custom errors to HTTP responses in one central handler
- Export your error classes so callers can use `instanceof`

QUICK SUMMARY

- `class X extends Error` — custom type
- `name` — readable type in logs
- extra fields — structured data
- base class — catch all app errors

Sheet 040

Global Handlers & Error Patterns

window.onerror · unhandledrejection · process.on('uncaughtException') · error-first callbacks · result objects

OVERVIEW

Some errors escape every `try/catch`. Global handlers are the **last line of defense**: log the error, report it, and (in Node) exit cleanly. Beyond exceptions, JavaScript code uses other error patterns: Node's legacy **error-first callbacks** and **result objects** that return errors as values instead of throwing.

NOTES

- Unhandled promise rejections crash Node by default
- After `uncaughtException`, the process is in an unknown state; exit
- Browsers fire `error` and `unhandledrejection` on `window`
- Global handlers are for logging, not recovery

GLOBAL HANDLERS AND PATTERNS

```
// Browser
window.addEventListener("error", e => report(e.error));
window.addEventListener("unhandledrejection", e => {
  report(e.reason);
  e.preventDefault();                // suppress console noise
});

// Node
process.on("uncaughtException", err => {
  log(err); process.exit(1);         // state is unsafe, exit
});
process.on("unhandledRejection", reason => { throw reason; });

// Error-first callback (legacy Node style)
fs.readFile("a.txt", "utf8", (err, data) => {
  if (err) return console.error(err);
  console.log(data);
});

// Result object: errors as values
function parse(s) {
  try { return { ok: true, value: JSON.parse(s) }; }
  catch (error) { return { ok: false, error }; }
}
const r = parse(input);
if (!r.ok) showError(r.error);
```

CODE EXPLAINED

- → `unhandledrejection` — a promise rejected with no `.catch`
- → `uncaughtException` — log, then exit
- → Error-first callbacks — check `err` before using `data`
- → Result objects — callers must check `ok`; nothing is thrown

TYPESCRIPT

- Result types shine with discriminated unions: `{ ok: true; value: T } | { ok: false; error: E }`
- TS then forces callers to check `ok` before touching `value`

GLOBAL HANDLERS

ENV	EVENT	CATCHES
browser	error	uncaught exceptions
browser	unhandledrejection	unhandled promises
Node	uncaughtException	uncaught exceptions
Node	unhandledRejection	unhandled promises

ERROR STYLES COMPARED

STYLE	BEST FOR
throw / catch	unexpected failures
result object	expected failures (validation)
error-first callback	old Node APIs only
promise rejection	async failures

⚠ COMMON MISTAKES

- Continuing to run after `uncaughtException`
- Using global handlers instead of handling errors where they happen
- Mixing callbacks and promises; use `fs/promises` or `util.promisify`

✓ TIPS & BEST PRACTICES

- Send global errors to a reporting service (Sentry, etc.)
- Use result objects for expected failures like form validation
- Convert callback APIs with `util.promisify(fn)`

QUICK SUMMARY

- `unhandledrejection` — unhandled promises
- `uncaughtException` — log and exit
- error-first callbacks — legacy
- result objects — errors as values

Declarations & Expressions

OVERVIEW

A **function declaration** (`function name() {}`) is hoisted, so you can call it before it appears in the file. A **function expression** assigns a function to a variable and isn't usable until that line runs. Functions return `undefined` unless they `return` something. An **IIFE** runs a function immediately, mostly in legacy code to create a private scope.

NOTES

- Functions are values: store, pass, and return them
- `return` with no value returns `undefined`
- Named function expressions can refer to themselves by name
- Modules and block scope replaced most IIFE uses

DECLARATIONS, EXPRESSIONS, IIFE

```
greet("Ana");                          // works: declaration is hoisted
function greet(name) {
  return `Hello, ${name}`;
}

// square(3);                          // ReferenceError (TDZ)
const square = function (n) { return n * n; };
const fact = function f(n) { return n <= 1 ? 1 : n * f(n - 1); };

function log(msg) { console.log(msg); }
log("x");                              // returns undefined

function minMax(arr) {
  return [Math.min(...arr), Math.max(...arr)]; // "multiple" returns
}
const [lo, hi] = minMax([3, 9, 1]);    // 1, 9

(function () {                         // IIFE
  const secret = 42;                   // not visible outside
})();
```

CODE EXPLAINED

- → Declarations hoist with their body
- → Expressions follow normal `const` rules (TDZ)
- → Return an array or object to send back several values
- → IIFE — define and call in one step

TYPESCRIPT

- Annotate parameters and the return type: `function greet(name: string): string`
- Return types are inferred, but explicit ones on exported functions catch mistakes early

WAYS TO DEFINE A FUNCTION

FORM	HOISTED?
`function f() {}`	yes
`const f = function () {}`	no
`const f = () => {}`	no
`{ f() {} } (method)`	n/a
`new Function("a", "return a")`	no (avoid)

RETURN VALUES

CODE	RETURNS
`return;`	undefined
`(no return)`	undefined
`return a, b;`	b (comma op)
`return [a, b];`	array
`return { a, b };`	object

⚠ COMMON MISTAKES

- Putting the return value on a new line after `return`; ASI returns `undefined`
- Calling a function expression before its definition
- Writing `return a, b` expecting two values; you get only `b`

✓ TIPS & BEST PRACTICES

- Use declarations for top-level named functions; they read like a table of contents
- Return objects (`{ min, max }`) when the values have different meanings
- Keep functions small enough to describe in one sentence

QUICK SUMMARY

- `function f(){}` — hoisted declaration
- `const f = function(){}` — expression
- no return — `undefined`
- IIFE — run immediately

Parameters & Defaults

OVERVIEW

Parameters are **positional** and never required: missing ones are `undefined`, extra ones are ignored. **Default values** apply when an argument is `undefined`. JavaScript has no named arguments, so the common substitute is passing an **options object** and destructuring it in the parameter list.

NOTES

- Defaults are evaluated at call time and can reference earlier params
- `null` does **not** trigger a default; only `undefined` does
- Arguments are passed by sharing: objects can be mutated, but reassigning a param doesn't affect the caller
- `fn.length` counts params before the first default

DEFAULTS AND OPTIONS OBJECTS

```
function greet(name = "friend", punct = "!") {
  return `Hi ${name}${punct}`;
}
greet();                  // "Hi friend!"
greet("Ana");             // "Hi Ana!"
greet(undefined, "?");    // "Hi friend?"
greet(null);              // "Hi null!"

function rect(w, h = w) { return w * h; }   // default uses w
rect(4);                  // 16

function connect({ host = "localhost", port = 5432, ssl = false } = {}) {
  return `${host}:${port} ssl=${ssl}`;
}
connect({ port: 6543 });  // "localhost:6543 ssl=false"
connect();                // works thanks to "= {}"

function rename(user) { user.name = "Bo"; user = null; }
const u = { name: "Al" }; rename(u); u.name;  // "Bo"
```

CODE EXPLAINED

- → Default applies only for `undefined`
- → Later defaults can use earlier parameters
- → Destructured options + `= {}` — named, optional arguments
- → Mutation is visible to the caller; reassignment is not

TYPESCRIPT

- Mark optional params with `?`: `function f(a: string, b?: number)`
- A parameter with a default is automatically optional; type the options object with an interface

ARGUMENT HANDLING

CALL TO F(A, B)	A	B
`f(1)`	1	undefined
`f(1, 2, 3)`	1	2 (3 ignored)
`f(undefined, 2)`	default	2
`f(null, 2)`	null	2

OPTIONS OBJECT PATTERN

BENEFIT	WHY
`named arguments`	self-documenting calls
`any order`	no positional mistakes
`easy to extend`	add keys without breaking callers
`defaults per key`	destructuring defaults

⚠ COMMON MISTAKES

- Passing `null` and expecting the default to apply
- Using mutable shared defaults by reference (e.g., one cached object reused across calls)
- More than three positional parameters; switch to an options object

✓ TIPS & BEST PRACTICES

- Put required parameters first and optional ones last
- Always add `= {}` after a destructured options parameter
- Don't mutate arguments; return a new value instead

QUICK SUMMARY

- missing args — `undefined`
- `param = default` — for undefined only
- options object — named arguments
- pass by sharing — objects mutable

Rest Parameters & Spread Arguments

...args · spread into calls · arguments vs rest · Math.max(...arr)

OVERVIEW

A **rest parameter** (`...args`) collects any number of remaining arguments into a real array. **Spread** does the opposite at the call site, expanding an array into separate arguments. Together they replace the old `arguments` object, which is array-like but not an array and doesn't exist in arrow functions.

NOTES

- Only one rest parameter, and it must be last
- Rest gives `[]` when no extra arguments are passed
- `arguments` still exists in regular functions, but avoid it
- Spread works with any iterable: Sets, strings, Maps

REST PARAMETERS AND SPREAD ARGUMENTS

```
function sum(...nums) {
  return nums.reduce((t, n) => t + n, 0);
}
sum();                      // 0
sum(1, 2, 3);               // 6

function tag(label, ...items) {
  return `${label}: ${items.join(", ")}`;
}
tag("Fruit", "apple", "fig");   // "Fruit: apple, fig"

const nums = [5, 1, 9];
Math.max(...nums);              // 9
sum(...nums, 10);               // 25
new Date(...[2026, 0, 15]);     // spread into constructor

function old() {
  return Array.from(arguments); // legacy array-like
}
const wrap = (fn) => (...args) => fn(...args);  // forward all args
```

CODE EXPLAINED

- → `...nums` — always an array, possibly empty
- → Leading named params take the first arguments
- → Spread in calls — pass array items as separate arguments
- → `(...args) => fn(...args)` — forward everything unchanged

TYPESCRIPT

- Type rest as an array: `(...nums: number[])`, or a tuple for fixed shapes: `(...args: [string, number])`
- `Parameters<typeof fn>` captures another function's argument list (sheet 125)

REST VS ARGUMENTS

	...REST	ARGUMENTS
Real array	yes	no (array-like)
In arrows	yes	no
Named	yes	no
Only extras	yes	all args

COMMON USES

PATTERN	EXAMPLE
variadic math	`sum(...nums)`
forward args	`(...a) => f(...a)`
head + tail	`(first, ...rest)`
array to args	`Math.min(...arr)`

⚠ COMMON MISTAKES

- Placing a parameter after rest: `(...a, b)` is a SyntaxError
- Using `arguments` inside an arrow function; it refers to the outer function's arguments
- Spreading very large arrays into calls; it can exceed the engine's argument limit

✓ TIPS & BEST PRACTICES

- Use `arr.reduce` instead of `Math.max(...arr)` for huge arrays
- Combine rest with destructuring: `([first, ...others]) => …`
- Prefer explicit parameters when the count is fixed

QUICK SUMMARY

- `...args` — collect extras into array
- `f(...arr)` — expand into arguments
- rest last — only one allowed
- `arguments` — legacy, avoid

Variable Scope

OVERVIEW

Scope decides where a variable is visible. JavaScript uses **lexical scope**: what a function can see depends on where it's *written*, not where it's called. `let`/`const` are block-scoped, `var` is function-scoped, and inner scopes can **shadow** outer names.

NOTES

- Lookups go inner → outer until found, else ReferenceError
- Modules have their own top-level scope
- `globalThis` is the global object everywhere (`window` in browsers)
- Function parameters live in the function's scope

SCOPE LEVELS AND SHADOWING

```
const app = "global";                // module/global scope

function outer() {
  const x = "outer";                 // function scope
  if (true) {
    const x = "block";               // shadows outer x
    let y = 1;                       // block scope
    console.log(x);                  // "block"
  }
  console.log(x);                    // "outer"
  // console.log(y);                 // ReferenceError
  function inner() {
    return `${app} > ${x}`;          // reads outer scopes
  }
  return inner();
}
outer();                             // "global > outer"

function leak() { if (true) { var v = 1; } return v; }
leak();                              // 1  (var ignores blocks)
```

CODE EXPLAINED

- → Inner scopes see outer variables, never the reverse
- → Shadowing — inner `x` hides outer `x` inside the block
- → `let y` disappears when its block ends
- → `var` belongs to the whole function

TYPESCRIPT

- Scope rules are identical; TS adds no runtime behavior
- Enable ESLint's `no-shadow` (typescript-eslint version) to flag accidental shadowing

SCOPE LEVELS

SCOPE	CREATED BY	HOLDS
`global`	script top level	var, function
`module`	each ES module file	all top-level names
`function`	every function	params, var, let, const
`block`	{ } if for while	let, const, class

NAME RESOLUTION

STEP	LOOKS IN
1	current block
2	enclosing blocks
3	enclosing functions
4	module scope
5	global → ReferenceError

⚠ COMMON MISTAKES

- Shadowing a variable by accident and updating the wrong one
- Relying on globals between files instead of imports
- Using `var` and expecting block scoping

✓ TIPS & BEST PRACTICES

- Keep variables in the smallest scope that works
- Avoid reusing outer names for inner variables
- Wrap scripts in modules rather than IIFEs to avoid globals

QUICK SUMMARY

- lexical scope — set by where code is written
- `let/const` — block scoped
- `var` — function scoped
- shadowing — inner name hides outer

Closures

OVERVIEW

A **closure** is a function plus the variables it captured from the scope where it was created. The function keeps access to those variables even after the outer function has returned. Closures power private state, factories, callbacks, memoization, and the module pattern.

NOTES

- Closures capture **variables**, not snapshots of their values
- Each call of the outer function creates a fresh set of captured variables
- Captured variables stay in memory as long as the closure is reachable
- Every function in JavaScript is technically a closure

CLOSURES IN PRACTICE

```
function makeCounter(start = 0) {
  let count = start;                // private state
  return {
    inc: () => ++count,
    get: () => count,
  };
}
const a = makeCounter(), b = makeCounter(10);
a.inc(); a.inc();                   // 2
b.inc();                            // 11  (independent)

const multiplier = n => x => x * n;  // factory
const double = multiplier(2);
double(7);                          // 14

function memo(fn) {
  const cache = new Map();
  return arg => cache.has(arg) ? cache.get(arg)
                               : (cache.set(arg, fn(arg)), cache.get(arg));
}

for (var i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 3 3 3
```

CODE EXPLAINED

- → `count` is unreachable except through `inc` / `get`
- → Each `makeCounter()` call gets its own `count`
- → Factories return configured functions
- → `var` loop — all callbacks share one `i`; use `let`

TYPESCRIPT

- Captured variables keep their inferred types; a closure's return type is inferred too
- Type factories with generics to keep them reusable (sheet 122)

USES OF CLOSURES

USE	EXAMPLE
private state	counter, cache
factories	multiplier(2)
callbacks	handlers using outer vars
memoization	memo(fn)
once / debounce	wrap a function

LOOP CLOSURE BUG

LOOP	LOGS
for (var i …) setTimeout	3 3 3
for (let i …) setTimeout	0 1 2
arr.forEach((x, i) …)	0 1 2

⚠ COMMON MISTAKES

- Expecting a closure to capture the value at creation time; it sees later changes
- Holding large objects in long-lived closures (e.g., event listeners), causing leaks
- Creating closures inside hot loops when one shared function would do

✓ TIPS & BEST PRACTICES

- Use closures for privacy in plain functions; use `#private` fields in classes
- Remove event listeners that capture big data when you're done
- Name returned functions to get clearer stack traces

QUICK SUMMARY

- closure — function + captured scope
- private state — hidden variables
- factories — functions returning functions
- captures variables — not values

Arrow Functions

=> · implicit return · returning ({}) · no own this / arguments · not constructible

OVERVIEW

Arrow functions are a compact syntax: `(params) => expression`. A single expression is returned implicitly; use braces and `return` for multiple statements. Arrows have no own `this`, `arguments`, or `prototype`. They inherit `this` from the surrounding code, which makes them ideal for callbacks.

NOTES

- One parameter can skip the parentheses: `x => x * 2`
- Return an object literal by wrapping it: `() => ({ a: 1 })`
- Arrows can't be used with `new`
- Avoid arrows as object methods that need `this`

ARROW SYNTAX FORMS

```
const sq = x => x * x;                 // implicit return
const add = (a, b) => a + b;
const now = () => Date.now();          // no params
const make = (id) => ({ id, ok: true }); // object literal: wrap in ()
const area = (w, h) => {               // block body: explicit return
  const a = w * h;
  return a;
};

[1, 2, 3].map(n => n * 2);             // [2, 4, 6]

class Timer {
  seconds = 0;
  start() {
    setInterval(() => this.seconds++, 1000);  // arrow keeps this
  }
}

const obj = {
  name: "x",
  bad: () => this.name,                // this is NOT obj
  good() { return this.name; },        // method syntax
};
```

CODE EXPLAINED

→ No braces — the expression is the return value

→ `({ … })` — parentheses stop `{` being read as a block

→ Arrow inside a method sees the method's `this`

→ Use method syntax for object methods

TYPESCRIPT

- Annotate inline: `const add = (a: number, b: number): number => a + b`
- In `.tsx` files, write generic arrows as `<T,>(x: T) => x` so the parser doesn't read JSX

ARROW VS REGULAR FUNCTION

	ARROW	FUNCTION
own `this`	no (lexical)	yes (call site)
`arguments`	no	yes
`new`	no	yes
hoisted	no	declarations yes
implicit return	yes	no

WHEN TO USE

USE ARROWS FOR	USE FUNCTION FOR
callbacks	object methods
short helpers	constructors
preserving `this`	functions needing `arguments`
one-line transforms	top-level named APIs

⚠ COMMON MISTAKES

- `() => { a: 1 }` returns `undefined`; braces became a block with a label
- Using an arrow as an object method and getting the wrong `this`
- Adding braces but forgetting `return`

✓ TIPS & BEST PRACTICES

- Use arrows for `map`, `filter`, event handlers, and promise chains
- Use class field arrows (`handle = () => {}`) for handlers passed around
- Keep implicit-return arrows to one readable line

QUICK SUMMARY

- `x => expr` — implicit return
- `() => ({})` — return an object
- lexical `this` — from surroundings
- no `new`, no `arguments`

this & Binding

OVERVIEW

`this` is set by **how a function is called**, not where it's defined (except arrows, which inherit it). As a method (`obj.f()`), `this` is `obj`. Called plainly (`f()`), it's `undefined` in strict mode. `call`, `apply`, and `bind` set `this` explicitly, and `new` sets it to the new object.

NOTES

- Extracting a method (`const f = obj.f`) loses its `this`
- Modules and classes are strict, so plain calls give `undefined`
- `bind` returns a new function; it doesn't change the original
- Event handlers set `this` to the element (non-arrow only)

HOW THIS GETS SET

```
const user = {
  name: "Ana",
  hi() { return `Hi, ${this.name}`; },
};
user.hi();                        // "Hi, Ana"  (method call)

const f = user.hi;
// f();                           // TypeError: this is undefined

function intro(greeting, punct) {
  return `${greeting}, ${this.name}${punct}`;
}
intro.call(user, "Hello", "!");     // "Hello, Ana!"
intro.apply(user, ["Hey", "?"]);    // "Hey, Ana?"
const bound = intro.bind(user, "Yo");
bound(".");                         // "Yo, Ana."

setTimeout(user.hi, 0);             // loses this
setTimeout(() => user.hi(), 0);     // keeps it
button.addEventListener("click", user.hi.bind(user));
```

CODE EXPLAINED

- → Method call — `this` is the object before the dot
- → `call(thisArg, ...args)` — call now with a `this`
- → `apply(thisArg, [args])` — same, args as array
- → `bind(thisArg, ...preset)` — new function, `this` fixed

TYPESCRIPT

- Declare `this` as a fake first parameter: `function intro(this: User, greeting: string)`
- `noImplicitThis` errors when `this` would be `any`

THIS RULES

CALL FORM	THIS IS
`obj.method()`	obj
`fn()`	undefined (strict)
`fn.call(x)`	x
`new Fn()`	the new object
`arrow`	outer this
`DOM handler (function)`	the element

CALL / APPLY / BIND

METHOD	RUNS NOW?	ARGS
`call`	yes	list
`apply`	yes	array
`bind`	no, returns fn	list (partial)

⚠ COMMON MISTAKES

- Passing `obj.method` as a callback and losing `this`
- Using an arrow function as a method and expecting `this` to be the object
- Calling `bind` repeatedly in render loops, creating new functions every time

✓ TIPS & BEST PRACTICES

- Wrap in an arrow (`() => obj.m()`) or `bind` once when passing methods around
- Use class-field arrow handlers to avoid binding in constructors
- Prefer spread over `apply` : `fn(...args)`

QUICK SUMMARY

- method call — this = object
- plain call — undefined (strict)
- `call / apply / bind` — set it explicitly
- arrows — inherit this

Higher-Order Functions

callbacks · returning functions · compose · pipe · currying · partial application

OVERVIEW

A **higher-order function** takes a function as an argument, returns a function, or both. Array methods (`map`, `filter`, `reduce`) are the everyday examples. Building your own lets you **compose** small functions into pipelines, **curry** multi-argument functions, and wrap behavior like logging or retries around existing code.

NOTES

- Callbacks are functions passed to be called later
- Pure functions (no side effects) compose most safely
- Partial application fixes some arguments up front
- `bind` can partially apply too: `fn.bind(null, a)`

TAKING AND RETURNING FUNCTIONS

```
const nums = [1, 2, 3, 4];
nums.filter(n => n % 2).map(n => n * 10);   // [10, 30]

const twice = fn => x => fn(fn(x));
twice(n => n + 3)(1);                       // 7

const compose = (...fns) => x => fns.reduceRight((v, f) => f(v), x);
const pipe = (...fns) => x => fns.reduce((v, f) => f(v), x);
const slug = pipe(s => s.trim(), s => s.toLowerCase(),
                  s => s.replaceAll(" ", "-"));
slug("  Hello World ");                     // "hello-world"

const curry = fn => a => b => fn(a, b);
const add = curry((a, b) => a + b);
add(2)(3);                                  // 5

const withLog = fn => (...args) => {
  console.log("call", args);
  return fn(...args);
};
```

CODE EXPLAINED

- → `twice` — takes a function, returns a new one
- → `pipe` — runs left to right; `compose` right to left
- → Curry — one argument at a time
- → Wrappers add behavior without changing the original

TYPESCRIPT

- Function types: `type Fn = (x: number) => number`
- Typing generic `pipe` / `compose` fully needs overloads; libraries like Remeda provide them

BUILT-IN HIGHER-ORDER FUNCTIONS

FUNCTION	TAKES
`arr.map(fn)`	transform each item
`arr.filter(fn)`	predicate
`arr.reduce(fn, init)`	reducer
`arr.sort(fn)`	comparator
`setTimeout(fn, ms)`	callback
`el.addEventListener(e, fn)`	handler

TERMS

TERM	MEANING
`callback`	function passed in
`compose`	combine f(g(x))
`curry`	f(a, b) → f(a)(b)
`partial`	fix some args
`pure`	same input → same output

⚠ COMMON MISTAKES

- Passing `fn()` (the result) when the API expects `fn` (the function)
- Passing `parseInt` straight to `map`: `["1","2","3"].map(parseInt)` gives `[1, NaN, NaN]`
- Over-abstracting simple code into point-free pipelines nobody can read

✓ TIPS & BEST PRACTICES

- Write `.map(s => parseInt(s, 10))` to control which arguments are passed
- Keep each function in a pipeline small and pure
- Name intermediate functions when a pipeline grows beyond 3 steps

QUICK SUMMARY

- higher-order — takes or returns functions
- `pipe / compose` — chain functions
- curry — one argument at a time
- wrappers — add behavior

Recursion

OVERVIEW

A **recursive** function calls itself on a smaller piece of the problem until it reaches a **base case** that returns without recursing. Recursion fits naturally shaped problems: trees, nested data, and divide-and-conquer algorithms. JavaScript engines (except Safari) don't do tail-call optimization, so very deep recursion overflows the stack.

NOTES

- Every recursive function needs a base case
- Each call adds a stack frame; depth limits are around 10,000
- Overflow throws `RangeError: Maximum call stack size exceeded`
- Memoize recursive functions that repeat work (like naive fibonacci)

CLASSIC RECURSION

```
function factorial(n) {
  if (n <= 1) return 1;              // base case
  return n * factorial(n - 1);       // recursive case
}
factorial(5);                        // 120

const fib = (n, memo = {}) =>
  n < 2 ? n : (memo[n] ??= fib(n - 1, memo) + fib(n - 2, memo));
fib(50);                             // 12586269025

function sumTree(node) {             // nested data
  if (!node) return 0;
  return node.value + (node.children ?? []).reduce((t, c) => t + sumTree(c), 0);
}

const flatten = arr =>
  arr.reduce((out, x) => out.concat(Array.isArray(x) ? flatten(x) : x), []);
flatten([1, [2, [3, [4]]]]);         // [1, 2, 3, 4]

// factorial(1e5);                   // RangeError: call stack exceeded
```

CODE EXPLAINED

- → Base case stops the recursion
- → Memo object — each `fib(n)` computed once
- → Tree walks recurse into each child
- → Use loops or explicit stacks for very deep inputs

TYPESCRIPT

- Recursive functions often need an explicit return type, or TS reports the type as implicitly `any`
- Recursive *types* work too: `type Tree = { value: number; children?: Tree[] }`

PROBLEM SHAPES

PROBLEM	BASE CASE
factorial	n ≤ 1
tree sum	node is null
flatten	item isn't an array
binary search	range empty
permutations	array length ≤ 1

RECURSION VS LOOP

	RECURSION	LOOP
Readability	natural for trees	natural for lists
Stack risk	yes	no
Speed	slower	faster
State	in parameters	in variables

⚠ COMMON MISTAKES

- Missing or unreachable base case, causing infinite recursion
- Naive `fib(n-1) + fib(n-2)` without memoization: exponential time
- Recursing over user-sized data (deep JSON, long lists) and overflowing the stack

✓ TIPS & BEST PRACTICES

- Write and test the base case first
- Convert to an explicit stack (`while (stack.length)`) for unbounded depth
- Use `arr.flat(Infinity)` instead of writing your own flatten

QUICK SUMMARY

- base case — stops recursion
- recursive case — smaller input
- memoize — avoid repeated work
- no TCO — watch depth

Arrays — Basics

[] · length · index · at() · Array.from · Array.of · holes · nested arrays

OVERVIEW

Arrays are ordered, zero-indexed lists that can hold any mix of types and grow or shrink freely. `length` is always one more than the highest index. Create them with literals `[]`, `Array.from()` (from iterables or with a mapping function), or `Array.of()`. Arrays are objects, so they're copied by reference.

NOTES

- `typeof []` is `"object"`; use `Array.isArray`
- Setting `arr.length = 0` empties the array
- Holes (`[1, , 3]`) behave inconsistently; avoid them
- `new Array(3)` makes 3 holes, not `[3]`

CREATING AND ACCESSING ARRAYS

```
const fruits = ["apple", "banana", "cherry"];
fruits[0];                    // "apple"
fruits.at(-1);                // "cherry"
fruits.length;                // 3
fruits[5];                    // undefined

fruits.push("date");          // add to end → length 4
fruits[1] = "blueberry";      // replace

Array.from("abc");            // ["a", "b", "c"]
Array.from({ length: 3 }, (_, i) => i * 2);  // [0, 2, 4]
Array.of(7);                  // [7]
new Array(3);                 // [ <3 empty items> ]
Array(3).fill(0);             // [0, 0, 0]

const grid = Array.from({ length: 2 }, () => Array(3).fill(0));
grid[1][2] = 5;               // [[0,0,0],[0,0,5]]

const copy = [...fruits];     // shallow copy
copy === fruits;              // false
```

CODE EXPLAINED

- → `at(-1)` — last element without `length - 1`
- → `Array.from({ length: n }, fn)` — build arrays of computed values
- → `fill` — set every slot to one value
- → Build 2D arrays with `Array.from`, not `fill([])`

TYPESCRIPT

- Type as `string[]` or `Array<string>`; `readonly string[]` blocks mutation
- Tuples fix length and position types: `[string, number]`

CREATION

CODE	RESULT
`[]`	empty array
`[1, "a", true]`	mixed types
`Array.from("hi")`	["h", "i"]
`Array.from(new Set(x))`	array from Set
`Array(3).fill("-")`	["-", "-", "-"]
`Array.of(1, 2)`	[1, 2]

ACCESS

CODE	GETS
`a[0]`	first
`a.at(-1)`	last
`a[a.length - 1]`	last (old style)
`a.length`	count
`a.slice(-2)`	last two

⚠ COMMON MISTAKES

- `Array(3).fill([])` puts the **same** inner array in every slot
- `new Array(5)` when you meant `[5]`
- Assigning `b = a` and thinking it's a copy; both names point to one array

✓ TIPS & BEST PRACTICES

- Prefer literals and `Array.from` over the `Array` constructor
- Use `const` for arrays; you can still push to them
- Check emptiness with `arr.length === 0`

QUICK SUMMARY

- `[]` — literal, zero-indexed
- `at(-1)` — negative index
- `Array.from` — from iterables or generated
- reference type — copy with spread

Array Methods — Mutating

OVERVIEW

These methods **change the original array** in place. `push`/`pop` work at the end, `shift`/`unshift` at the start (slower on large arrays), and `splice` inserts or removes anywhere. `sort` and `reverse` also mutate and return the same array. ES2023 added non-mutating twins: `toSorted`, `toReversed`, `toSpliced`.

NOTES

- `push` / `unshift` return the new length
- `pop` / `shift` return the removed item (or `undefined`)
- `splice` returns an array of removed items
- Mutating arrays held in React/Vue state won't trigger re-renders

MUTATING METHODS WITH RESULTS

```
const a = [1, 2, 3];
a.push(4, 5);          // 5     a = [1,2,3,4,5]
a.pop();               // 5     a = [1,2,3,4]
a.unshift(0);          // 5     a = [0,1,2,3,4]
a.shift();             // 0     a = [1,2,3,4]

const b = ["a", "b", "c", "d"];
b.splice(1, 2);        // ["b","c"]  b = ["a","d"]
b.splice(1, 0, "x");   // []         b = ["a","x","d"]  (insert)
b.splice(-1, 1, "z");  // ["d"]      b = ["a","x","z"]  (replace)

[3, 1, 2].sort();      // [1, 2, 3]
[10, 9, 1].sort();     // [1, 10, 9]   (string sort!)
[10, 9, 1].sort((x, y) => x - y);  // [1, 9, 10]
[1, 2, 3].reverse();   // [3, 2, 1]
[0, 0, 0].fill(7, 1);  // [0, 7, 7]
[1, 2, 3, 4].copyWithin(0, 2);  // [3, 4, 3, 4]
```

CODE EXPLAINED

- → `splice(start, deleteCount, ...items)` — remove and/or insert
- → Negative `start` counts from the end
- → `sort()` with no comparator sorts as strings
- → `fill(value, start, end)` — overwrite a range

TYPESCRIPT

- None of these are allowed on a `readonly T[]` or `ReadonlyArray<T>`
- `pop()` / `shift()` return `T | undefined`

MUTATING METHODS

METHOD	RETURNS	EFFECT
`push(...x)`	new length	add to end
`pop()`	removed item	remove last
`unshift(...x)`	new length	add to start
`shift()`	removed item	remove first
`splice(i, n, ...x)`	removed items	remove/insert
`sort(fn)`	same array	sort in place
`reverse()`	same array	reverse in place

NON-MUTATING TWINS

MUTATES	COPY VERSION
`sort()`	`toSorted()`
`reverse()`	`toReversed()`
`splice()`	`toSpliced()`
`a[i] = v`	`with(i, v)`

⚠ COMMON MISTAKES

- Sorting numbers without a comparator
- Calling `sort()` on a shared array and surprising other code that uses it
- Using `shift` in a loop on huge arrays; it's O(n) each time

✓ TIPS & BEST PRACTICES

- Use the `to…` versions whenever you need to keep the original
- Remove by value: `arr.splice(arr.indexOf(x), 1)` (check for -1 first)
- Use `push(...items)` to append several values at once

QUICK SUMMARY

- `push / pop` — end
- `unshift / shift` — start
- `splice` — anywhere
- `toSorted / toReversed` — copies

Array Methods — Transform

OVERVIEW

Transform methods return a **new** array (or value) and leave the original alone. `map` transforms each item, `filter` keeps items that pass a test, and `reduce` folds everything into a single value. `flat`/`flatMap` remove nesting, and `slice`/`concat` copy and combine. Chain them for readable data pipelines.

NOTES

- Callbacks receive `(item, index, array)`
- `map` always returns an array of the same length
- `reduce` without an initial value throws on an empty array
- Holes are skipped by most of these methods

TRANSFORM PIPELINES

```
const orders = [
  { item: "pen", qty: 3, price: 1.5 },
  { item: "pad", qty: 0, price: 4 },
  { item: "ink", qty: 2, price: 6 },
];

orders.map(o => o.item);                    // ["pen", "pad", "ink"]
orders.filter(o => o.qty > 0).length;       // 2
orders.reduce((t, o) => t + o.qty * o.price, 0);   // 16.5

const byItem = orders.reduce((acc, o) => {
  acc[o.item] = o.qty; return acc;
}, {});                                     // { pen: 3, pad: 0, ink: 2 }

[[1, 2], [3, [4]]].flat();                  // [1, 2, 3, [4]]
[[1, 2], [3, [4]]].flat(Infinity);          // [1, 2, 3, 4]
["a b", "c"].flatMap(s => s.split(" "));    // ["a", "b", "c"]
[1, 2, 3, 4].slice(1, 3);                   // [2, 3]
[1].concat([2], 3);                         // [1, 2, 3]
[1, 2, 3].with(1, 9);                       // [1, 9, 3]
```

CODE EXPLAINED

- → `reduce(fn, init)` — always pass `init`
- → Reduce to an object — build lookups or counts
- → `flat(depth)` — `Infinity` flattens everything
- → `flatMap` — map then flatten one level

TYPESCRIPT

- `reduce` often needs a typed initial value: `reduce<Record<string, number>>(fn, {})`
- `filter` with a type predicate narrows: `.filter((x): x is string => x != null)`

TRANSFORM METHODS

METHOD	RETURNS
`map(fn)`	array, same length
`filter(fn)`	array of matches
`reduce(fn, init)`	single value
`flat(depth)`	flattened array
`flatMap(fn)`	mapped + flat(1)
`slice(a, b)`	copied section
`concat(...x)`	combined array

RECIPES

GOAL	CODE
sum	`a.reduce((t, n) => t + n, 0)`
remove falsy	`a.filter(Boolean)`
pluck field	`a.map(o => o.id)`
count by	`Object.groupBy(a, fn)`
max	`Math.max(...a)`

⚠ COMMON MISTAKES

- Using `map` for side effects and ignoring the result; use `forEach` or `for...of`
- Omitting `reduce`'s initial value and crashing on empty arrays
- Forgetting to `return` the accumulator in a `reduce` with a block body

✓ TIPS & BEST PRACTICES

- Filter before map to do less work
- Prefer clear `for...of` loops over clever `reduce` calls
- Use `Object.groupBy` (sheet 055) instead of hand-written grouping reduces

QUICK SUMMARY

- `map` — transform each
- `filter` — keep matches
- `reduce` — fold to one value
- `flat / flatMap` — remove nesting

Array Methods — Search & Test

find · findIndex · findLast · includes · indexOf · some · every

OVERVIEW

Search and test methods answer questions about an array without changing it. `find`/`findLast` return the matching **item**, `findIndex`/`findLastIndex` return its **position**, `includes` checks for a value, and `some`/`every` return booleans. All the callback-based ones stop as soon as the answer is known.

NOTES

- `find` returns `undefined` when nothing matches; `findIndex` returns `-1`
- `includes` uses SameValueZero, so it finds `NaN`; `indexOf` doesn't
- `includes` and `indexOf` compare objects by reference
- `every` on an empty array is `true`

SEARCHING ARRAYS

```
const users = [
  { id: 1, name: "Ana", admin: false },
  { id: 2, name: "Bo",  admin: true  },
  { id: 3, name: "Cy",  admin: true  },
];

users.find(u => u.admin);             // { id: 2, name: "Bo", … }
users.findLast(u => u.admin);         // { id: 3, name: "Cy", … }
users.findIndex(u => u.id === 3);     // 2
users.find(u => u.id === 9);          // undefined

users.some(u => u.admin);             // true
users.every(u => u.name.length > 1); // true
[].every(x => false);                 // true (vacuous)

const nums = [1, 2, NaN, 2];
nums.includes(2);                     // true
nums.indexOf(2);                      // 1
nums.lastIndexOf(2);                  // 3
nums.includes(NaN);                   // true
nums.indexOf(NaN);                    // -1
```

CODE EXPLAINED

- → `find` vs `filter` — first match vs all matches
- → `findLast` — searches from the end
- → `some` — at least one; `every` — all
- → Use `includes` for simple value membership

TYPESCRIPT

- `find` returns `T | undefined`, so handle the miss before using the result
- A type-predicate callback narrows: `find((x): x is Admin => x.admin)`

SEARCH METHODS

METHOD	RETURNS	MISS
`find(fn)`	item	undefined
`findLast(fn)`	item	undefined
`findIndex(fn)`	index	-1
`findLastIndex(fn)`	index	-1
`indexOf(v)`	index	-1
`includes(v)`	boolean	false
`some(fn) / every(fn)`	boolean	false / true

WHICH TO USE

QUESTION	METHOD
is x in the list?	includes
which item matches?	find
where is it?	findIndex
any match?	some
all match?	every

⚠ COMMON MISTAKES

- Using `filter(...)[0]` instead of `find`; it scans the whole array
- Testing `if (arr.indexOf(x))`, which is false when x is at index 0
- Searching for an object with `includes(obj)` using a different but equal object

✓ TIPS & BEST PRACTICES

- Use a `Set` for repeated membership checks on large lists
- Use `findLast` instead of `reverse().find()`, which mutates
- Combine with `??`: `users.find(fn)?.name ?? "none"`

QUICK SUMMARY

- `find` — first matching item
- `findIndex` — its position
- `includes` — value present?
- `some / every` — any / all

Objects — Basics

OVERVIEW

Objects are collections of **key–value pairs**. Keys are strings or symbols; values can be anything, including functions (methods). Access properties with dot notation for fixed names and **bracket notation** for dynamic or unusual keys. Modern literals support shorthand properties, shorthand methods, and computed keys.

NOTES

- Number keys are converted to strings: `o[1]` is `o["1"]`
- Missing properties read as `undefined`, not an error
- Objects are passed and compared by reference
- Key order: integer-like keys ascending, then strings in insertion order

OBJECT LITERAL FEATURES

```
const name = "Ana", field = "role";

const user = {
  name,                          // shorthand: name: name
  age: 30,
  "first-name": "Ana",           // quoted key
  [field]: "admin",              // computed key → role
  greet() { return `Hi ${this.name}`; },  // method shorthand
};

user.age;                        // 30
user["first-name"];              // "Ana"
user[field];                     // "admin"
user.email;                      // undefined

user.email = "a@b.co";           // add
user.age += 1;                   // update
delete user["first-name"];       // remove

"age" in user;                   // true
Object.hasOwn(user, "greet");    // true
```

CODE EXPLAINED

→ Shorthand `name` — uses a variable of the same name

→ `[field]` — key computed at runtime

→ Brackets — for variables and keys with dashes or spaces

→ `in` / `Object.hasOwn` — check existence

TYPESCRIPT

- Describe shapes with `type` or `interface` (sheet 115); adding unknown keys becomes an error
- Use `Record<string, T>` for dictionary-style objects

DOT VS BRACKET

KEY	DOT	BRACKET
`name`	o.name	o["name"]
`first-name`	✖	o["first-name"]
`variable k`	✖	o[k]
`1`	✖	o[1]
`symbol s`	✖	o[s]

OPERATIONS

TASK	CODE
`add / update`	`o.k = v`
`remove`	`delete o.k`
`exists (own)`	`Object.hasOwn(o, "k")`
`count keys`	`Object.keys(o).length`
`safe read`	`o?.a?.b`

⚠ COMMON MISTAKES

- Using `o.k` when the key is in a variable `k`; that reads a property literally named "k"
- Comparing objects with `===` expecting a deep comparison
- Using objects as maps with user-supplied keys like `__proto__`; use `Map`

✓ TIPS & BEST PRACTICES

- Use shorthand properties and methods; they're cleaner and standard
- Use `Map` when keys are dynamic, non-string, or added and removed often
- Freeze config objects with `Object.freeze`

QUICK SUMMARY

- `{ key: value }` — literal
- `o.k` / `o[k]` — dot vs dynamic
- `[expr]: v` — computed key
- `in` / `hasOwn` — existence

Object Static Methods

OVERVIEW

The `Object` constructor has static methods for working with objects as data. `keys`, `values`, and `entries` turn an object into arrays; `fromEntries` turns pairs back into an object. `assign` copies properties, `freeze`/`seal` lock objects down, and `groupBy` builds grouped objects from arrays.

NOTES

- These only include **own, enumerable, string** keys
- `freeze` is shallow: nested objects can still change
- `Object.groupBy` returns a null-prototype object (ES2024)
- `entries` + `map` + `fromEntries` transforms objects

OBJECT STATIC METHODS

```
const prices = { apple: 1.2, pear: 0.8, fig: 2 };

Object.keys(prices);          // ["apple", "pear", "fig"]
Object.values(prices);        // [1.2, 0.8, 2]
Object.entries(prices);       // [["apple",1.2], ["pear",0.8], ["fig",2]]

const doubled = Object.fromEntries(
  Object.entries(prices).map(([k, v]) => [k, v * 2])
);                            // { apple: 2.4, pear: 1.6, fig: 4 }

Object.assign({}, prices, { fig: 3 });  // copy + override
const cfg = Object.freeze({ port: 80 });
cfg.port = 1;                 // ignored (throws in strict mode)
Object.isFrozen(cfg);         // true

const people = [{ n: "Ana", t: "a" }, { n: "Bo", t: "b" }, { n: "Cy", t: "a" }];
Object.groupBy(people, p => p.t);
// { a: [{n:"Ana"…}, {n:"Cy"…}], b: [{n:"Bo"…}] }
```

CODE EXPLAINED

- → `entries` → transform → `fromEntries` — map over an object
- → `assign(target, ...sources)` — mutates and returns target
- → `freeze` — no adds, deletes, or changes (top level)
- → `groupBy(arr, fn)` — key returned by `fn` becomes a group

TYPESCRIPT

- `Object.keys` returns `string[]`, not `(keyof T)[]`, because objects may have extra keys at runtime
- `Object.entries(o)` types values but keys stay `string`

STATIC METHODS

METHOD	RETURNS
`keys(o)`	string[]
`values(o)`	value[]
`entries(o)`	[key, value][]
`fromEntries(pairs)`	object
`assign(t, ...s)`	target
`groupBy(arr, fn)`	{ key: items[] }
`getOwnPropertyNames(o)`	incl. non-enumerable

LOCKING OBJECTS

METHOD	ADD	DELETE	CHANGE
`preventExtensions`	✖	✔	✔
`seal`	✖	✖	✔
`freeze`	✖	✖	✖

⚠ COMMON MISTAKES

- Assuming `freeze` is deep; nested objects stay mutable
- Using `Object.assign(target, …)` and accidentally mutating the target
- Expecting symbol or inherited keys in `Object.keys`

✓ TIPS & BEST PRACTICES

- Use spread `{ ...a, ...b }` instead of `Object.assign({}, a, b)`
- Use `Map.groupBy` when group keys aren't strings
- Write a small `deepFreeze` helper for config constants

QUICK SUMMARY

- `keys / values / entries` — to arrays
- `fromEntries` — back to object
- `freeze` — shallow lock
- `groupBy` — group array items

Destructuring

OVERVIEW

Destructuring unpacks values from arrays (by position) and objects (by key) into variables in one statement. It supports defaults, renaming, nesting, and rest. It's most useful in function parameters, loop heads, and when working with returned arrays or objects.

NOTES

- Defaults apply only when the value is `undefined`
- Destructuring `null` or `undefined` throws a TypeError
- Skip array items with empty commas: `[, second]`
- Assignment without `const` needs parentheses for objects: `({ a } = o)`

ARRAY AND OBJECT DESTRUCTURING

```
const [a, b] = [10, 20];                // a=10, b=20
const [first, , third] = [1, 2, 3];     // skip one
const [x = 5, y = 7] = [1];             // x=1, y=7
const [head, ...tail] = [1, 2, 3];      // 1, [2, 3]
let p = 1, q = 2; [p, q] = [q, p];      // swap

const user = { id: 7, name: "Ana", address: { city: "Rome" } };
const { id, name } = user;              // 7, "Ana"
const { name: userName } = user;        // rename → "Ana"
const { role = "guest" } = user;        // default
const { address: { city } } = user;     // nested → "Rome"
const { id: _, ...rest } = user;        // rest → { name, address }

function show({ name, age = 0 } = {}) { return `${name} ${age}`; }
for (const [key, value] of Object.entries(user)) {}
const { data: { items = [] } = {} } = response;  // safe nested
```

CODE EXPLAINED

- → Arrays — by position; objects — by key name
- → `name: userName` — read `name`, store as `userName`
- → Nested patterns follow the shape of the data
- → `= {}` defaults keep nested destructuring from throwing

TYPESCRIPT

- Annotate the whole pattern: `const { id, name }: User = user`
- In parameters: `function show({ name, age = 0 }: Props)`

ARRAY PATTERNS

PATTERN	FROM [1, 2, 3]
`[a]`	a = 1
`[, b]`	b = 2
`[a, ...r]`	r = [2, 3]
`[a, b, c, d = 0]`	d = 0

OBJECT PATTERNS

PATTERN	DOES
`{ k }`	read k
`{ k: v }`	rename to v
`{ k = 1 }`	default
`{ k: { j } }`	nested
`{ k, ...r }`	rest of keys

⚠ COMMON MISTAKES

- Destructuring a value that can be `null`; add a default (`= {}`) or check first
- Starting a line with `{ a } = obj` without `const` or parentheses (parsed as a block)
- Confusing rename and default: `{ a: b = 1 }` renames to `b` with default 1

✓ TIPS & BEST PRACTICES

- Destructure options in function parameters for named arguments
- Keep nested destructuring to two levels; deeper gets hard to read
- Use rest to omit fields: `const { password, ...safe } = user`

QUICK SUMMARY

- `[a, b] = arr` — by position
- `{ a, b } = obj` — by key
- `{ a: x = 1 }` — rename + default
- `...rest` — the remainder

Copying & Equality

OVERVIEW

Assigning an object or array copies the **reference**, not the data. Spread (`{...o}`, `[...a]`) and `Object.assign` make **shallow** copies: the top level is new but nested objects are shared. `structuredClone` makes a true **deep** copy, including Maps, Sets, Dates, and circular references.

NOTES

- Primitives are always copied by value
- `structuredClone` can't copy functions, DOM nodes, or class prototypes
- The JSON round-trip loses Dates, `undefined`, Maps, and Sets
- Equality (`===`) is by reference for objects

REFERENCE, SHALLOW, DEEP

```
const a = { n: 1, tags: ["x"] };
const ref = a;                        // same object
ref.n = 2;  a.n;                      // 2

const shallow = { ...a };
shallow.n = 3;      a.n;              // 2  (top level copied)
shallow.tags.push("y"); a.tags;       // ["x", "y"]  (shared!)

const deep = structuredClone(a);
deep.tags.push("z"); a.tags;          // ["x", "y"]  (independent)

const withDate = { d: new Date() };
structuredClone(withDate).d instanceof Date;        // true
JSON.parse(JSON.stringify(withDate)).d;             // string!

// structuredClone({ fn() {} });      // DataCloneError

{ a: 1 } === { a: 1 };                // false (different objects)
JSON.stringify(x) === JSON.stringify(y);  // crude deep equality
```

CODE EXPLAINED

→ Reference copy — two names, one object

→ Shallow copy — nested arrays and objects still shared

→ `structuredClone` — independent deep copy

→ JSON round-trip — only for plain JSON-safe data

TYPESCRIPT

- `Readonly<T>` and `as const` give immutability at type level; `ReadonlyDeep` from type-fest goes deeper
- `structuredClone<T>(x)` returns the same type `T`

COPY METHODS

METHOD	DEPTH	NOTES
`b = a`	none	same object
`{ ...a } / [...a]`	shallow	fast, common
`Object.assign({}, a)`	shallow	like spread
`arr.slice()`	shallow	arrays only
`structuredClone(a)`	deep	Maps, Dates, cycles
`JSON round-trip`	deep	lossy

STRUCTUREDCLONE SUPPORT

TYPE	CLONED?
`objects / arrays`	✔
`Date, RegExp, Map, Set`	✔
`circular refs`	✔
`functions`	✖ throws
`class instances`	becomes plain object

⚠ COMMON MISTAKES

- Copying with spread and then mutating a nested array, changing the original
- Using the JSON round-trip on data with Dates or `undefined`
- Comparing objects with `===` to check if their contents match

✓ TIPS & BEST PRACTICES

- Update nested state immutably: `{ ...s, user: { ...s.user, name } }`
- Use `structuredClone` by default when you need a real copy
- Use a library like `fast-deep-equal` or Node's `util.isDeepStrictEqual` for deep comparison

QUICK SUMMARY

- `=` — copies the reference
- spread — shallow copy
- `structuredClone` — deep copy
- `===` — reference equality

Map & WeakMap

OVERVIEW

A Map is a key–value collection where keys can be **any type** (objects, functions, numbers) and insertion order is preserved. It has a proper size, is directly iterable, and is faster than plain objects for frequent adds and deletes. A WeakMap holds object keys **weakly**, letting them be garbage-collected.

NOTES

- Keys compare with SameValueZero: `NaN` works as a key
- Object keys match by reference, not contents
- `map[key] = v` sets an object property, not a Map entry
- WeakMap isn't iterable and has no `size`

MAP AND WEAKMAP

```
const m = new Map();
m.set("a", 1).set(42, "num");       // set is chainable
const objKey = { id: 1 };
m.set(objKey, "object key");

m.get("a");                         // 1
m.get({ id: 1 });                   // undefined (different object)
m.get(objKey);                      // "object key"
m.has(42);                          // true
m.size;                             // 3
m.delete("a");                      // true

const scores = new Map([["ana", 9], ["bo", 7]]);
for (const [name, s] of scores) {}  // insertion order
[...scores.keys()];                 // ["ana", "bo"]
new Map(Object.entries({ x: 1 }));  // object → Map
Object.fromEntries(scores);         // Map → { ana: 9, bo: 7 }
Map.groupBy([1, 2, 3], n => n % 2 ? "odd" : "even");

const meta = new WeakMap();
meta.set(domNode, { clicks: 0 });   // freed with the node
```

CODE EXPLAINED

- → `set` returns the Map, so calls chain
- → Object keys — must be the same reference to match
- → Convert with `Object.entries` and `Object.fromEntries`
- → WeakMap — attach data to objects without leaking memory

TYPESCRIPT

- Type as `Map<string, number>`; `get` returns `V | undefined`
- `WeakMap<object, V>` keys must be objects (or non-registered symbols)

MAP API

METHOD	RETURNS
`set(k, v)`	the map
`get(k)`	value | undefined
`has(k)`	boolean
`delete(k)`	boolean
`clear()`	undefined
`size`	count
`keys / values / entries`	iterators

MAP VS OBJECT

	MAP	OBJECT
`Key types`	any	string / symbol
`Size`	m.size	Object.keys(o).length
`Iterable`	yes	no (use entries)
`Frequent add/delete`	faster	slower
`JSON`	convert first	native

⚠ COMMON MISTAKES

- Using bracket syntax (`m["a"] = 1`) on a Map; it bypasses the Map entirely
- Expecting `JSON.stringify(map)` to work; it gives `{}`
- Using a new object literal to look up an object key

✓ TIPS & BEST PRACTICES

- Use Map for caches, counters, and lookups keyed by objects or IDs
- Count with `m.set(k, (m.get(k) ?? 0) + 1)`
- Use WeakMap for per-object metadata like DOM node state

QUICK SUMMARY

- `Map` — any key type, ordered
- `set / get / has / delete` — API
- `size` — entry count
- `WeakMap` — GC-friendly object keys

Set & WeakSet

OVERVIEW

A `Set` stores **unique** values of any type in insertion order. It's the fastest way to remove duplicates and to check membership. ES2025 added set algebra methods: `union`, `intersection`, `difference`, `symmetricDifference`, and subset checks. A `WeakSet` holds objects weakly.

NOTES

- Uniqueness uses SameValueZero (`NaN` equals `NaN`)
- Objects are unique by reference
- `has` is O(1), unlike `array.includes`
- Set methods accept any set-like object (has `size`, `has`, `keys`)

SET OPERATIONS

```
const s = new Set([1, 2, 2, 3]);     // Set {1, 2, 3}
s.add(4).add(1);                     // 1 ignored (duplicate)
s.has(2);                            // true
s.delete(3);                         // true
s.size;                              // 3

[...new Set(["a", "b", "a"])];       // ["a", "b"]  dedupe

const a = new Set([1, 2, 3]);
const b = new Set([2, 3, 4]);
a.union(b);                          // Set {1, 2, 3, 4}
a.intersection(b);                   // Set {2, 3}
a.difference(b);                     // Set {1}
a.symmetricDifference(b);            // Set {1, 4}
new Set([2]).isSubsetOf(a);          // true
a.isDisjointFrom(new Set([9]));      // true

for (const v of a) {}                // insertion order
const seen = new WeakSet();
seen.add(obj); seen.has(obj);        // true
```

CODE EXPLAINED

→ `add` ignores values already present

→ Spread into an array to dedupe

→ Set methods return **new** Sets

→ WeakSet — tag objects without preventing GC

TYPESCRIPT

- Type as `Set<string>`; the set methods need `"lib": ["ES2025"]` or `"ESNext"`
- `new Set(arr)` infers its element type from the array

SET API

METHOD	RETURNS
`add(v)`	the set
`has(v)`	boolean
`delete(v)`	boolean
`clear()`	undefined
`size`	count
`values() / keys()`	iterator

SET ALGEBRA (ES2025)

METHOD	RESULT
`union(b)`	in a or b
`intersection(b)`	in both
`difference(b)`	in a, not b
`symmetricDifference(b)`	in exactly one
`isSubsetOf(b)`	all of a in b
`isSupersetOf(b)`	all of b in a

⚠ COMMON MISTAKES

- Expecting two equal-looking objects to dedupe; they're different references
- Using `array.includes` in a loop over large data instead of a Set
- Calling set methods in old runtimes without checking support

✓ TIPS & BEST PRACTICES

- Convert to a Set once, then call `has` many times
- Use `new Set(arr).size !== arr.length` to detect duplicates
- Dedupe objects by key with a Map: `[...new Map(a.map(o => [o.id, o])).values()]`

QUICK SUMMARY

- `Set` — unique values, ordered
- `has` — fast membership
- `[...new Set(a)]` — dedupe
- `union / intersection / difference` — algebra

Sorting & Searching

OVERVIEW

`sort()` sorts **in place** and, without a comparator, converts items to strings and sorts by UTF-16 order, which breaks numbers. Pass a **comparator** `(a, b) => number`: negative puts a first, positive puts b first. Sorting is stable since ES2019. For a sorted copy, use `toSorted()`.

NOTES

- `a - b` sorts numbers ascending; `b - a` descending
- `localeCompare` sorts strings like a human would
- Stable sort keeps the original order for equal items
- Binary search only works on sorted data

SORTING AND SEARCHING

```
[10, 1, 5].sort();                          // [1, 10, 5]  wrong
[10, 1, 5].sort((a, b) => a - b);           // [1, 5, 10]
[10, 1, 5].toSorted((a, b) => b - a);       // [10, 5, 1] (copy)

["b", "a", "C"].sort();                     // ["C", "a", "b"]
["b", "a", "C"].sort((a, b) => a.localeCompare(b)); // ["a", "b", "C"]
["item10", "item2"].sort(new Intl.Collator(undefined,
  { numeric: true }).compare);              // ["item2", "item10"]

const people = [{ n: "Bo", age: 30 }, { n: "Ana", age: 30 }, { n: "Cy", age: 25 }];
people.toSorted((a, b) => a.age - b.age || a.n.localeCompare(b.n));
// Cy 25, Ana 30, Bo 30   (age, then name)

function binarySearch(arr, x) {
  let lo = 0, hi = arr.length - 1;
  while (lo <= hi) {
    const mid = (lo + hi) >> 1;
    if (arr[mid] === x) return mid;
    arr[mid] < x ? (lo = mid + 1) : (hi = mid - 1);
  }
  return -1;
}
```

CODE EXPLAINED

- → Comparator sign decides the order
- → `Intl.Collator({ numeric: true })` — natural sort for "item2" < "item10"
- → `||` chains tie-breakers: first key, then second
- → Binary search — O(log n) on sorted arrays

TYPESCRIPT

- Comparator type is `(a: T, b: T) => number`; returning a boolean is a type error
- `toSorted` needs lib ES2023 or later

COMPARATORS

SORT BY	COMPARATOR
numbers ↑	(a, b) => a - b
numbers ↓	(a, b) => b - a
strings	(a, b) => a.localeCompare(b)
dates	(a, b) => a - b
field	(a, b) => a.age - b.age
booleans first	(a, b) => b.ok - a.ok

SEARCHING

NEED	USE
unsorted, once	find / indexOf
many lookups	Set / Map
sorted, large	binary search
min / max	Math.min(...a)

⚠ COMMON MISTAKES

- Sorting numbers with no comparator
- Returning `a > b` (a boolean) from a comparator; it must return a number
- Sorting a shared array in place and changing someone else's data

✓ TIPS & BEST PRACTICES

- Use `toSorted` unless you specifically want in-place mutation
- Create one `Intl.Collator` and reuse its `compare` for big sorts
- Sort by several keys with `||` between comparisons

QUICK SUMMARY

- `sort(fn)` — in place, comparator required
- `toSorted(fn)` — sorted copy
- `localeCompare` — string order
- binary search — sorted data only

Iterators & Iterables

OVERVIEW

An **iterable** is any object with a `[Symbol.iterator]()` method that returns an **iterator**: an object whose `next()` returns `{ value, done }`. Arrays, strings, Maps, Sets, and NodeLists are all iterable, which is why `for...of`, spread, and destructuring work on them. ES2025 **iterator helpers** add lazy `map`, `filter`, `take`, and more.

NOTES

- Iterators are usually single-use
- Iterator helpers are lazy: nothing runs until you consume
- `Iterator.from(x)` wraps any iterable or iterator
- Generators (sheet 062) are the easy way to write iterators

ITERATOR PROTOCOL AND HELPERS

```
const it = [10, 20][Symbol.iterator]();
it.next();                          // { value: 10, done: false }
it.next();                          // { value: 20, done: false }
it.next();                          // { value: undefined, done: true }

const range = {
  from: 1, to: 3,
  [Symbol.iterator]() {
    let cur = this.from, last = this.to;
    return { next: () => cur <= last
      ? { value: cur++, done: false } : { value: undefined, done: true } };
  },
};
[...range];                         // [1, 2, 3]
for (const n of range) {}           // 1, 2, 3

const firstEvens = new Set([1, 2, 3, 4, 5, 6]).values()
  .filter(n => n % 2 === 0)
  .map(n => n * 10)
  .take(2)
  .toArray();                       // [20, 40]
```

CODE EXPLAINED

→ `next()` — returns the next `{ value, done }`

→ `[Symbol.iterator]()` — makes your object work with `for...of`

→ Helpers chain lazily on any iterator

→ `toArray()` — consumes the iterator into an array

TYPESCRIPT

- Type with `Iterable<T>`, `Iterator<T>`, and `IterableIterator<T>`
- Iterator helpers need lib `ESNext` (or `ES2025`) in tsconfig

WHAT USES ITERABLES

SYNTAX	EXAMPLE
`for...of`	`for (x of it)`
spread	`[...it]`
destructuring	`const [a, b] = it`
`Array.from`	`Array.from(it)`
`new Map / Set`	`new Set(it)`
`Promise.all`	`Promise.all(it)`

ITERATOR HELPERS

METHOD	DOES
`map / filter`	lazy transform
`take(n) / drop(n)`	limit / skip
`flatMap`	map + flatten
`reduce / some / every / find`	consume
`toArray()`	collect
`forEach`	consume with side effects

⚠ COMMON MISTAKES

- Iterating the same iterator twice and getting nothing the second time
- Calling iterator helpers on an array (arrays have their own `map`); call `.values()` first
- Building infinite iterators and spreading them; it never ends

✓ TIPS & BEST PRACTICES

- Use iterator helpers to process large or infinite sequences without temporary arrays
- Return `this` from `[Symbol.iterator]()` to make an iterator iterable too
- Prefer generators over hand-written `next()` methods

QUICK SUMMARY

- iterable — has `[Symbol.iterator]`
- iterator — `next()` → `{ value, done }`
- `for...of` / spread — consume iterables
- helpers — lazy map/filter/take

Generators

OVERVIEW

A **generator function** (`function*`) returns an iterator that runs the function body lazily. Each `yield` pauses execution and hands out a value; the next `next()` call resumes right after it. Generators make custom iterators, lazy pipelines, and infinite sequences short and readable. `async function*` yields values over time.

NOTES

- Calling a generator function doesn't run it; it returns a generator object
- `yield*` delegates to another iterable
- `return` ends the generator; its value has `done: true` and is skipped by `for...of`
- `next(value)` sends a value back in as the result of `yield`

GENERATORS

```
function* count(to) {
  for (let i = 1; i <= to; i++) yield i;
}
[...count(3)];                      // [1, 2, 3]

function* ids() {                   // infinite, lazy
  let id = 0;
  while (true) yield ++id;
}
const gen = ids();
gen.next().value;                   // 1
gen.next().value;                   // 2

function* both() { yield* [1, 2]; yield* count(2); }  // delegate
[...both()];                        // [1, 2, 1, 2]

const obj = { *[Symbol.iterator]() { yield "a"; yield "b"; } };
[...obj];                           // ["a", "b"]

async function* pages(url) {        // async generator
  while (url) { const r = await (await fetch(url)).json();
                yield r.items; url = r.next; }
}
```

CODE EXPLAINED

→ `yield` — pause and output a value

→ Infinite generators are fine; consume with care

→ `yield*` — yield every value of another iterable

→ `*[Symbol.iterator]()` — generator method makes an object iterable

TYPESCRIPT

- Type as `Generator<T, TReturn, TNext>`, or let TS infer it from `yield`
- Async generators are `AsyncGenerator<T>`; consume with `for await`

GENERATOR OBJECT

CALL	EFFECT
`g.next()`	run to next yield
`g.next(v)`	resume, yield returns v
`g.return(v)`	finish early
`g.throw(err)`	throw at the yield
`[...g]`	collect remaining

GOOD USES

USE	EXAMPLE
`custom iteration`	tree traversal
`lazy sequences`	ids(), fib()
`pagination`	async function*
`chunking`	yield arr.slice(i, i+n)

⚠ COMMON MISTAKES

- Calling `count(3)` and expecting it to run; you must iterate it
- Spreading an infinite generator
- Reusing a finished generator; create a new one with another call

✓ TIPS & BEST PRACTICES

- Combine generators with iterator helpers: `ids().take(5).toArray()`
- Use `async function*` for paginated APIs and streams
- Write recursive tree walks as `yield* walk(child)`

QUICK SUMMARY

- `function*` — lazy iterator factory
- `yield` — pause and output
- `yield*` — delegate
- `async function*` — values over time

Prototypes

OVERVIEW

Every object has a hidden link to another object, its **prototype**. When a property isn't found on the object itself, JavaScript looks up the **prototype chain** until it finds it or reaches `null`. Classes are built on this mechanism: methods live once on the prototype and are shared by every instance.

NOTES

- `Object.getPrototypeOf(o)` reads the link; avoid `__proto__`
- `Object.create(proto)` makes an object with a chosen prototype
- `Object.create(null)` makes an object with no prototype at all
- Functions have a `.prototype` property used by `new`

THE PROTOTYPE CHAIN

```
const animal = { eats: true, walk() { return `${this.name} walks`; } };
const dog = Object.create(animal);   // dog → animal → Object.prototype
dog.name = "Rex";
dog.walk();                          // "Rex walks" (found on animal)
dog.eats;                            // true (inherited)
Object.hasOwn(dog, "eats");          // false
Object.getPrototypeOf(dog) === animal;  // true

function Person(name) { this.name = name; }   // constructor function
Person.prototype.hi = function () { return `Hi, ${this.name}`; };
const p = new Person("Ana");
p.hi();                              // "Hi, Ana"
p.hi === new Person("Bo").hi;        // true (shared)

class Cat { meow() {} }
typeof Cat;                          // "function"
Object.getPrototypeOf(new Cat()) === Cat.prototype;  // true

const dict = Object.create(null);    // no inherited keys
"toString" in dict;                  // false
```

CODE EXPLAINED

→ Lookup walks the chain until a match or `null`

→ Writing a property always creates it on the object itself

→ `new F()` links the new object to `F.prototype`

→ Classes are syntax on top of constructor functions

TYPESCRIPT

- Prefer classes; TS can't easily type methods added to `Function.prototype` by hand
- Augmenting built-in prototypes needs declaration merging (sheet 128)

PROTOTYPE API

CALL	DOES
`Object.getPrototypeOf(o)`	read prototype
`Object.setPrototypeOf(o, p)`	change it (slow)
`Object.create(p, props)`	new object with prototype p
`p.isPrototypeOf(o)`	p in o's chain?
`o instanceof F`	F.prototype in chain?

CHAIN FOR [1, 2]

LEVEL	PROVIDES
`[1, 2]`	0, 1, length
`Array.prototype`	map, push, …
`Object.prototype`	toString, hasOwnProperty
`null`	end of chain

⚠ COMMON MISTAKES

- Adding methods to built-in prototypes like `Array.prototype`; it breaks other code and future standards
- Using `__proto__` in new code; it's legacy
- Putting arrays or objects on a prototype, so every instance shares one mutable value

✓ TIPS & BEST PRACTICES

- Use `class` syntax for new code; know prototypes to debug it
- Use `Object.create(null)` or `Map` for dictionaries with arbitrary keys
- Inspect chains in DevTools by expanding `[[Prototype]]`

QUICK SUMMARY

- prototype — fallback object
- chain — lookup ends at null
- `Object.create(p)` — set prototype
- classes — prototypes underneath

Classes & Objects

OVERVIEW

A **class** is a template for creating objects with shared methods. Define it with `class Name { … }`, create instances with `new`, and put shared behavior in methods. Class bodies always run in strict mode. Unlike function declarations, classes aren't hoisted, so define them before use.

NOTES

- Class names use PascalCase
- Methods are non-enumerable and live on the prototype
- Calling a class without `new` throws a TypeError
- Classes can be expressions: `const A = class {}`

A COMPLETE MINIMAL CLASS

```
class BankAccount {
  owner;                              // public field
  balance = 0;                        // field with default

  constructor(owner) {
    this.owner = owner;
  }

  deposit(amount) {
    if (amount <= 0) throw new RangeError("amount must be > 0");
    this.balance += amount;
    return this;                      // allows chaining
  }

  toString() { return `${this.owner}: $${this.balance}`; }
}

const acct = new BankAccount("Ana");
acct.deposit(50).deposit(25);
acct.balance;                         // 75
`${acct}`;                            // "Ana: $75"
acct instanceof BankAccount;          // true
// BankAccount("x");                  // TypeError: needs new
```

CODE EXPLAINED

- → Fields — declared at the top, set before the constructor body runs
- → `constructor` — runs once per `new`
- → Methods — shared through the prototype
- → `return this` — enables method chaining

TYPESCRIPT

- Fields must be declared with types: `balance: number = 0`
- `strictPropertyInitialization` errors if a field isn't set in the constructor or with a default

CLASS MEMBERS

MEMBER	SYNTAX
`field`	x = 0;
`constructor`	constructor(a) {}
`method`	m() {}
`getter / setter`	get x() {} / set x(v) {}
`static`	static create() {}
`private`	#secret = 1;

CLASS FACTS

QUESTION	ANSWER
`typeof Class`	"function"
`hoisted?`	no (TDZ)
`strict mode?`	always
`call without new?`	TypeError
`methods enumerable?`	no

⚠ COMMON MISTAKES

- Forgetting `this.` inside methods and reading a global or undeclared variable
- Passing a method as a callback and losing `this` (sheet 047)
- Using a class when a plain function or object would do

✓ TIPS & BEST PRACTICES

- Implement `toString()` for readable logging and template output
- Keep constructors simple; put logic in methods or static factories
- Validate inputs in the constructor so invalid instances never exist

QUICK SUMMARY

- `class X {}` — define
- `new X()` — instantiate
- fields + methods — state + behavior
- not hoisted — define before use

Constructors

OVERVIEW

The `constructor` runs when you call `new`. Its job is to set up the instance from its arguments. Field initializers run **before** the constructor body. In a subclass you must call `super(...)` before touching `this`. JavaScript allows only one constructor per class, so "overloads" use defaults, options objects, or static factory methods.

NOTES

- A class without a constructor gets a default one
- Subclass default constructor passes all args to `super`
- Returning an object from a constructor replaces the instance (rarely useful)
- `new.target` tells you which class was actually instantiated

CONSTRUCTORS, ORDER, FACTORIES

```
class Point {
  x = 0; y = 0;                          // 1. fields initialize
  constructor(x, y) {                    // 2. then constructor body
    this.x = x ?? this.x;
    this.y = y ?? this.y;
  }
  static fromArray([x, y]) { return new Point(x, y); }   // factory
  static origin() { return new Point(); }
}
new Point(3, 4);                         // Point { x: 3, y: 4 }
Point.fromArray([1, 2]);                 // Point { x: 1, y: 2 }

class Point3D extends Point {
  z;
  constructor(x, y, z = 0) {
    super(x, y);                         // must come before this
    this.z = z;
  }
}

class Shape {
  constructor() {
    if (new.target === Shape) throw new TypeError("abstract");
  }
}
```

CODE EXPLAINED

→ Fields first, then constructor body

→ Static factories — named alternatives to overloaded constructors

→ `super(...)` — runs the parent constructor

→ `new.target` — fake abstract classes in plain JS

TYPESCRIPT

- **Parameter properties** declare and assign in one step: `constructor(private x: number) {}`
- Real overloads are possible with multiple signatures above one implementation

INITIALIZATION ORDER (SUBCLASS)

STEP	RUNS
1	subclass constructor called
2	super() → parent fields init
3	parent constructor body
4	subclass fields init
5	rest of subclass constructor

OVERLOAD ALTERNATIVES

TECHNIQUE	EXAMPLE
`defaults`	constructor(x = 0)
`options object`	constructor({ x, y })
`static factory`	Point.fromArray(a)
`type check`	typeof a === "string"

⚠ COMMON MISTAKES

- Using `this` before `super()` in a subclass: ReferenceError
- Doing async work in a constructor; constructors can't be `async`
- Defining two constructors; only the last one survives (actually a SyntaxError)

✓ TIPS & BEST PRACTICES

- Use `static async create()` when setup needs `await`
- Prefer field initializers for defaults; the constructor stays short
- Name factories by intent: `fromJSON`, `parse`, `empty`

QUICK SUMMARY

- `constructor` — runs on `new`
- fields before body
- `super()` first in subclasses
- static factories — named constructors

Fields, Static & Private Members

OVERVIEW

Class members come in three flavors. **Instance fields** belong to each object. **Static** members belong to the class itself and are shared. **Private** members start with # and are truly hidden: code outside the class body can't read them, and trying is a SyntaxError. **Static blocks** run once when the class is defined.

NOTES

- `#private` works for fields, methods, getters, and statics
- `#x in obj` checks whether obj has that private field
- Static members are inherited by subclasses
- Private names don't show in `Object.keys` or JSON

INSTANCE, STATIC, PRIVATE

```
class Counter {
  static #instances = 0;              // private static
  static MAX = 100;                   // public static
  #count = 0;                         // private instance field
  label;                              // public instance field

  static { console.log("ready"); }    // static block: runs once

  constructor(label) {
    this.label = label;
    Counter.#instances++;
  }
  #clamp(n) { return Math.min(n, Counter.MAX); }  // private method
  inc() { this.#count = this.#clamp(this.#count + 1); return this.#count; }
  static get created() { return Counter.#instances; }
  static isCounter(o) { return #count in o; }     // brand check
}

const c = new Counter("clicks");
c.inc();                              // 1
Counter.created;                      // 1
// c.#count;                          // SyntaxError
Counter.isCounter(c);                 // true
```

CODE EXPLAINED

- → `#field` — only accessible inside the class body
- → `static` — accessed through the class name
- → `static { }` — one-time class setup
- → `#x in o` — safe check for real instances

TYPESCRIPT

- TS `private` / `protected` are compile-time only; at runtime the field is public. `#private` is enforced at runtime
- `readonly` blocks reassignment after construction

MEMBER KINDS

KIND	SYNTAX	ACCESSED VIA
`instance field`	x = 1	this.x
`static field`	static x = 1	Class.x
`private field`	#x = 1	this.#x
`private method`	#m() {}	this.#m()
`static block`	static { }	runs once

WHEN TO USE

USE	FOR
`instance`	per-object state
`static`	factories, constants, counters
`#private`	internal state and helpers
`static #`	class-wide hidden state

⚠ COMMON MISTAKES

- Using `_underscore` names and assuming they're private
- Accessing static members through `this` in instance methods (it's the instance, not the class)
- Proxies or `structuredClone` on objects with `#private` fields; they don't carry over

✓ TIPS & BEST PRACTICES

- Default to `#private` for anything callers shouldn't touch
- Use static methods for alternate constructors
- Use `#x in obj` instead of `instanceof` for brand checks across realms

QUICK SUMMARY

- fields — per instance
- `static` — on the class
- `#private` — runtime-enforced privacy
- `static { }` — class init block

Getters & Setters

get · set · computed properties · Object.defineProperty · validation in setters

OVERVIEW

Getters and **setters** look like plain properties to callers but run code when read or written. Use them for computed values, validation, and keeping a stable public API while the internal representation changes. Pair a setter with a private field to store the real value.

NOTES

- A getter without a setter makes a read-only property
- Writing to a getter-only property is ignored (throws in strict mode)
- Getters live on the prototype, so they're not own properties
- `Object.defineProperty` creates them on existing objects

GET, SET, DEFINEPROPERTY

```
class Temperature {
  #celsius = 0;
  get celsius() { return this.#celsius; }
  set celsius(v) {
    if (v < -273.15) throw new RangeError("below absolute zero");
    this.#celsius = v;
  }
  get fahrenheit() { return this.#celsius * 9 / 5 + 32; }   // computed
  set fahrenheit(f) { this.celsius = (f - 32) * 5 / 9; }
}

const t = new Temperature();
t.celsius = 25;           // runs setter
t.fahrenheit;             // 77 (runs getter)
t.fahrenheit = 212;
t.celsius;                // 100

const user = { first: "Ana", last: "Diaz",
  get full() { return `${this.first} ${this.last}`; } };
user.full;                // "Ana Diaz"

Object.defineProperty(user, "id", { value: 7, writable: false });
user.id = 9;  user.id;    // 7
```

CODE EXPLAINED

→ Setter validation keeps invalid values out

→ Computed getter — derived on every read

→ Object literals support `get` / `set` too

→ `defineProperty` — fine-grained control (writable, enumerable)

TYPESCRIPT

- Getter and setter types can differ (TS 5.1+): accept `string | number`, return `number`
- A getter with no setter is inferred as `readonly`

PROPERTY DESCRIPTOR

KEY	MEANING
`value`	stored value
`writable`	can be reassigned
`enumerable`	shows in keys/for…in
`configurable`	can be deleted/redefined
`get / set`	accessor functions

GOOD USES

USE	EXAMPLE
`computed value`	get area()
`validation`	set age(v)
`read-only view`	get items() → copy
`lazy value`	compute on first read

⚠ COMMON MISTAKES

- Setter that assigns to its own name (`set x(v) { this.x = v }`): infinite recursion
- Expensive work in getters that look cheap to callers
- Expecting getters to appear in `JSON.stringify` of class instances; they don't

✓ TIPS & BEST PRACTICES

- Store backing values in `#private` fields
- Return copies from getters that expose internal arrays
- Add a `toJSON()` method to include computed values in JSON

QUICK SUMMARY

- `get x()` — computed read
- `set x(v)` — validated write
- getter only — read-only
- `defineProperty` — descriptors

Inheritance

OVERVIEW

`extends` creates a subclass that inherits the parent's fields and methods. Override a method by defining one with the same name; call the parent version with `super.method()`. Subclass constructors must call `super()` first. You can also extend built-ins like `Error`, `Array`, and `Map`.

NOTES

- JavaScript supports single inheritance only
- Static methods are inherited too
- `super.method()` works in methods and object literals
- `instanceof` is true for the class and every ancestor

EXTENDING AND OVERRIDING

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { return `${this.name} makes a sound`; }
  static create(name) { return new this(name); }   // this = called class
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);                        // parent constructor
    this.breed = breed;
  }
  speak() {                             // override
    return `${super.speak()} — woof`;
  }
}

const d = new Dog("Rex", "lab");
d.speak();                              // "Rex makes a sound — woof"
d instanceof Animal;                    // true
Dog.create("Fido") instanceof Dog;      // true (inherited static)

class Stack extends Array {
  peek() { return this.at(-1); }
}
const s = Stack.from([1, 2]);
s.peek();                               // 2
s.map(x => x) instanceof Stack;         // true
```

CODE EXPLAINED

- → `super(name)` — sets up the parent part of the object
- → `super.speak()` — reuse the parent method
- → `new this()` in statics — creates the subclass when called on it
- → Extending `Array` keeps array methods returning your type

TYPESCRIPT

- Mark overrides with `override`; turn on `noImplicitOverride` to require it
- `abstract` classes and methods force subclasses to implement them (sheet 121)

INHERITANCE SYNTAX

SYNTAX	DOES
`class B extends A`	B inherits A
`super(args)`	call A's constructor
`super.m()`	call A's method
`m() {} in B`	override A.m
`static m() in A`	inherited by B

EXTENDING BUILT-INS

CLASS	GOOD FOR
`Error`	custom errors
`Array`	collection helpers
`Map`	typed caches
`EventTarget`	custom event emitters
`HTMLElement`	web components

⚠ COMMON MISTAKES

- Deep inheritance chains that are hard to change; prefer composition
- Forgetting `super()` in a subclass constructor
- Overriding a method and accidentally dropping important parent behavior

✓ TIPS & BEST PRACTICES

- Keep hierarchies shallow, two levels if possible
- Call `super.method()` when extending rather than replacing behavior
- Use `new this()` in static factories so subclasses get the right type

QUICK SUMMARY

- `extends` — inherit
- `super()` — parent constructor
- `super.m()` — parent method
- override — same method name

Polymorphism, Duck Typing & Mixins

OVERVIEW

Polymorphism means different objects respond to the same method call in their own way. JavaScript uses **duck typing**: if an object has the method you need, you can use it, regardless of its class. **Mixins** copy reusable behavior into classes, and **composition** (objects holding other objects) is often simpler than inheritance.

NOTES

- Check capabilities, not classes: `typeof x.render === "function"`
- Mixin functions take a base class and return an extended class
- Order of mixins matters when methods collide
- Composition: "has a" instead of "is a"

DUCK TYPING, MIXINS, COMPOSITION

```
class Circle { constructor(r) { this.r = r; }
               area() { return Math.PI * this.r ** 2; } }
class Square { constructor(s) { this.s = s; }
               area() { return this.s ** 2; } }
const total = shapes => shapes.reduce((t, s) => t + s.area(), 0);
total([new Circle(1), new Square(2), { area: () => 10 }]); // ≈17.14

const Serializable = Base => class extends Base {
  toJSON() { return { type: this.constructor.name, ...this }; }
};
const Timestamped = Base => class extends Base {
  createdAt = new Date();
};
class User { constructor(n) { this.name = n; } }
class Model extends Serializable(Timestamped(User)) {}
JSON.stringify(new Model("Ana"));  // {"type":"Model","name":"Ana",…}

class Car {                        // composition
  constructor() { this.engine = new Engine(); }
  start() { return this.engine.start(); }   // delegate
}
```

CODE EXPLAINED

- → Any object with `area()` works in `total`, even a plain literal
- → Mixin = function from class to class
- → Stack mixins by nesting calls
- → Composition delegates work to contained objects

TYPESCRIPT

- **Structural typing** is duck typing at compile time: any object matching an interface's shape is accepted
- Mixins need a constructor type: `type Ctor = new (...a: any[]) => {}`

REUSE STRATEGIES

STRATEGY	RELATIONSHIP	BEST WHEN
`inheritance`	is a	true hierarchy
`mixin`	can do	shared capability
`composition`	has a	swappable parts
`duck typing`	acts like	loose protocols

CAPABILITY CHECKS

CHECK	CODE
`has method`	`typeof o.m === "function"`
`iterable`	`Symbol.iterator in o`
`thenable`	`typeof o?.then === "function"`
`has key`	`"k" in o`

⚠ COMMON MISTAKES

- Checking `instanceof` when any object with the right method should be accepted
- Mixins that overwrite each other's methods silently
- Inheriting just to reuse one method; compose or extract a function instead

✓ TIPS & BEST PRACTICES

- Favor composition over inheritance for flexible designs
- Document the "protocol" (methods expected) for duck-typed APIs
- Keep mixins small and focused on one capability

QUICK SUMMARY

- polymorphism — same call, different behavior
- duck typing — check for the method
- mixins — class → class functions
- composition — has-a over is-a

Protocol Methods & Well-Known Symbols

OVERVIEW

JavaScript has no operator overloading, but objects can hook into language behavior with special methods. `toString` and `valueOf` control conversion, `toJSON` controls serialization, and **well-known symbols** like `Symbol.iterator`, `Symbol.toPrimitive`, and `Symbol.dispose` plug objects into `for...of`, type coercion, and `using` declarations.

NOTES

- `Symbol.toPrimitive` overrides both `toString` and `valueOf`
- Its `hint` argument is `"number"`, `"string"`, or `"default"`
- `toJSON` is called by `JSON.stringify` automatically
- These are the closest JS equivalent to Python dunder methods

HOOKING INTO THE LANGUAGE

```
class Money {
  constructor(cents) { this.cents = cents; }
  [Symbol.toPrimitive](hint) {
    const d = this.cents / 100;
    return hint === "number" ? d : `$${d.toFixed(2)}`;
  }
  toJSON() { return { amount: this.cents / 100, currency: "USD" }; }
}
const m = new Money(1250);
`${m}`;                          // "$12.50"
+m;  m > 10;                     // 12.5, true (number hint)
JSON.stringify(m);               // '{"amount":12.5,"currency":"USD"}'

class Range {
  constructor(a, b) { this.a = a; this.b = b; }
  *[Symbol.iterator]() {
    for (let i = this.a; i <= this.b; i++) yield i;
  }
}
[...new Range(1, 3)];            // [1, 2, 3]

class TempFile { [Symbol.dispose]() { console.log("cleaned"); } }
{ using f = new TempFile(); }    // "cleaned" at block end
```

CODE EXPLAINED

- → `toPrimitive` — one method for all conversions
- → `toJSON` — custom JSON shape
- → `*[Symbol.iterator]` — make instances iterable
- → `Symbol.dispose` + `using` — automatic cleanup

TYPESCRIPT

- Implement interfaces to document the protocol: `class Range implements Iterable<number>`
- `using` and `Symbol.dispose` are supported since TS 5.2 (`Disposable` interface)

PROTOCOL METHODS

METHOD	TRIGGERED BY
`toString()`	String(x), templates
`valueOf()`	math, comparisons
`toJSON()`	JSON.stringify
`[Symbol.toPrimitive]`	any coercion
`[Symbol.iterator]`	for…of, spread
`[Symbol.dispose]`	using block end

MORE WELL-KNOWN SYMBOLS

SYMBOL	CUSTOMIZES
`Symbol.hasInstance`	instanceof
`Symbol.toStringTag`	[object Tag]
`Symbol.asyncDispose`	await using

⚠ COMMON MISTAKES

- Defining `valueOf`, then `+` with strings surprises you
- Returning a string from `toJSON` when you meant an object
- Using `using` without checking runtime support

✓ TIPS & BEST PRACTICES

- Prefer `toPrimitive` over separate `toString` / `valueOf`
- Give classes a `toString` for readable logs
- Use generator methods for `Symbol.iterator`; it's the shortest form

QUICK SUMMARY

- `toString / valueOf` — conversion
- `toJSON` — serialization
- `Symbol.iterator` — iteration
- `Symbol.dispose` — cleanup with using

JSON

OVERVIEW

`JSON.stringify` turns values into a JSON string and `JSON.parse` turns JSON text back into values. Both take optional hooks: a **replacer** and indent for stringify, a **reviver** for parse. JSON supports only objects, arrays, strings, finite numbers, booleans, and `null`, so Dates, Maps, `undefined`, and BigInts need handling.

NOTES

- JSON keys must be double-quoted; no trailing commas, no comments
- `undefined`, functions, and symbols are dropped from objects (become `null` in arrays)
- `NaN` and `Infinity` become `null`
- `JSON.parse` throws `SyntaxError` on invalid input

ROUND-TRIP WITH HOOKS

```
const order = { id: 7, items: ["pen"], at: new Date(), note: undefined };

JSON.stringify(order);
// '{"id":7,"items":["pen"],"at":"2026-10-02T14:00:00.000Z"}'
JSON.stringify(order, null, 2);           // pretty, 2-space indent
JSON.stringify(order, ["id", "items"]);   // only these keys
JSON.stringify({ n: 1n }, (k, v) =>
  typeof v === "bigint" ? v.toString() : v);  // '{"n":"1"}'

const text = '{"id":7,"at":"2026-10-02T14:00:00.000Z"}';
JSON.parse(text).at;                      // string, not Date
const revived = JSON.parse(text, (k, v) =>
  k === "at" ? new Date(v) : v);
revived.at instanceof Date;               // true

try { JSON.parse("{bad}"); }
catch (e) { e.name; }                     // "SyntaxError"
```

CODE EXPLAINED

- → Indent argument — number of spaces or a string like `"\t"`
- → Array replacer — whitelist of keys
- → Function replacer/reviver — called for every key/value
- → Dates serialize via `toJSON` to ISO strings

TYPESCRIPT

- `JSON.parse` returns `any`, so validate it (Zod, sheet 085) or type it as `unknown` and narrow
- `JSON.stringify` returns `string` (or `undefined` for `undefined` input)

VALUE HANDLING

VALUE	STRINGIFY →
`Date`	ISO string
`undefined (prop)`	omitted
`undefined (array)`	null
`NaN / Infinity`	null
`Map / Set`	{}
`BigInt`	TypeError

SIGNATURES

CALL	ARGS
`JSON.stringify(v, r, s)`	value, replacer, space
`JSON.parse(t, rev)`	text, reviver
`obj.toJSON()`	custom output
`structuredClone(v)`	deep copy, no JSON

⚠ COMMON MISTAKES

- Parsing untrusted JSON without `try/catch`
- Expecting Dates to come back as Date objects after `parse`
- Using the JSON round-trip as a deep clone for data with Maps or Dates

✓ TIPS & BEST PRACTICES

- Pretty-print logs with `JSON.stringify(x, null, 2)`
- Convert Maps first: `Object.fromEntries(map)`
- Validate parsed data at boundaries (APIs, files, localStorage)

QUICK SUMMARY

- `JSON.stringify` — value → text
- `JSON.parse` — text → value
- replacer / reviver — custom handling
- lossy — Dates, Maps, undefined

Reading Files (Node)

OVERVIEW

Node reads files through the `node:fs/promises` module. `readFile` loads the whole file at once, which is simplest for small and medium files. For large files, read **line by line** with `readline` over a stream, or process chunks with `createReadStream`, so memory use stays flat.

NOTES

- Pass an encoding (`"utf8"`) to get a string; otherwise you get a Buffer
- Relative paths resolve from the **current working directory**, not the file
- Sync versions (`readFileSync`) block the event loop; fine for CLI startup
- Missing files throw an error with `code: "ENOENT"`

READING FILES

```ts
import { readFile } from "node:fs/promises";
import { createReadStream } from "node:fs";
import { createInterface } from "node:readline";

const text = await readFile("notes.txt", "utf8");   // whole file
const lines = text.split(/\r?\n/);

const cfg = JSON.parse(await readFile("config.json", "utf8"));

const rl = createInterface({ input: createReadStream("big.log") });
for await (const line of rl) {                      // one line at a time
  if (line.includes("ERROR")) console.log(line);
}

try {
  await readFile("missing.txt", "utf8");
} catch (err) {
  if (err.code === "ENOENT") console.log("no such file");
  else throw err;
}
```

CODE EXPLAINED

- → `readFile(path, "utf8")` — returns a string
- → `/\r?\n/` — handles Windows and Unix line endings
- → `readline` + stream — constant memory for huge files
- → `err.code` — check `ENOENT`, `EACCES`, etc.

TYPESCRIPT

- Install `@types/node`; `readFile` with an encoding is typed `Promise<string>`, without it `Promise<Buffer>`
- Catch variables are `unknown`; narrow with `err instanceof Error && "code" in err`

READING OPTIONS

METHOD	BEST FOR
`readFile`	small/medium files
`readFileSync`	CLI startup, config
`readline + stream`	line-by-line, huge files
`createReadStream`	piping, chunks
`fs.open + read`	random access

ERROR CODES

CODE	MEANING
`ENOENT`	file not found
`EACCES`	permission denied
`EISDIR`	path is a directory
`EMFILE`	too many open files

⚠ COMMON MISTAKES

- Forgetting the encoding and printing a Buffer
- Using relative paths that break when the script runs from another folder
- Loading multi-GB files with `readFile` and running out of memory

✓ TIPS & BEST PRACTICES

- Resolve paths from the module: `new URL("./data.txt", import.meta.url)`
- Use `fs.existsSync` sparingly; just try the read and handle `ENOENT`
- Use `readFile(path, { signal })` to make long reads cancelable

QUICK SUMMARY

- `readFile(p, "utf8")` — whole file
- `readline` — line by line
- streams — large files
- `ENOENT` — missing file

Sheet 073

Writing & Appending Files (Node)

writeFile · appendFile · flags · mkdir recursive · rm · createWriteStream

OVERVIEW

`writeFile` creates or **replaces** a file; `appendFile` adds to the end. Both create the file if it's missing but not its parent folders, so call `mkdir` with `recursive: true` first. For many writes or large output, use `createWriteStream`. `rm` deletes files and folders.

NOTES

- `writeFile` overwrites without warning
- Flag `"wx"` fails if the file already exists
- Writes aren't atomic; write to a temp file and `rename` for safety
- Call `stream.end()` to flush and close a write stream

WRITING, APPENDING, FOLDERS

```
import { writeFile, appendFile, mkdir, rm, rename } from "node:fs/promises";
import { createWriteStream } from "node:fs";

await mkdir("out/logs", { recursive: true });       // like mkdir -p
await writeFile("out/data.json", JSON.stringify(data, null, 2));
await appendFile("out/logs/app.log", `${new Date().toISOString()} start\n`);

await writeFile("out/once.txt", "x", { flag: "wx" }); // EEXIST if present

await writeFile("out/cfg.tmp", text);                 // atomic replace
await rename("out/cfg.tmp", "out/cfg.json");

const out = createWriteStream("out/report.csv");
out.write("id,name\n");
for (const u of users) out.write(`${u.id},${u.name}\n`);
out.end();                                            // flush + close

await rm("out/old", { recursive: true, force: true }); // like rm -rf
```

CODE EXPLAINED

- → `recursive: true` — creates all missing parent folders
- → `flag: "wx"` — exclusive create
- → Write temp then `rename` — readers never see half a file
- → `force: true` — no error if the path doesn't exist

TYPESCRIPT

- Same API; data can be `string`, `Buffer`, or a typed array
- Option objects are typed, so wrong flag names are caught

FILE FLAGS

FLAG	MEANING
w	write, truncate (default)
a	append
wx	write, fail if exists
r+	read/write, must exist
a+	read and append

OTHER OPERATIONS

FUNCTION	DOES
mkdir(p, {recursive})	create folders
rm(p, {recursive})	delete
rename(a, b)	move/rename
copyFile(a, b)	copy
readdir(p)	list folder
stat(p)	size, dates, isFile()

⚠ COMMON MISTAKES

- Writing objects directly; you get `[object Object]` (stringify first)
- Writing into a folder that doesn't exist (`ENOENT`)
- Running `rm` with `recursive: true` on a path built from user input

✓ TIPS & BEST PRACTICES

- Always end lines with `\n` when appending logs
- Use streams and `pipeline` for large or generated output
- Check `stat(p).isFile()` before deleting unknown paths

QUICK SUMMARY

- `writeFile` — create or replace
- `appendFile` — add to end
- `mkdir -p` — `recursive: true`
- temp + `rename` — atomic writes

File Paths (Node)

OVERVIEW

Build file paths with the `node:path` module instead of string concatenation; it handles separators on every OS. In ES modules there's no `__dirname`; use `import.meta.dirname` (Node 20.11+) or derive it from `import.meta.url`. Remember that relative paths resolve from the process's working directory.

NOTES

- `path.join` normalizes; `path.resolve` produces an absolute path
- `process.cwd()` is where the command was run
- `import.meta.filename` is this file's absolute path
- File URLs (`file:///…`) and paths are different; convert with `fileURLToPath`

PATH OPERATIONS

```
import path from "node:path";
import { fileURLToPath } from "node:url";

path.join("src", "lib", "..", "app.js");   // "src/app.js"
path.resolve("data", "a.txt");             // "/home/me/proj/data/a.txt"
path.basename("/a/b/photo.png");           // "photo.png"
path.basename("/a/b/photo.png", ".png");   // "photo"
path.extname("archive.tar.gz");            // ".gz"
path.dirname("/a/b/c.txt");                // "/a/b"
path.parse("/a/b/c.txt");
// { root: "/", dir: "/a/b", base: "c.txt", ext: ".txt", name: "c" }
path.relative("/a/b", "/a/c/d");           // "../c/d"
path.sep;                                  // "/" (or "\\" on Windows)

const here = import.meta.dirname;          // folder of this module
const data = path.join(here, "data.json"); // works from any cwd
const legacy = path.dirname(fileURLToPath(import.meta.url));
process.cwd();                             // where node was launched
```

CODE EXPLAINED

- → `join` — combine segments and normalize `..`
- → `resolve` — absolute path from cwd
- → `parse` — all parts at once
- → `import.meta.dirname` — ESM replacement for `__dirname`

TYPESCRIPT

- Identical in TypeScript; `import.meta.dirname` types need a recent `@types/node`
- Set `"module": "nodenext"` so `import.meta` is allowed

PATH API

FUNCTION	RETURNS
`join(...p)`	joined, normalized
`resolve(...p)`	absolute path
`basename(p, ext)`	file name
`dirname(p)`	folder
`extname(p)`	extension
`isAbsolute(p)`	boolean

WHERE AM I?

EXPRESSION	GIVES
`import.meta.dirname`	module folder
`import.meta.filename`	module file
`process.cwd()`	launch folder
`__dirname`	CommonJS only
`os.homedir()`	user home

⚠ COMMON MISTAKES

- Concatenating with `"/"`, which breaks on Windows
- Using `__dirname` in an ES module: ReferenceError
- Assuming relative paths are relative to the script file

✓ TIPS & BEST PRACTICES

- Anchor data files to `import.meta.dirname`
- Use `path.resolve` before comparing paths
- Guard against `../` traversal: check `resolved.startsWith(baseDir)`

QUICK SUMMARY

- `path.join` — combine safely
- `path.resolve` — absolute path
- `basename / extname` — parts
- `import.meta.dirname` — module folder

File & Blob APIs (Browser)

OVERVIEW

Browsers can't touch the file system directly, but users can hand you files through `<input type="file">` or drag and drop. Each is a `File` (a `Blob` with a name). Read contents with `file.text()`, `arrayBuffer()`, or `stream()`. To let users download generated data, wrap it in a `Blob` and link to `URL.createObjectURL`.

NOTES

- `input.files` is a `FileList`; convert with `Array.from`
- `accept=".csv,image/*"` filters the picker (not enforced)
- Call `URL.revokeObjectURL` when done to free memory
- `FileReader` is the older callback API; `blob.text()` is simpler

UPLOAD, READ, DOWNLOAD

```
// <input type="file" id="picker" accept=".csv" multiple>
picker.addEventListener("change", async () => {
  for (const file of picker.files) {
    file.name; file.size;            // "data.csv", 2048
    const text = await file.text();
    console.log(text.split("\n").length, "rows");
  }
});

dropZone.addEventListener("dragover", e => e.preventDefault());
dropZone.addEventListener("drop", async e => {
  e.preventDefault();
  const [file] = e.dataTransfer.files;
  img.src = URL.createObjectURL(file);     // preview an image
});

function download(name, content, type = "text/plain") {
  const url = URL.createObjectURL(new Blob([content], { type }));
  const a = Object.assign(document.createElement("a"),
    { href: url, download: name });
  a.click();
  URL.revokeObjectURL(url);
}
download("report.json", JSON.stringify(data), "application/json");
```

CODE EXPLAINED

- → `file.text()` — Promise of the contents as a string
- → `dragover` must call `preventDefault` or `drop` won't fire
- → Object URLs — temporary `blob:` links for previews
- → `download` attribute — saves instead of navigating

TYPESCRIPT

- `input.files` is `FileList | null`; check before looping
- Type the element with `querySelector<HTMLInputElement>(…)`

READING A BLOB/FILE

METHOD	RETURNS
`text()`	Promise<string>
`arrayBuffer()`	Promise<ArrayBuffer>
`bytes()`	Promise<Uint8Array>
`stream()`	ReadableStream

FILE PROPERTIES

PROPERTY	EXAMPLE
`name`	"photo.jpg"
`size`	348211 (bytes)
`type`	"image/jpeg"
`lastModified`	timestamp ms

⚠ COMMON MISTAKES

- Trusting `file.type` or `accept`; validate content on the server
- Never revoking object URLs, which leaks memory in long-lived pages
- Forgetting `preventDefault` on `dragover`

✓ TIPS & BEST PRACTICES

- Show file size with `Intl.NumberFormat` and `unit: "kilobyte"`
- Process huge files with `file.stream()` instead of `text()`
- Upload with `FormData` and `fetch` (sheet 102 / 103)

QUICK SUMMARY

- `<input type=file>` — get Files
- `file.text()` — read contents
- `Blob` + object URL — downloads
- revoke URLs when done

ES Modules

import · export · default vs named · as · import * as · re-export · import() · import.meta

OVERVIEW

ES modules are the standard module system. Each file has its own scope and shares values with `export` and `import`. Imports are static and hoisted, which lets tools tree-shake unused code. Use `import()` to load a module dynamically at runtime, which returns a Promise.

NOTES

- Imports are live, read-only bindings, not copies
- A module runs once, no matter how many files import it
- Browsers and Node need full paths with extensions: `./util.js`
- `import.meta` holds module info like `url`

EXPORTS AND IMPORTS

```
// math.js
export const PI = 3.14159;
export function area(r) { return PI * r * r; }
export default class Circle { /* … */ }

// app.js
import Circle, { PI, area } from "./math.js";    // default + named
import { area as circleArea } from "./math.js";   // rename
import * as math from "./math.js";               // namespace
math.area(2);                                    // 12.56636

export { area } from "./math.js";                // re-export
export * from "./shapes.js";                     // re-export all
export { default as Circle } from "./math.js";

const { default: Chart } = await import("./chart.js");  // dynamic
if (needsAdmin) await import("./admin.js");      // conditional load

import data from "./data.json" with { type: "json" };  // JSON module
import.meta.url;                                 // "file:///…/app.js"
```

CODE EXPLAINED

→ Named exports — imported by exact name in `{ }`

→ Default export — one per module, any name on import

→ `import * as ns` — all exports as an object

→ `import()` — lazy loading, code splitting

TYPESCRIPT

- `import type { User } from "./types.js"` imports only types; it's erased from output
- Enable `verbatimModuleSyntax` so type-only imports must say so explicitly

EXPORT FORMS

SYNTAX	KIND
`export const x = 1`	named
`export function f() {}`	named
`export { a, b as c }`	named list
`export default value`	default
`export * from "./m.js"`	re-export

IMPORT FORMS

SYNTAX	GETS
`import x from`	default
`import { a } from`	named
`import * as m from`	namespace
`import "./m.js"`	side effects only
`await import("./m.js")`	module object

⚠ COMMON MISTAKES

- Leaving off `.js` in Node or browser imports
- Mixing up default and named imports: `import { Circle }` when it's a default export
- Creating circular imports that read values before they're initialized

✓ TIPS & BEST PRACTICES

- Prefer named exports; they rename consistently and autocomplete better
- Use `index.js` barrel files sparingly; they can defeat tree-shaking
- Lazy-load heavy features with `import()`

QUICK SUMMARY

- `export` / `import` — share values
- default vs named — one vs many
- `import()` — dynamic, returns Promise
- static imports — tree-shakable

CommonJS & Interop

OVERVIEW

CommonJS is Node's original module system: `require()` loads modules synchronously and `module.exports` defines what a file exports. Many npm packages still use it. Node decides which system a `.js` file uses from the nearest `package.json` `"type"` field; `.mjs` and `.cjs` force ESM or CommonJS.

NOTES

- ESM can `import` CommonJS modules (default import = `module.exports`)
- Modern Node can `require()` ES modules without top-level await
- `__dirname`, `__filename`, `require` exist only in CommonJS
- `exports.x = …` works; reassigning `exports = …` doesn't

COMMONJS AND INTEROP

```
// math.cjs
function add(a, b) { return a + b; }
module.exports = { add, PI: 3.14 };      // or: exports.add = add;

// app.cjs
const { add } = require("./math.cjs");
const fs = require("node:fs");
console.log(__dirname, add(1, 2));       // 3

// app.mjs (ESM importing CommonJS)
import math from "./math.cjs";           // module.exports object
import { add } from "./math.cjs";        // named, when detectable
import { createRequire } from "node:module";
const require = createRequire(import.meta.url);
const legacy = require("./old.cjs");     // require inside ESM

// package.json
{ "type": "module",
  "exports": { "import": "./dist/index.js",
               "require": "./dist/index.cjs" } }
```

CODE EXPLAINED

→ `module.exports` — the object `require` returns

→ Default import of CJS — always works

→ `createRequire` — use `require` from ESM

→ Conditional `exports` — ship both formats

TYPESCRIPT

- Set `"module": "nodenext"` so TS follows Node's ESM/CJS rules exactly
- `esModuleInterop` makes `import x from "cjs-pkg"` work for CommonJS packages

ESM VS COMMONJS

	ESM	COMMONJS
syntax	import / export	require / module.exports
loading	async, static	sync, dynamic
top-level await	yes	no
__dirname	import.meta.dirname	yes
file ext	.mjs or type:module	.cjs or default

HOW NODE DECIDES

FILE	TREATED AS
.mjs	ESM
.cjs	CommonJS
.js + "type": "module"	ESM
.js, no type field	CommonJS

⚠ COMMON MISTAKES

- `exports = { … }` (reassigns the local variable, exports nothing)
- Using `require` in a `"type": "module"` project without `createRequire`
- Publishing packages without an `exports` map, causing dual-format confusion

✓ TIPS & BEST PRACTICES

- Write new code as ESM
- Use `.cjs` for config files that tools still load with `require`
- Check a package's `exports` field to see which formats it ships

QUICK SUMMARY

- `require` / `module.exports` — CommonJS
- `.mjs` / `.cjs` — force a format
- `"type": "module"` — .js is ESM
- `createRequire` — require from ESM

npm & package.json

OVERVIEW

npm installs packages into `node_modules` and records them in `package.json`. **Dependencies** ship with your app; **devDependencies** are for building and testing. The **lockfile** (`package-lock.json`) pins exact versions so every install is identical. Scripts in `package.json` are run with `npm run`.

NOTES

- Commit `package-lock.json`; never commit `node_modules`
- Use `npm ci` in CI: clean install exactly from the lockfile
- `^1.2.3` allows minor/patch updates; `~1.2.3` allows patch only
- `npx` runs a package's binary without installing it globally

EVERYDAY NPM

```
$ npm install express          # add dependency
$ npm i -D vitest eslint       # add devDependencies
$ npm uninstall lodash         # remove
$ npm ci                       # exact install from lockfile
$ npm outdated                 # what's behind
$ npm update                   # update within semver ranges
$ npm ls express               # where a package comes from
$ npm audit                    # known vulnerabilities
$ npm run build                # run a script
$ npx create-vite@latest app   # run without installing

// package.json
{
  "name": "my-app", "version": "1.0.0", "type": "module",
  "scripts": { "dev": "vite", "test": "vitest", "build": "vite build" },
  "dependencies":    { "express": "^5.1.0" },
  "devDependencies": { "vitest": "^3.2.0" },
  "engines": { "node": ">=22" }
}
```

CODE EXPLAINED

→ `-D` — saves to devDependencies

→ `npm ci` — fast, reproducible, deletes node_modules first

→ `scripts` — `npm test` and `npm start` work without `run`

→ `engines` — documents the required Node version

TYPESCRIPT

- Packages ship types via a `"types"` field, or you install them from `@types/*`: `npm i -D @types/node`
- Add `"typecheck": "tsc --noEmit"` to scripts

VERSION RANGES

RANGE	ALLOWS
`1.2.3`	exactly 1.2.3
`^1.2.3`	1.x.x ≥ 1.2.3
`~1.2.3`	1.2.x ≥ 1.2.3
`>=1.2 <2`	explicit range
`latest`	newest (avoid)

KEY PACKAGE.JSON FIELDS

FIELD	PURPOSE
`scripts`	named commands
`dependencies`	runtime packages
`devDependencies`	build/test tools
`exports`	public entry points
`type`	module or commonjs
`engines`	Node version

⚠ COMMON MISTAKES

- Installing build tools as regular dependencies, bloating production installs
- Deleting the lockfile to "fix" problems, which pulls in untested versions
- Running `npm install` in CI instead of `npm ci`

✓ TIPS & BEST PRACTICES

- Use `npm pkg set scripts.lint="eslint ."` to edit package.json safely
- Review `npm audit` results; not every advisory affects you
- Pin the package manager with the `packageManager` field and Corepack

QUICK SUMMARY

- `npm i` / `-D` — add deps
- `npm ci` — reproducible install
- lockfile — exact versions
- `npm run` — scripts

Math & Random

round / floor / ceil / trunc · min / max · abs · pow · random ints · crypto.randomUUID · getRandomValues

OVERVIEW

`Math` is a built-in object of static functions and constants: rounding, min/max, powers, roots, and trigonometry. `Math.random()` gives a float in [0, 1), which is fine for games and shuffles but **not** for security. For IDs, tokens, and passwords use the **Web Crypto** API: `crypto.randomUUID()` and `crypto.getRandomValues()`.

NOTES

- `Math.round` rounds .5 up toward +∞: `Math.round(-2.5)` is `-2`
- `Math.trunc` drops decimals; `Math.floor` rounds down
- `crypto` is global in browsers and modern Node
- Angles in trig functions are radians

MATH AND RANDOM

```
Math.round(4.5);       // 5        Math.round(-4.5);  // -4
Math.floor(-4.2);      // -5       Math.ceil(4.2);    // 5
Math.trunc(-4.7);      // -4       Math.sign(-3);     // -1
Math.abs(-7);          // 7        Math.max(3, 9, 1); // 9
Math.min(...[4, 2]);   // 2        Math.pow(2, 8);    // 256
Math.sqrt(81);         // 9        Math.cbrt(27);     // 3
Math.hypot(3, 4);      // 5        Math.log10(1000);  // 3
Math.PI;               // 3.141592653589793

const randInt = (min, max) =>                // inclusive
  Math.floor(Math.random() * (max - min + 1)) + min;
randInt(1, 6);                               // 1-6
const pick = arr => arr[Math.floor(Math.random() * arr.length)];

const clamp = (n, lo, hi) => Math.min(Math.max(n, lo), hi);
clamp(15, 0, 10);                            // 10

crypto.randomUUID();      // "3b241101-e2bb-4255-8caf-4136c566a962"
crypto.getRandomValues(new Uint32Array(1))[0];  // secure int
```

CODE EXPLAINED

- → `randInt` — the standard inclusive range formula
- → `clamp` — keep a value inside bounds
- → `randomUUID` — RFC 4122 v4 IDs
- → `getRandomValues` — cryptographically secure numbers

TYPESCRIPT

- Identical in TypeScript; `Math` functions take and return `number`
- `crypto.randomUUID()` is typed as a template literal type of the UUID shape

ROUNDING COMPARED

X	ROUND	FLOOR	CEIL	TRUNC
2.5	3	2	3	2
2.4	2	2	3	2
-2.5	-2	-3	-2	-2
-2.6	-3	-3	-2	-2

MORE MATH

FUNCTION	DOES
`Math.sin/cos/tan(rad)`	trig
`Math.atan2(y, x)`	angle of a point
`Math.log / exp`	natural log / e^x
`Math.fround(x)`	32-bit float
`Math.sumPrecise(arr)`	exact sum (new)

⚠ COMMON MISTAKES

- Using `Math.random()` for tokens, passwords, or session IDs
- `Math.round(x * 100) / 100` assuming exact decimal results; floats still apply
- Calling `Math.max()` on an empty array: it returns `-Infinity`

✓ TIPS & BEST PRACTICES

- Shuffle fairly with Fisher–Yates, not `sort(() => Math.random() - 0.5)`
- Convert degrees: `rad = deg * Math.PI / 180`
- Use `Number.EPSILON` for float comparisons

QUICK SUMMARY

- `round / floor / ceil / trunc` — rounding
- `min / max / abs / sqrt` — basics
- `Math.random` — not secure
- `crypto.randomUUID` — secure IDs

Date

OVERVIEW

`Date` stores a single moment as milliseconds since 1970-01-01 UTC. Getters and setters read and change parts in **local time** (or UTC with `getUTC…`). It's mutable, months are **0-based**, and parsing non-ISO strings is unreliable. For display, use `Intl.DateTimeFormat`; for serious date math, prefer Temporal (sheet 081).

NOTES

- `new Date()` is now; `Date.now()` is now in ms
- Months: 0 = January, 11 = December; days of week: 0 = Sunday
- ISO strings without a time zone are parsed as UTC (date-only) or local (date-time)
- Setters overflow: setting day 32 rolls into the next month

CREATING, READING, FORMATTING

```
const now = new Date();
Date.now();                              // 1790935200000 (ms)
new Date("2026-03-15T09:30:00Z");        // ISO, UTC
new Date(2026, 2, 15);                   // Mar 15 2026, local (month 2!)

const d = new Date(2026, 0, 31);
d.getFullYear();  d.getMonth();          // 2026, 0
d.getDate();      d.getDay();            // 31, 6 (Saturday)
d.setDate(d.getDate() + 1);              // Feb 1 (rolls over)
d.toISOString();                         // "2026-02-01T…Z"

const days = (b - a) / 86_400_000;       // difference in days

new Intl.DateTimeFormat("en-US", { dateStyle: "medium" }).format(d);
// "Feb 1, 2026"
d.toLocaleDateString("de-DE");           // "1.2.2026"
d.toLocaleTimeString("en-US", { timeStyle: "short" });  // "12:00 AM"
new Intl.RelativeTimeFormat("en", { numeric: "auto" })
  .format(-1, "day");                    // "yesterday"
```

CODE EXPLAINED

- → Month argument is 0-based; ISO strings use 1-based months
- → Subtracting Dates gives milliseconds
- → `toISOString` — always UTC, good for storage
- → `Intl.DateTimeFormat` — localized display

TYPESCRIPT

- `Date` is a class type; `getTime()` returns `number`
- `Intl` option objects are typed, so invalid `dateStyle` values are caught

GETTERS

METHOD	RANGE
`getFullYear()`	e.g. 2026
`getMonth()`	0–11
`getDate()`	1–31
`getDay()`	0 (Sun)–6
`getHours()`	0–23
`getTime()`	ms since epoch

DATESTYLE OUTPUT (EN-US)

STYLE	EXAMPLE
`short`	2/1/26
`medium`	Feb 1, 2026
`long`	February 1, 2026
`full`	Sunday, February 1, 2026

⚠ COMMON MISTAKES

- Forgetting months are 0-based when constructing dates
- Parsing strings like `"03/04/2026"`; the result depends on the engine
- Mutating a shared Date with setters and changing it everywhere

✓ TIPS & BEST PRACTICES

- Store and transmit dates as ISO strings or epoch ms
- Copy before changing: `const next = new Date(d)`
- Use `Intl.DateTimeFormat` with `timeZone` to show other zones

QUICK SUMMARY

- `new Date()` — now
- 0-based months
- `toISOString` — storage format
- `Intl.DateTimeFormat` — display

Temporal

OVERVIEW

Temporal is the modern replacement for `Date`. It provides **immutable** types for each kind of time value: `PlainDate` for calendar dates, `PlainTime` for wall-clock times, `ZonedDateTime` for a moment in a specific time zone, `Instant` for an exact point on the timeline, and `Duration` for lengths of time. Months are 1-based and arithmetic is DST-safe.

NOTES

- Every operation returns a new object; nothing mutates
- Check current browser/Node support; use the `@js-temporal/polyfill` otherwise
- `Temporal.Now` gives the current time in any form
- Convert from a legacy Date with `date.toTemporalInstant()`

TEMPORAL TYPES

```
const today = Temporal.Now.plainDateISO();           // 2026-10-02
const due = Temporal.PlainDate.from("2026-12-25");
today.until(due).days;                               // 84
due.add({ months: 1 }).toString();                   // "2027-01-25"
due.dayOfWeek;                                       // 5 (Friday, 1 = Mon)

const t = Temporal.PlainTime.from("09:30");
t.add({ minutes: 45 }).toString();                   // "10:15:00"

const meeting = Temporal.ZonedDateTime.from(
  "2026-03-08T01:30[America/Chicago]");
meeting.add({ hours: 1 }).toString();                // DST-aware
meeting.withTimeZone("Asia/Tokyo").toLocaleString();

const d = Temporal.Duration.from({ hours: 1, minutes: 90 });
d.round({ largestUnit: "hour" }).toString();         // "PT2H30M"

Temporal.PlainDate.compare(today, due);              // -1
today.equals(Temporal.PlainDate.from("2026-10-02")); // true
```

CODE EXPLAINED

- → `Plain…` — no time zone attached
- → `until` / `since` — durations between values
- → `ZonedDateTime` — handles DST transitions
- → `compare` — use for sorting

TYPESCRIPT

- Types are available with lib `esnext` or from the polyfill package
- Each Temporal class is its own type, so mixing `PlainDate` and `Instant` is a compile error

TEMPORAL TYPES

TYPE	REPRESENTS
`Instant`	exact moment (UTC)
`ZonedDateTime`	moment + time zone
`PlainDate`	calendar date
`PlainTime`	wall-clock time
`PlainDateTime`	date + time, no zone
`Duration`	length of time

TEMPORAL VS DATE

	TEMPORAL	DATE
`mutable`	no	yes
`months`	1-based	0-based
`time zones`	any IANA zone	local or UTC
`date-only type`	yes	no

⚠ COMMON MISTAKES

- Using `PlainDateTime` for events that happen at a real moment; use `ZonedDateTime`
- Comparing with `<`; use `Temporal.X.compare` or `equals`
- Shipping Temporal code without checking support or loading a polyfill

✓ TIPS & BEST PRACTICES

- Pick the most specific type: dates-only data should be `PlainDate`
- Store `Instant` or `ZonedDateTime.toString()`; both round-trip losslessly
- Format with `toLocaleString` or `Intl.DateTimeFormat`, which accept Temporal values

QUICK SUMMARY

- `PlainDate` — calendar date
- `ZonedDateTime` — real moment + zone
- `Duration` — lengths
- immutable — every op returns new

Testing with Vitest

OVERVIEW

Vitest is a fast test runner with a Jest-compatible API that works out of the box with ES modules and TypeScript. Group tests with `describe`, write them with `it` (or `test`), and assert with `expect`. `vi.fn()` creates mock functions and `vi.mock()` replaces whole modules.

NOTES

- Test files match `*.test.js` / `*.spec.ts` by default
- `npx vitest` runs in watch mode; `vitest run` runs once
- `toBe` uses `Object.is`; `toEqual` compares deeply
- Async tests just `await` inside an `async` test function

A PASSING TEST FILE

```
// price.test.js
import { describe, it, expect, vi, beforeEach } from "vitest";
import { withTax, fetchRate } from "./price.js";

describe("withTax", () => {
  it("adds 7% by default", () => {
    expect(withTax(100)).toBe(107);
  });
  it("rejects negatives", () => {
    expect(() => withTax(-1)).toThrow(RangeError);
  });
  it("loads the rate", async () => {
    await expect(fetchRate("IA")).resolves.toBeCloseTo(0.07);
  });
});

it("calls the logger", () => {
  const log = vi.fn();
  withTax(10, { log });
  expect(log).toHaveBeenCalledWith("tax applied");
});
```

CODE EXPLAINED

- → `describe` — groups related tests
- → `toThrow(Type)` — pass a function, not a call
- → `resolves` / `rejects` — assert on Promises
- → `vi.fn()` — records calls for assertions

TYPESCRIPT

- Runs `.ts` test files with no extra setup
- `expectTypeOf(fn).returns.toBeNumber()` tests types themselves

COMMON MATCHERS

MATCHER	CHECKS
`toBe(v)`	Object.is equality
`toEqual(v)`	deep equality
`toBeCloseTo(n)`	float equality
`toContain(x)`	array/string contains
`toThrow()`	function throws
`toHaveBeenCalled()`	mock was called

CLI & HOOKS

COMMAND / HOOK	DOES
`vitest`	watch mode
`vitest run`	single run (CI)
`vitest --coverage`	coverage report
`beforeEach / afterEach`	per-test setup
`it.only / it.skip`	focus / skip

⚠ COMMON MISTAKES

- `expect(fn()).toThrow()` calls fn too early; wrap it: `expect(() => fn())`
- Using `toBe` on objects or arrays; use `toEqual`
- Forgetting to `await` async expectations, so failures go unnoticed

✓ TIPS & BEST PRACTICES

- Name tests as behavior: `it("rejects negative amounts")`
- Use `vi.useFakeTimers()` to test timeouts instantly
- Jest users: the API is nearly identical, mostly swap `jest.` for `vi.`

QUICK SUMMARY

- `describe / it` — structure
- `expect(x).toBe/toEqual` — assert
- `vi.fn / vi.mock` — mocks
- `vitest run` — CI mode

ESLint & Prettier

OVERVIEW

ESLint finds bugs and enforces conventions; **Prettier** formats code so style debates disappear. Modern ESLint uses a **flat config** file, `eslint.config.js`, which exports an array of config objects. Let Prettier own formatting and ESLint own correctness, and don't let them fight.

NOTES

- Flat config replaced `.eslintrc` (ESLint 9+)
- `eslint --fix` auto-fixes what it can
- `eslint-config-prettier` turns off style rules that conflict with Prettier
- Most editors run both on save

ESLINT.CONFIG.JS AND PRETTIER

```
// eslint.config.js
import js from "@eslint/js";
import globals from "globals";
import prettier from "eslint-config-prettier";

export default [
  js.configs.recommended,
  { languageOptions: { globals: { ...globals.browser, ...globals.node } } },
  { rules: {
      "no-unused-vars": ["warn", { argsIgnorePattern: "^_" }],
      eqeqeq: ["error", "always", { null: "ignore" }],
      "prefer-const": "error",
  } },
  { ignores: ["dist/", "coverage/"] },
  prettier,                          // last: disables style conflicts
];

// .prettierrc
{ "semi": true, "singleQuote": false, "printWidth": 90 }

$ npx eslint . --fix
$ npx prettier . --write
```

CODE EXPLAINED

→ Config is an array; later entries override earlier ones

→ `globals` — tells ESLint which environment variables exist

→ Rule level: `"off"`, `"warn"`, `"error"`

→ `ignores` object — files to skip

TYPESCRIPT

- Use **typescript-eslint**: `tseslint.configs.recommended` and its type-aware `recommendedTypeChecked`
- Type-aware rules like `no-floating-promises` catch unawaited Promises

USEFUL RULES

RULE	CATCHES
`no-unused-vars`	dead variables
`eqeqeq`	== usage
`no-undef`	typos in names
`prefer-const`	let never reassigned
`no-console`	stray logs
`no-fallthrough`	missing break

COMMANDS

COMMAND	DOES
`eslint .`	lint everything
`eslint . --fix`	lint + autofix
`prettier . --check`	verify formatting
`prettier . --write`	format files

⚠ COMMON MISTAKES

- Enabling ESLint stylistic rules and Prettier together, causing fix loops
- Disabling rules inline everywhere instead of fixing the config
- Leaving `.eslintrc` files around that flat config ignores

✓ TIPS & BEST PRACTICES

- Run both in a pre-commit hook (lint-staged + husky)
- Add `"lint": "eslint ."` and `"format": "prettier . --write"` scripts
- Fail CI on `eslint .` and `prettier --check`

QUICK SUMMARY

- ESLint — correctness
- Prettier — formatting
- `eslint.config.js` — flat config array
- `eslint-config-prettier` — no conflicts

Vite & Build Tools

OVERVIEW

Vite is the standard dev server and build tool for front-end JavaScript. In development it serves native ES modules with instant **hot module replacement (HMR)**; for production, `vite build` bundles, minifies, and tree-shakes into `dist/`. Environment variables prefixed `VITE_` are exposed through `import.meta.env`.

NOTES

- Templates exist for vanilla, React, Vue, Svelte, Solid, and more
- Files in `public/` are copied as-is
- Only `VITE_*` env vars reach browser code; others stay private
- esbuild and Rollup do the heavy lifting under the hood

CREATE, CONFIGURE, BUILD

```
$ npm create vite@latest my-app -- --template vanilla
$ cd my-app && npm install
$ npm run dev             # http://localhost:5173 with HMR
$ npm run build           # → dist/
$ npm run preview         # serve dist/ locally

// vite.config.js
import { defineConfig } from "vite";
export default defineConfig({
  server: { port: 3000, proxy: { "/api": "http://localhost:8080" } },
  build: { outDir: "dist", sourcemap: true },
  resolve: { alias: { "@": "/src" } },
});

// .env
VITE_API_URL=https://api.example.com
// src/main.js
const api = import.meta.env.VITE_API_URL;
import.meta.env.DEV;     // true in dev server
import.meta.env.MODE;    // "development" / "production"
```

CODE EXPLAINED

- → `npm create vite` — scaffolds a project
- → `proxy` — forward API calls in dev to avoid CORS
- → `alias` — `import x from "@/x.js"`
- → `import.meta.env` — build-time constants

TYPESCRIPT

- Vite **strips** types but doesn't type-check; run `tsc --noEmit` in CI or a script
- Declare custom env vars in `vite-env.d.ts` to type `import.meta.env`

BUILD TOOLS

TOOL	ROLE
Vite	dev server + build
esbuild	fast transpile/bundle
Rollup / Rolldown	production bundling
webpack	older, highly configurable
tsup	library bundling

BUILD TERMS

TERM	MEANING
bundle	combine modules
tree-shaking	drop unused exports
minify	shrink code
code splitting	lazy chunks via import()
HMR	swap modules without reload

⚠ COMMON MISTAKES

- Putting secrets in `VITE_` variables; they end up in public bundles
- Deploying the project root instead of `dist/`
- Assuming a successful build means no type errors

✓ TIPS & BEST PRACTICES

- Use `npm run preview` to test the production build before deploying
- Lazy-load routes with `import()` for smaller first loads
- Inspect bundle size with `rollup-plugin-visualizer`

QUICK SUMMARY

- `npm create vite` — scaffold
- `vite` — dev server + HMR
- `vite build` — production bundle
- `import.meta.env.VITE_*` — env vars

Runtime Validation with Zod

OVERVIEW

Data from outside your program (APIs, forms, files, `localStorage`) can be anything. **Zod** lets you declare a **schema** once and validate data against it at runtime. `parse` returns clean data or throws; `safeParse` returns a result object you can branch on. Schemas can also transform and coerce values.

NOTES

- Zod 4 is current; the core API is the same as Zod 3
- Unknown keys are stripped by default
- `z.coerce.number()` converts strings like form inputs
- Error messages can be customized per check

SCHEMAS AND VALIDATION

```ts
import { z } from "zod";

const User = z.object({
  id: z.number().int().positive(),
  email: z.email(),
  name: z.string().min(1).max(50),
  role: z.enum(["admin", "user"]).default("user"),
  tags: z.array(z.string()).optional(),
});

const user = User.parse(await res.json());    // throws ZodError if bad

const result = User.safeParse(input);
if (!result.success) {
  console.log(z.flattenError(result.error).fieldErrors);
  // { email: ["Invalid email address"] }
} else {
  result.data.role;                           // "user" (default applied)
}

const Form = z.object({ age: z.coerce.number().min(18) });
Form.parse({ age: "21" });                    // { age: 21 }
const Password = z.string().refine(s => /\d/.test(s),
  { message: "needs a digit" });
```

CODE EXPLAINED

- → `z.object({...})` — describes the expected shape
- → `parse` — returns typed data or throws
- → `safeParse` — `{ success, data }` or `{ success, error }`
- → `refine` — custom checks

TYPESCRIPT

- `type User = z.infer<typeof User>` derives the static type from the schema, so you write it once
- After `safeParse` succeeds, `result.data` is fully typed

SCHEMA TYPES

SCHEMA	VALIDATES
`z.string()`	strings
`z.number().int()`	integers
`z.boolean()`	booleans
`z.enum([...])`	one of values
`z.array(s)`	arrays of s
`z.union([a, b])`	either
`z.literal("x")`	exact value

MODIFIERS

MODIFIER	EFFECT
`.optional()`	may be undefined
`.nullable()`	may be null
`.default(v)`	fill when missing
`.transform(fn)`	change output
`.strict()`	reject extra keys

⚠ COMMON MISTAKES

- Trusting `res.json()` without validating
- Using `parse` in request handlers without catching the ZodError
- Writing a TypeScript type and a schema separately so they drift apart

✓ TIPS & BEST PRACTICES

- Validate once at the boundary, then trust the typed data inside
- Share schemas between client and server
- Use `z.coerce` for query strings and form fields

QUICK SUMMARY

- schema — runtime shape
- `parse` — validate or throw
- `safeParse` — validate, no throw
- `z.infer` — type from schema

Sheet 086

Regular Expressions — Syntax

/pattern/flags · RegExp() · character classes · quantifiers · anchors · groups · flags g i m s u v y

OVERVIEW

A **regular expression** describes a text pattern. Write literals between slashes (`/ab+c/i`) or build them with `new RegExp(string, flags)` when the pattern is dynamic. Patterns combine literal characters, **character classes**, **quantifiers**, **anchors**, and **groups**. Flags change matching behavior.

NOTES

- Escape special characters with `\`: `. * + ? ( ) [ ] { } ^ $ | \ /`
- Quantifiers are greedy by default; add `?` for lazy
- In `new RegExp("…")`, backslashes must be doubled
- The `g` and `y` flags make regexes stateful via `lastIndex`

PATTERN BUILDING BLOCKS

```
/cat/;                    // literal "cat"
/c.t/;                    // . any char except newline: cat, cot
/[aeiou]/;                // one vowel
/[^0-9]/;                 // not a digit
/\d{3}-\d{4}/;            // 555-1234
/^\w+@\w+\.\w{2,}$/;     // simple email shape (anchored)
/colou?r/;                // optional u
/(ha)+/;                  // group repeated: ha, haha
/(?<year>\d{4})-(?<mon>\d{2})/;  // named groups
/<.+?>/;                  // lazy: shortest match
/\bcat\b/;               // whole word only
/cat|dog/;                // alternation

const word = "c++";
new RegExp(RegExp.escape(word), "g");  // escape user input (ES2025)
new RegExp("\\d+", "g");  // string form needs double backslashes
```

CODE EXPLAINED

- → `[…]` — one character from a set
- → `{n,m}` — repeat between n and m times
- → `^` `$` — start and end of string (or line with `m`)
- → `RegExp.escape` — safely embed literal text

TYPESCRIPT

- Identical in TypeScript; regex literals are typed `RegExp`
- Set `target` to ES2018+ for named groups and lookbehind in compiled output

CLASSES & ANCHORS

TOKEN	MATCHES
`\d / \D`	digit / non-digit
`\w / \W`	word char / not
`\s / \S`	whitespace / not
`\b`	word boundary
`^ / $`	start / end
`.`	any except newline

QUANTIFIERS

TOKEN	COUNT
`*`	0 or more
`+`	1 or more
`?`	0 or 1
`{3}`	exactly 3
`{2,}`	2 or more
`*? +?`	lazy versions

FLAGS

FLAG	EFFECT
`g`	all matches
`i`	ignore case
`m`	^ $ per line
`s`	. matches newline
`u / v`	Unicode / sets
`y`	sticky at lastIndex

⚠ COMMON MISTAKES

- Forgetting to escape `.` in patterns like file extensions: use `\.png$`
- Building a RegExp from user input without escaping it
- Writing one giant regex to validate emails or HTML; use a parser or library

✓ TIPS & BEST PRACTICES

- Test patterns interactively on regex101.com with the JavaScript flavor
- Use named groups instead of counting parentheses
- Add the `u` (or `v`) flag when matching non-ASCII text

QUICK SUMMARY

- `/pattern/flags` — literal
- classes, quantifiers, anchors — building blocks
- `(?<name>…)` — named groups
- `RegExp.escape` — safe dynamic patterns

Regular Expressions — Methods

test · exec · match · matchAll · replace with $1 / fn · named groups · lookarounds

OVERVIEW

Regexes are used through methods on both `RegExp` and `String`. `test` answers yes/no, `match` and `exec` return match details, `matchAll` iterates every match with groups, and `replace`/`replaceAll` substitute text using `$1`-style references or a callback. **Lookarounds** check context without consuming it.

NOTES

- `match` with `g` returns only the matched strings, no groups
- `matchAll` requires the `g` flag
- Replacement callbacks receive the match, then each group
- Reusing a `g` regex with `test` can skip matches because of `lastIndex`

USING REGEXES

```
const s = "Order 2026-03-15, ship 2026-04-01";
const date = /(?<y>\d{4})-(?<m>\d{2})-(?<d>\d{2})/g;

/\d/.test("a1");                        // true
s.match(/\d{4}/);                       // ["2026", index: 6, …]
s.match(/\d{4}/g);                      // ["2026", "2026"]

for (const m of s.matchAll(date)) {
  console.log(m.groups.y, m.groups.m, m.index);  // 2026 03 6 …
}

s.replace(date, "$<m>/$<d>/$<y>");      // "Order 03/15/2026, ship 04/01/2026"
"a-b_c".replace(/[-_](\w)/g, (_, c) => c.toUpperCase());  // "aBC"
"  x  ".replace(/^\s+|\s+$/g, "");      // "x"

"price: $42".match(/(?<=\$)\d+/)[0];    // "42"   lookbehind
"foobar".match(/foo(?=bar)/)[0];        // "foo"  lookahead
"a1b2".split(/\d/);                     // ["a", "b", ""]
```

CODE EXPLAINED

- → `matchAll` — every match with groups and index
- → `$<name>` / `$1` — insert groups in replacements
- → Callback replacement — compute the new text
- → `(?<=…)` / `(?=…)` — context checks that don't consume

TYPESCRIPT

- `match` returns `RegExpMatchArray | null`; check before indexing
- `m.groups` is typed as `{ [key: string]: string } | undefined`

METHODS

METHOD	RETURNS
`re.test(s)`	boolean
`re.exec(s)`	match | null
`s.match(re)`	match | array | null
`s.matchAll(re)`	iterator of matches
`s.replace(re, r)`	new string
`s.search(re)`	index | -1
`s.split(re)`	array

LOOKAROUNDS

SYNTAX	MEANS
`X(?=Y)`	X followed by Y
`X(?!Y)`	X not followed by Y
`(?<=Y)X`	X preceded by Y
`(?<!Y)X`	X not preceded by Y

⚠ COMMON MISTAKES

- Calling `match(...)[0]` without checking for `null`
- Using `replace` with a string pattern and expecting all occurrences changed
- Reusing a global regex across `test` calls and getting alternating results

✓ TIPS & BEST PRACTICES

- Use `replaceAll` with a regex only when it has the `g` flag
- Use a callback replacement for anything conditional
- Prefer `matchAll` over `exec` loops

QUICK SUMMARY

- `test` — yes/no
- `match / matchAll` — details & groups
- `replace` — `$1`, `$<name>`, or callback
- lookarounds — check context

Proxy & Reflect

OVERVIEW

A **Proxy** wraps an object and intercepts operations on it, like reading, writing, deleting, or calling, through **trap** functions in a handler. **Reflect** provides the default behavior for each trap, so you can add logic and then forward the operation. They power validation layers, reactive frameworks (Vue), and API wrappers.

NOTES

- Traps you don't define pass straight through to the target
- Use `Reflect.*` inside traps to keep correct default behavior
- Proxies are slower than plain objects; avoid in hot paths
- `Proxy.revocable` creates a proxy you can switch off

VALIDATION AND LOGGING PROXIES

```
const user = { name: "Ana", age: 30 };

const validated = new Proxy(user, {
  set(target, key, value, receiver) {
    if (key === "age" && (!Number.isInteger(value) || value < 0)) {
      throw new TypeError(`Invalid age: ${value}`);
    }
    return Reflect.set(target, key, value, receiver);
  },
  get(target, key, rcv) {
    if (!(key in target)) console.warn(`No "${String(key)}"`);
    return Reflect.get(target, key, rcv);
  },
});

validated.age = 31;          // OK
// validated.age = -5;       // TypeError
validated.nmae;              // warns: No "nmae"

const withDefault = new Proxy({}, { get: (t, k) => k in t ? t[k] : 0 });
withDefault.missing;         // 0
Reflect.ownKeys(user);       // ["name", "age"]
```

CODE EXPLAINED

- → `set` trap — return `true` for success
- → `Reflect.set/get` — forward with default behavior
- → Default-value proxy — like Python's defaultdict
- → `Reflect` mirrors operators as functions

TYPESCRIPT

- Type handlers as `ProxyHandler<T>`; `new Proxy<T>(target, handler)` returns `T`
- TS can't see properties a proxy invents dynamically; type them explicitly

COMMON TRAPS

TRAP	INTERCEPTS
get	obj.key
set	obj.key = v
has	key in obj
deleteProperty	delete obj.key
apply	fn(...args)
ownKeys	Object.keys(obj)

USE CASES

USE	TRAP
validation	set
default values	get
change tracking	set + deleteProperty
read-only views	set → false
logging/tracing	get + apply

⚠ COMMON MISTAKES

- Returning nothing (`undefined`) from a `set` trap; strict mode treats it as failure and throws
- Modifying the target directly, bypassing the proxy's checks
- Expecting `===` between a proxy and its target to be true

✓ TIPS & BEST PRACTICES

- Always forward with `Reflect` unless you're intentionally blocking
- Keep the raw target private so all access goes through the proxy
- Prefer setters or plain functions when a proxy is overkill

QUICK SUMMARY

- `new Proxy(target, handler)` — intercept
- traps — get, set, has, apply…
- `Reflect` — default behavior
- uses — validation, reactivity

Typed Arrays & Binary Data

OVERVIEW

An `ArrayBuffer` is a fixed-size block of raw bytes. You read and write it through **views**: **typed arrays** like `Uint8Array` or `Float32Array` treat it as a list of fixed-width numbers, and `DataView` reads mixed types at any offset with explicit endianness. `TextEncoder`/`TextDecoder` convert between strings and UTF-8 bytes.

NOTES

- Typed arrays have a fixed length and a single element type
- Values outside range wrap (`Uint8Array`) or clamp (`Uint8ClampedArray`)
- Several views can share one buffer
- Node's `Buffer` is a `Uint8Array` subclass

BYTES, VIEWS, ENCODING

```
const buf = new ArrayBuffer(8);            // 8 zeroed bytes
const bytes = new Uint8Array(buf);
bytes[0] = 255;  bytes[1] = 256;           // 255, 0 (wraps)
new Uint8ClampedArray([300, -5]);          // [255, 0]

const floats = new Float32Array([1.5, 2.5]);
floats.byteLength;                         // 8
floats.map(x => x * 2);                    // Float32Array [3, 5]

const view = new DataView(buf);
view.setUint16(0, 0x1234);                 // big-endian by default
view.getUint8(0).toString(16);             // "12"
view.getUint16(0, true).toString(16);      // "3412" (little-endian)

const enc = new TextEncoder().encode("héllo");  // Uint8Array(6)
new TextDecoder().decode(enc);             // "héllo"

const hex = [...enc].map(b => b.toString(16).padStart(2, "0")).join("");
// "68c3a96c6c6f"
const resp = await fetch("/img.png");
const data = new Uint8Array(await resp.arrayBuffer());
```

CODE EXPLAINED

- → Typed arrays — array-like but fixed type and length
- → `DataView` — mixed types and endianness control
- → `TextEncoder` — always UTF-8
- → Hex string from bytes with `padStart(2, "0")`

TYPESCRIPT

- Typed arrays are generic over their buffer type since TS 5.7 (`Uint8Array<ArrayBuffer>`)
- That can cause new errors when passing `SharedArrayBuffer` -backed arrays; adjust types if so

TYPED ARRAY TYPES

TYPE	BYTES	RANGE
`Int8Array`	1	-128 … 127
`Uint8Array`	1	0 … 255
`Int16Array`	2	±32,767
`Uint32Array`	4	0 … 4.29e9
`Float32Array`	4	≈7 digits
`Float64Array`	8	same as Number
`BigInt64Array`	8	64-bit BigInt

WHERE BYTES SHOW UP

SOURCE	GIVES
`fetch().arrayBuffer()`	ArrayBuffer
`file.bytes()`	Uint8Array
`crypto.getRandomValues`	typed array
`canvas getImageData`	Uint8ClampedArray
`WebSocket binaryType`	ArrayBuffer/Blob

⚠ COMMON MISTAKES

- Expecting typed arrays to grow with `push` ; they don't have it
- Ignoring endianness when parsing binary file formats
- Using `String.fromCharCode` on UTF-8 bytes; use `TextDecoder`

✓ TIPS & BEST PRACTICES

- Use `subarray` for zero-copy slices (it shares the buffer)
- Use `Uint8Array.prototype.toHex()` / `fromBase64()` where supported (ES2026)
- Transfer buffers to workers instead of copying (sheet 090)

QUICK SUMMARY

- `ArrayBuffer` — raw bytes
- typed arrays — fixed-type views
- `DataView` — mixed types, endianness
- `TextEncoder/Decoder` — string ↔ UTF-8

Workers & Parallelism

OVERVIEW

JavaScript runs your code on one thread, so heavy CPU work blocks everything else. **Web Workers** (browser) and **worker_threads** (Node) run scripts on separate threads. They don't share variables; they communicate by **message passing** with `postMessage`. Large buffers can be **transferred** instead of copied, and `SharedArrayBuffer` with `Atomics` allows true shared memory.

NOTES

- Workers have no DOM access
- Messages are copied with the structured clone algorithm
- Transferring a buffer makes it unusable in the sender
- SharedArrayBuffer in browsers needs cross-origin isolation headers

OFFLOADING WORK TO A WORKER

```
// main.js (browser)
const worker = new Worker(new URL("./prime.worker.js", import.meta.url),
                          { type: "module" });
worker.postMessage({ limit: 5_000_000 });
worker.onmessage = e => console.log("primes:", e.data.count);
worker.onerror = e => console.error(e.message);

// prime.worker.js
self.onmessage = ({ data }) => {
  let count = 0;
  for (let n = 2; n < data.limit; n++) if (isPrime(n)) count++;
  self.postMessage({ count });
};

// Node
import { Worker, isMainThread, parentPort, workerData } from "node:worker_threads";
if (isMainThread) {
  new Worker(new URL(import.meta.url), { workerData: 42 })
    .on("message", m => console.log(m));       // 84
} else {
  parentPort.postMessage(workerData * 2);
}

worker.postMessage(bigBuffer, [bigBuffer]);    // transfer, no copy
```

CODE EXPLAINED

- → `new URL(…, import.meta.url)` — lets bundlers find the worker file
- → `postMessage` / `onmessage` — the whole communication channel
- → Node: same file can act as main or worker
- → Transfer list — moves ownership of buffers

TYPESCRIPT

- Messages are untyped; wrap them in a typed helper or use a library like Comlink
- Add `"lib": ["WebWorker"]` for worker files so `self` is typed correctly

BROWSER VS NODE

	WEB WORKER	WORKER_THREADS
create	new Worker(url)	new Worker(file)
send	postMessage	postMessage
receive	onmessage	on("message")
input data	first message	workerData
stop	terminate()	terminate()

WHEN TO USE A WORKER

TASK	WORKER?
image/video processing	yes
parsing huge files	yes
crypto, compression	yes
waiting on network	no (async is enough)
DOM updates	no (impossible)

⚠ COMMON MISTAKES

- Using workers for I/O-bound work that async already handles
- Sending huge objects repeatedly; copying costs more than the work saved
- Forgetting to `terminate()` workers you no longer need

✓ TIPS & BEST PRACTICES

- Use a worker pool (e.g., `piscina` in Node) for many small jobs
- Measure first: worker startup has overhead
- Use Comlink to call worker functions like normal async functions

QUICK SUMMARY

- workers — real parallel threads
- `postMessage` — communicate
- transfer — move buffers, no copy
- no DOM — compute only

Memory & Resource Management

OVERVIEW

JavaScript frees memory automatically with **garbage collection**: once nothing can reach an object, it can be reclaimed. Leaks happen when references linger unexpectedly. For external resources (files, locks, connections), **explicit resource management** with `using` and `Symbol.dispose` guarantees cleanup when a block exits.

NOTES

- GC timing is unpredictable; never rely on it for cleanup
- `WeakRef` lets you hold an object without keeping it alive
- `FinalizationRegistry` callbacks may run late or never
- `await using` handles async cleanup with `Symbol.asyncDispose`

LEAKS, WEAK REFS, USING

```
// Common leak: listener holds a big object forever
const big = new Array(1e6).fill("x");
window.addEventListener("resize", () => big.length);   // never removed

const ac = new AbortController();                      // fix: removable
window.addEventListener("resize", onResize, { signal: ac.signal });
ac.abort();                                            // removes it

const cache = new WeakMap();                           // auto-cleans
cache.set(node, computeLayout(node));

const ref = new WeakRef(expensiveObject);
ref.deref()?.render();              // undefined once collected

class DbConnection {
  [Symbol.asyncDispose]() { return this.close(); }
}
async function report() {
  await using db = await connect();  // closed when the function exits,
  return db.query("SELECT …");       // even if query throws
}
```

CODE EXPLAINED

→ AbortController `signal` — remove many listeners at once

→ WeakMap — cache that doesn't block GC

→ `deref()` — may return `undefined`

→ `using` — dispose at block end, like try/finally

TYPESCRIPT

- `using` and `await using` are supported since TS 5.2; add `esnext.disposable` to `lib` if needed
- Implement the `Disposable` / `AsyncDisposable` interfaces

COMMON LEAKS

CAUSE	FIX
event listeners	removeEventListener / signal
timers	clearInterval
global caches	WeakMap or size limit
closures over big data	null out references
detached DOM nodes	drop references

TOOLS

TOOL	USE
DevTools Memory tab	heap snapshots
performance.memory	Chrome heap size
node --inspect	Node heap snapshots
process.memoryUsage()	Node RSS/heap

⚠ COMMON MISTAKES

- Relying on `FinalizationRegistry` for essential cleanup
- Unbounded `Map` caches that grow forever
- Using `using` in runtimes without support; check before shipping

✓ TIPS & BEST PRACTICES

- Compare two heap snapshots to find what's growing
- Give caches a max size or TTL (or use an LRU library)
- Use `using` for anything with a `close()`, `release()`, or `unlock()`

QUICK SUMMARY

- GC — frees unreachable objects
- leaks — lingering references
- `WeakMap / WeakRef` — weak holds
- `using` — guaranteed cleanup

The Event Loop & Timers

OVERVIEW

JavaScript runs one thing at a time on a **call stack**. Async work (timers, network, I/O) is handled outside, and its callbacks wait in queues. The **event loop** runs all synchronous code first, then drains the **microtask queue** (Promise callbacks, `queueMicrotask`), then takes one **task** (timer, event) and repeats.

NOTES

- Microtasks always run before the next task
- `setTimeout(fn, 0)` means "after current work and microtasks", not now
- Timer delays are minimums, never guarantees
- Long synchronous code freezes the page: no clicks, no rendering

EXECUTION ORDER PUZZLE

```
console.log("1 sync");

setTimeout(() => console.log("5 timeout (task)"), 0);

Promise.resolve().then(() => console.log("3 promise (microtask)"));

queueMicrotask(() => console.log("4 microtask"));

console.log("2 sync");

// Output: 1 sync, 2 sync, 3 promise, 4 microtask, 5 timeout
```

```
const id = setTimeout(save, 1000);       // run once after ~1s
clearTimeout(id);                        // cancel
const tick = setInterval(update, 500);   // repeat every ~500ms
clearInterval(tick);
await new Promise(r => setTimeout(r, 300));   // sleep 300ms
await scheduler.yield?.();               // let the browser breathe
```

CODE EXPLAINED

→ Sync code always finishes first

→ Then every queued microtask, in order

→ Then one macrotask (timer), then microtasks again

→ Promise + setTimeout — the standard `sleep`

TYPESCRIPT

- Timer IDs differ: `number` in browsers, `NodeJS.Timeout` in Node; use `ReturnType<typeof setTimeout>`
- Otherwise identical in TypeScript

WHAT GOES WHERE

SOURCE	QUEUE
`promise .then/.catch`	microtask
`await continuation`	microtask
`queueMicrotask`	microtask
`setTimeout / setInterval`	task
`DOM events, I/O`	task
`requestAnimationFrame`	before paint

TIMER API

FUNCTION	DOES
`setTimeout(fn, ms)`	once, returns id
`setInterval(fn, ms)`	repeat, returns id
`clearTimeout(id)`	cancel
`clearInterval(id)`	stop
`queueMicrotask(fn)`	run soon

⚠ COMMON MISTAKES

- Expecting `setTimeout(fn, 0)` to run before Promise callbacks
- Using `setInterval` with async work that can take longer than the interval, so calls pile up
- Running heavy loops on the main thread and freezing the UI

✓ TIPS & BEST PRACTICES

- Use chained `setTimeout` instead of `setInterval` for async polling
- Break long tasks into chunks (or use a worker)
- Visualize the loop with Jake Archibald's "In the Loop" talk or loupe

QUICK SUMMARY

- call stack — one thing at a time
- microtasks — promises, run first
- tasks — timers, events
- `setTimeout` — minimum delay

Promises

OVERVIEW

A **Promise** represents a value that will exist later. It starts **pending** and settles exactly once, either **fulfilled** with a value or **rejected** with a reason. Attach handlers with `.then`, `.catch`, and `.finally`. Each returns a new Promise, so calls **chain**, and returning a Promise inside `.then` waits for it.

NOTES

- A settled Promise never changes state
- Handlers always run asynchronously (as microtasks)
- An error thrown in `.then` becomes a rejection down the chain
- Unhandled rejections are reported and crash Node

CREATING AND CHAINING

```
function delay(ms, value) {
  return new Promise((resolve, reject) => {
    if (ms < 0) reject(new RangeError("negative delay"));
    setTimeout(() => resolve(value), ms);
  });
}

delay(500, 42)
  .then(v => v * 2)                       // 84
  .then(v => delay(100, v + 1))           // waits, then 85
  .then(v => console.log(v))              // 85
  .catch(err => console.error(err))       // any error above
  .finally(() => console.log("done"));    // always

fetch("/api/user")
  .then(res => {
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return res.json();
  })
  .then(user => console.log(user.name));

Promise.resolve(5);                       // already fulfilled
Promise.reject(new Error("no"));          // already rejected
```

CODE EXPLAINED

- → Executor `(resolve, reject)` — runs immediately
- → Return a value from `.then` — next handler gets it
- → Return a Promise — chain waits for it
- → One `.catch` at the end handles every step

TYPESCRIPT

- Type as `Promise<T>`: `new Promise<number>(resolve => …)`
- Rejection reasons are untyped (`any`); narrow in `.catch`

PROMISE STATES

STATE	MEANING
`pending`	not settled yet
`fulfilled`	resolved with a value
`rejected`	failed with a reason
`settled`	fulfilled or rejected

INSTANCE METHODS

METHOD	RUNS WHEN
`.then(onOk, onErr)`	fulfilled (or rejected)
`.catch(fn)`	rejected
`.finally(fn)`	either; value passes through

⚠ COMMON MISTAKES

- Forgetting to `return` inside `.then`, so the next step gets `undefined`
- Nesting `.then` inside `.then` ("promise pyramid") instead of chaining
- Creating a Promise around something that already returns one (the constructor anti-pattern)

✓ TIPS & BEST PRACTICES

- Prefer `async/await` (sheet 094) for readability; know promises to understand it
- Always end chains with `.catch` or return them to a caller who will
- Use `util.promisify` to convert callback APIs

QUICK SUMMARY

- Promise — future value
- `then / catch / finally` — handlers
- chaining — return values flow on
- settles once

async / await

OVERVIEW

`async` functions always return a Promise, and `await` pauses the function until a Promise settles, then gives you its value (or throws its rejection). This lets async code read top to bottom with normal `try/catch`. Be deliberate about running awaits **in sequence** versus starting work **in parallel**.

NOTES

- `await` only works in async functions and module top level
- A `return` value is wrapped in a resolved Promise
- A `throw` becomes a rejected Promise
- `await` in `forEach` doesn't wait; use `for...of`

ASYNC/AWAIT PATTERNS

```
async function loadUser(id) {
  try {
    const res = await fetch(`/api/users/${id}`);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.json();
  } catch (err) {
    console.error("loadUser failed", err);
    return null;
  }
}

const a = await loadUser(1);            // sequential: 2× time
const b = await loadUser(2);

const [c, d] = await Promise.all([      // parallel: 1× time
  loadUser(1), loadUser(2),
]);

for (const id of ids) await process(id);       // one at a time
// ids.forEach(async id => await process(id)); // doesn't wait!

for await (const chunk of stream) {}    // async iterables
const fetchJSON = async url => (await fetch(url)).json();
```

CODE EXPLAINED

- → `try/catch` handles awaited rejections
- → Two `await`s in a row — second starts after first finishes
- → `Promise.all` — start together, wait for all
- → `for...of` with `await` — ordered processing

TYPESCRIPT

- Return type is `Promise<T>`; annotate as `async function f(): Promise<User | null>`
- `Awaited<T>` unwraps Promise types; typescript-eslint's `no-floating-promises` flags missing awaits

RULES

WRITE	MEANS
`async function f()`	returns a Promise
`return x`	resolves with x
`throw e`	rejects with e
`await p`	value of p, or throws
`top-level await`	modules only

SEQUENTIAL VS PARALLEL

CODE	TIMING
`await a(); await b();`	a then b
`await Promise.all([a(), b()])`	together
`const p = a(); …; await p`	start early
`for (x of xs) await f(x)`	one by one

⚠ COMMON MISTAKES

- Awaiting independent calls one after another and doubling load time
- Using `await` inside `forEach` or `map` and expecting it to wait
- Calling an async function without `await` or `.catch`, losing its errors

✓ TIPS & BEST PRACTICES

- Start independent work first, await later
- Use `Promise.all` with `map` for parallel loops: `await Promise.all(ids.map(load))`
- Limit concurrency for big batches (p-limit) to avoid overwhelming servers

QUICK SUMMARY

- `async` — returns a Promise
- `await` — pause for a result
- `try/catch` — handle rejections
- `Promise.all` — parallel awaits

Promise Combinators

OVERVIEW

Combinators coordinate several Promises at once. `Promise.all` waits for every one and fails fast on the first rejection. `allSettled` waits for all and reports each outcome. `race` settles with whichever finishes first, and `any` returns the first **success**. `withResolvers` and `try` are newer helpers for creating promises.

NOTES

- All combinators take any iterable of Promises or values
- `all` doesn't cancel the others when one fails
- `any` rejects with an `AggregateError` if every input rejects
- Results keep input order, not completion order

COMBINATORS IN ACTION

```
const [user, posts] = await Promise.all([
  getUser(id), getPosts(id),
]);                                          // fails fast

const results = await Promise.allSettled(urls.map(u => fetch(u)));
const ok = results.filter(r => r.status === "fulfilled").map(r => r.value);
const failed = results.filter(r => r.status === "rejected").length;

const timeout = ms => new Promise((_, rej) =>
  setTimeout(() => rej(new Error("timeout")), ms));
await Promise.race([fetch(url), timeout(5000)]);   // first to settle

const fastest = await Promise.any([              // first success
  fetch("https://mirror1/x"), fetch("https://mirror2/x"),
]);

const { promise, resolve, reject } = Promise.withResolvers();
button.onclick = () => resolve("clicked");
await promise;                                   // "clicked"

Promise.try(() => maybeSyncOrAsync());           // always a Promise
```

CODE EXPLAINED

- → `all` — every result, or the first error
- → `allSettled` — `{ status, value | reason }` for each
- → `race` — timeouts and first response wins
- → `withResolvers` — resolve from outside the executor

TYPESCRIPT

- `Promise.all` infers a tuple: `[User, Post[]]` from the example
- `allSettled` results are `PromiseSettledResult<T>[]`, narrowed by `status`

COMBINATORS

METHOD	RESOLVES WHEN	REJECTS WHEN
`all`	all fulfill	any rejects
`allSettled`	all settle	never
`race`	first fulfills	first rejects
`any`	first fulfills	all reject

HELPERS

METHOD	RETURNS
`Promise.resolve(v)`	fulfilled promise
`Promise.reject(e)`	rejected promise
`Promise.withResolvers()`	{ promise, resolve, reject }
`Promise.try(fn)`	wraps sync or async fn

⚠ COMMON MISTAKES

- Using `all` when partial failure is acceptable; use `allSettled`
- Racing against a timeout without cancelling the losing request (see sheet 096)
- Passing functions instead of Promises: `Promise.all([load])` instead of `[load()]`

✓ TIPS & BEST PRACTICES

- Use `AbortSignal.timeout(ms)` with fetch instead of hand-rolled race timeouts
- Use `allSettled` for batch jobs and report failures together
- Check runtime support for `Promise.try` (ES2025)

QUICK SUMMARY

- `all` — all or fail fast
- `allSettled` — every outcome
- `race` — first to settle
- `any` — first success

Cancellation, Timeouts & Debouncing

OVERVIEW

Promises can't be cancelled by themselves, so the platform uses `AbortController`: create a controller, pass its `signal` to `fetch`, event listeners, or your own functions, and call `abort()` to stop them. `AbortSignal.timeout(ms)` gives a ready-made timeout. **Debounce** and **throttle** limit how often a function runs in response to rapid events.

NOTES

- An aborted fetch rejects with a `DOMException` named `AbortError`
- A timeout signal aborts with `TimeoutError`
- `AbortSignal.any([a, b])` aborts when either does
- One signal can cancel many operations at once

ABORT, TIMEOUTS, DEBOUNCE

```
const ac = new AbortController();
fetch("/api/search?q=js", { signal: ac.signal })
  .then(r => r.json())
  .catch(err => { if (err.name !== "AbortError") throw err; });
ac.abort();                                  // cancel it

const res = await fetch(url, { signal: AbortSignal.timeout(5000) });

function debounce(fn, ms) {                  // wait for a pause
  let t;
  return (...args) => { clearTimeout(t); t = setTimeout(() => fn(...args), ms); };
}
input.addEventListener("input", debounce(e => search(e.target.value), 300));

function throttle(fn, ms) {                  // at most once per ms
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= ms) { last = now; fn(...args); }
  };
}
window.addEventListener("scroll", throttle(updateHeader, 100));
```

CODE EXPLAINED

- → `signal` — pass it to anything cancelable
- → Check `err.name` to ignore expected aborts
- → Debounce — runs after events stop for `ms`
- → Throttle — runs at most once every `ms`

TYPESCRIPT

- Type parameters as `AbortSignal`; generic `debounce<T extends (...a: any[]) => void>(fn: T, ms: number)`
- Narrow errors: `err instanceof DOMException && err.name === "AbortError"`

ABORT API

API	DOES
`new AbortController()`	controller + signal
`ac.abort(reason)`	trigger abort
`signal.aborted`	boolean
`signal.throwIfAborted()`	throw if aborted
`AbortSignal.timeout(ms)`	auto-abort signal
`AbortSignal.any([...])`	combine signals

DEBOUNCE VS THROTTLE

USE	PICK
`search-as-you-type`	debounce
`window resize end`	debounce
`scroll position`	throttle
`mouse move tracking`	throttle / rAF

⚠ COMMON MISTAKES

- Treating `AbortError` as a real failure and showing an error message
- Reusing an aborted controller; create a new one per operation
- Firing a fetch on every keystroke without debounce or abort

✓ TIPS & BEST PRACTICES

- Abort the previous search request when a new one starts
- Pass `{ signal }` to `addEventListener` to remove listeners in bulk
- Use `requestAnimationFrame` for visual throttling

QUICK SUMMARY

- `AbortController` — cancel operations
- `AbortSignal.timeout` — time limits
- debounce — after a pause
- throttle — at most every n ms

Selecting & Traversing

OVERVIEW

The **DOM** is the browser's object tree for the page. `querySelector` returns the first element matching a CSS selector, and `querySelectorAll` returns a static `NodeList` of all matches. From any element you can move up (`parentElement`, `closest`), down (`children`, `firstElementChild`), and sideways (`nextElementSibling`).

NOTES

- `querySelector` returns `null` when nothing matches
- `querySelectorAll` results don't update when the page changes
- `getElementsByClassName` returns a **live** HTMLCollection
- Scope searches to an element: `form.querySelector("input")`

SELECTING AND TRAVERSING

```
const title = document.querySelector("h1");
const btn = document.querySelector("#save");              // by id
const items = document.querySelectorAll(".item");         // NodeList
const firstLink = document.querySelector("nav a[href^='http']");
const byId = document.getElementById("save");             // fastest

items.forEach(el => el.classList.add("ready"));           // NodeList has forEach
[...items].map(el => el.textContent);                     // to array

const card = btn.closest(".card");          // nearest ancestor match
card.parentElement;                         // up one level
card.children;                              // HTMLCollection of elements
card.firstElementChild;  card.lastElementChild;
card.nextElementSibling;                    // next element
card.querySelector(".price");               // search inside card

btn.matches(".primary");                    // true/false
document.querySelector(".missing")?.remove();   // safe when null
```

CODE EXPLAINED

- → Any CSS selector works, including attributes and pseudo-classes
- → Spread a NodeList to use array methods
- → `closest` — walks up, including the element itself
- → Use `?.` when an element may not exist

TYPESCRIPT

- `querySelector` returns `Element | null`; tag selectors infer types (`"input"` → `HTMLInputElement`)
- For other selectors pass a generic: `querySelector<HTMLButtonElement>("#save")`

SELECTION METHODS

METHOD	RETURNS
`querySelector(sel)`	Element | null
`querySelectorAll(sel)`	static NodeList
`getElementById(id)`	Element | null
`getElementsByClassName`	live HTMLCollection
`closest(sel)`	ancestor | null
`matches(sel)`	boolean

TRAVERSAL

PROPERTY	GIVES
`parentElement`	parent
`children`	child elements
`firstElementChild`	first child element
`nextElementSibling`	next element
`childNodes`	includes text nodes

⚠ COMMON MISTAKES

- Running selection code before the DOM exists; use `defer` or `type="module"` scripts
- Using `childNodes` / `nextSibling` and getting whitespace text nodes
- Calling methods on a `null` result from `querySelector`

✓ TIPS & BEST PRACTICES

- Prefer `data-*` attributes as JS hooks: `[data-action="save"]`
- Cache elements you use repeatedly in variables
- Use `element.querySelector` to keep searches local and fast

QUICK SUMMARY

- `querySelector` — first match
- `querySelectorAll` — static list
- `closest` — nearest ancestor
- `children / parentElement` — move around

Content, Attributes & Classes

OVERVIEW

Once you have an element, you can change its **text**, **HTML**, **attributes**, **classes**, and **inline styles**. Use `textContent` for plain text (it's safe and fast). Use `innerHTML` only with trusted markup. `classList` manages classes without string juggling, and `dataset` reads and writes `data-*` attributes.

NOTES

- `textContent` escapes HTML automatically
- `innerHTML` with user input is an XSS vulnerability
- `dataset` converts `data-user-id` to `dataset.userId`
- `style` sets inline styles only; computed values come from `getComputedStyle`

CHANGING CONTENT AND ATTRIBUTES

```
const el = document.querySelector(".msg");

el.textContent = "Saved!";                  // plain text, safe
el.innerHTML = "<strong>Saved</strong>";    // parses HTML (trusted only)
el.textContent = userInput;                 // always use for user data

el.classList.add("visible", "success");
el.classList.remove("error");
el.classList.toggle("open");                // add/remove
el.classList.toggle("dark", isDark);        // force on/off
el.classList.contains("open");              // true

const link = document.querySelector("a");
link.getAttribute("href");                  // "/docs"
link.setAttribute("target", "_blank");
link.removeAttribute("title");
link.href;                                  // full URL (property)

// <div data-user-id="42" data-role="admin">
el.dataset.userId;                          // "42" (string)
el.dataset.status = "active";               // adds data-status

el.style.setProperty("--accent", "teal");   // CSS variable
el.hidden = true;                           // hide via attribute
getComputedStyle(el).color;                 // "rgb(0, 128, 128)"
```

CODE EXPLAINED

- → `textContent` vs `innerHTML` — text vs parsed markup
- → `toggle(cls, force)` — boolean decides add or remove
- → Properties (`link.href`) return resolved values; attributes return raw strings
- → CSS variables via `style.setProperty`

TYPESCRIPT

- `dataset` values are typed `string | undefined`
- Element type matters: `.href` exists on `HTMLAnchorElement`, not plain `Element`

CONTENT PROPERTIES

PROPERTY	USE
`textContent`	plain text (safe)
`innerText`	rendered text (slower)
`innerHTML`	HTML string
`outerHTML`	element + contents
`value`	form field value

CLASSLIST

METHOD	DOES
`add(...c)`	add classes
`remove(...c)`	remove
`toggle(c, force?)`	flip or set
`contains(c)`	check
`replace(a, b)`	swap

⚠ COMMON MISTAKES

- Inserting user input with `innerHTML`
- Setting `el.className = "x"` and wiping out existing classes
- Setting many styles in JS instead of toggling a CSS class

✓ TIPS & BEST PRACTICES

- Toggle classes and let CSS handle the visuals
- Use `setHTML` / a sanitizer (DOMPurify) if you must insert user HTML
- Batch DOM reads and writes to avoid layout thrashing

QUICK SUMMARY

- `textContent` — safe text
- `classList` — manage classes
- `get/setAttribute` — raw attributes
- `dataset` — data-* values

Sheet 099

Creating & Removing Elements

createElement · append · prepend · before / after · remove · replaceWith · template · DocumentFragment

OVERVIEW

Create elements with `document.createElement`, configure them, then insert them with `append`, `prepend`, `before`, `after`, or `replaceWith`. Remove with `remove()`. For repeated structures, clone a `<template>`. When adding many nodes, build them in a `DocumentFragment` so the page updates once.

NOTES

- `append` accepts multiple nodes and strings
- Inserting an existing node **moves** it
- `cloneNode(true)` copies the whole subtree
- `insertAdjacentHTML` inserts markup at a position without wiping siblings

BUILDING DOM NODES

```
const li = document.createElement("li");
li.textContent = "New task";
li.dataset.id = "7";
list.append(li);                          // end of list
list.prepend(header);                     // start
li.before(divider);  li.after(note);      // siblings
li.replaceWith(updatedLi);
li.remove();

const frag = document.createDocumentFragment();
for (const t of tasks) {
  const item = document.createElement("li");
  item.textContent = t.title;
  frag.append(item);
}
list.append(frag);                        // one insertion

// <template id="row"><tr><td class="n"></td><td class="p"></td></tr></template>
const tpl = document.querySelector("#row");
const row = tpl.content.cloneNode(true);
row.querySelector(".n").textContent = "Pen";
table.tBodies[0].append(row);

list.replaceChildren();                   // remove all children
```

CODE EXPLAINED

→ Configure before inserting to avoid extra reflows

→ Fragment — builds offscreen, inserts once

→ `<template>` — inert HTML you clone and fill

→ `replaceChildren()` — clear an element

TYPESCRIPT

- `createElement("li")` returns `HTMLLIElement`, so tag-specific properties are typed
- `tpl.content.cloneNode(true)` returns `Node`; cast to `DocumentFragment` to query it

INSERTION METHODS

METHOD	INSERTS
`parent.append(n)`	last child
`parent.prepend(n)`	first child
`el.before(n)`	before el
`el.after(n)`	after el
`el.replaceWith(n)`	instead of el
`el.remove()`	removes el

INSERTADJACENTHTML

POSITION	WHERE
`beforebegin`	before el
`afterbegin`	first inside
`beforeend`	last inside
`afterend`	after el

⚠ COMMON MISTAKES

- Building HTML strings with user data and inserting via `innerHTML`
- Appending nodes one at a time in a large loop
- Expecting `append(existingNode)` to copy it; it moves it

✓ TIPS & BEST PRACTICES

- Use `<template>` for anything more than a couple of nested elements
- Clear and repopulate lists with `replaceChildren(...nodes)`
- Consider a small library (lit-html) or framework when UI logic grows

QUICK SUMMARY

- `createElement` — new node
- `append / prepend / before / after` — insert
- `DocumentFragment` — batch inserts
- `<template>` — clone and fill

Events

addEventListener · event object · target vs currentTarget · once / passive · removeEventListener · preventDefault

OVERVIEW

`addEventListener(type, handler, options)` runs a handler whenever an event fires. The handler receives an **event object** with details: `target` (what was actually interacted with), `currentTarget` (the element the listener is on), key or mouse info, and methods like `preventDefault()` to stop the browser's default action.

NOTES

- Use the same function reference to remove a listener
- `once: true` removes the listener after the first call
- `passive: true` promises not to call `preventDefault` (smoother scrolling)
- Avoid inline `onclick="…"` attributes

LISTENING FOR EVENTS

```
const btn = document.querySelector("#buy");

function onClick(event) {
  event.target;            // element clicked (may be a child)
  event.currentTarget;     // btn
  event.type;              // "click"
}
btn.addEventListener("click", onClick);
btn.removeEventListener("click", onClick);   // same reference

btn.addEventListener("click", () => track("buy"), { once: true });

form.addEventListener("submit", e => {
  e.preventDefault();      // stop page reload
  save(new FormData(form));
});

document.addEventListener("keydown", e => {
  if (e.key === "Escape") closeModal();
  if (e.ctrlKey && e.key === "s") { e.preventDefault(); save(); }
});

window.addEventListener("scroll", onScroll, { passive: true });
```

CODE EXPLAINED

- → `target` vs `currentTarget` — origin vs listener's element
- → `once` — auto-remove after one call
- → `preventDefault` — cancel default action (submit, link, key)
- → `e.key` — readable key names

TYPESCRIPT

- Event types are inferred from the name: `"keydown"` → `KeyboardEvent`, `"click"` → `MouseEvent`
- `e.target` is `EventTarget | null`; cast or check `instanceof HTMLElement`

COMMON EVENTS

EVENT	FIRES WHEN
`click`	pressed and released
`input / change`	field edited / committed
`submit`	form submitted
`keydown / keyup`	key pressed / released
`focus / blur`	gains / loses focus
`DOMContentLoaded`	HTML parsed

EVENT OBJECT

MEMBER	GIVES
`target`	originating element
`currentTarget`	listening element
`preventDefault()`	cancel default
`key / code`	key name / physical key
`clientX / clientY`	mouse position

⚠ COMMON MISTAKES

- Removing a listener with a new arrow function; it never matches
- Forgetting `preventDefault` on form submit and losing state to a reload
- Using `e.target` when a child element was clicked and you needed the button

✓ TIPS & BEST PRACTICES

- Use `currentTarget` when the listener is on a container with children
- Clean up many listeners with one `AbortController` signal
- Prefer `pointer*` events to handle mouse, touch, and pen together

QUICK SUMMARY

- `addEventListener` — attach
- event object — details
- `preventDefault` — stop default
- `once / passive / signal` — options

Propagation & Delegation

capture · bubble · stopPropagation · delegation with closest() · CustomEvent · dispatchEvent

OVERVIEW

Most events **propagate**: they travel down from `window` to the target (**capture** phase), then back up (**bubble** phase). Because events bubble, one listener on a parent can handle events for all its children. This is **event delegation**: efficient, and it works for elements added later. You can also dispatch your own `CustomEvents`.

NOTES

- Listeners run in the bubble phase by default
- `{ capture: true }` runs during the capture phase
- `focus`, `blur`, `mouseenter`, `mouseleave` don't bubble
- `stopPropagation` hides events from other code; use sparingly

BUBBLING AND DELEGATION

```
// <ul id="todos"><li><span>Buy milk</span><button class="del">×</button></li>…</ul
const todos = document.querySelector("#todos");

todos.addEventListener("click", e => {          // one listener for all
  const del = e.target.closest(".del");
  if (!del || !todos.contains(del)) return;
  del.closest("li").remove();
});

child.addEventListener("click", e => e.stopPropagation()); // parent won't hear
document.addEventListener("click", logAll, { capture: true }); // runs first

const evt = new CustomEvent("cart:add", {
  detail: { id: 7, qty: 2 }, bubbles: true,
});
button.dispatchEvent(evt);
document.addEventListener("cart:add", e => updateBadge(e.detail.qty));
```

CODE EXPLAINED

- → `closest` — find the relevant element from whatever was clicked
- → `contains` check — ignore matches outside the container
- → Capture listeners run before bubble listeners
- → `CustomEvent` — `detail` carries your data

TYPESCRIPT

- Type custom events as `CustomEvent<{ id: number; qty: number }>`
- Add them to `DocumentEventMap` via declaration merging so listeners get typed `detail`

EVENT PHASES

PHASE	DIRECTION
1 capture	window → target
2 target	at the element
3 bubble	target → window

STOPPING EVENTS

METHOD	EFFECT
`preventDefault()`	cancel browser action
`stopPropagation()`	stop moving up/down
`stopImmediatePropagation()`	also skip other listeners here

NON-BUBBLING EVENTS

EVENT	BUBBLING ALTERNATIVE
`focus`	focusin
`blur`	focusout
`mouseenter`	mouseover
`mouseleave`	mouseout

⚠ COMMON MISTAKES

- Checking `e.target.matches(".del")` when the click lands on an icon inside the button; use `closest`
- Calling `stopPropagation` everywhere and breaking analytics or delegation elsewhere
- Delegating events that don't bubble (`focus`) instead of their bubbling versions

✓ TIPS & BEST PRACTICES

- Delegate for lists, tables, and anything rendered dynamically
- Namespace custom event names (`cart:add`) to avoid collisions
- Set `bubbles: true` on custom events meant for delegation

QUICK SUMMARY

- bubbling — events travel up
- delegation — one parent listener
- `closest` — find the real target
- `CustomEvent` — your own events

Forms & Input

OVERVIEW

Forms collect user input. Listen for `submit` on the form (not `click` on the button) so Enter works too, call `preventDefault()` to handle it in JavaScript, and read all fields at once with `FormData`. The built-in **constraint validation** API (`required`, `type="email"`, `pattern`) checks input before you write custom code.

NOTES

- `input` fires on every change; `change` fires when the value is committed
- Field values are always strings (checkboxes: `checked`)
- `FormData` includes only fields with a `name` attribute
- `form.elements.email` reaches fields by name

HANDLING A FORM

```
// <form id="signup"><input name="email" type="email" required>
//   <input name="age" type="number" min="18"><input name="terms" type="checkbox">
//   <button>Join</button></form>
const form = document.querySelector("#signup");

form.addEventListener("submit", async e => {
  e.preventDefault();
  if (!form.checkValidity()) { form.reportValidity(); return; }

  const data = Object.fromEntries(new FormData(form));
  // { email: "a@b.co", age: "21", terms: "on" }
  data.age = Number(data.age);

  await fetch("/api/signup", { method: "POST", body: new FormData(form) });
  form.reset();
});

const email = form.elements.email;
email.addEventListener("input", () => {
  email.setCustomValidity(
    email.value.endsWith("@example.com") ? "Use a real address" : "");
});

form.elements.terms.checked;      // true / false
form.querySelector("select").value;
```

CODE EXPLAINED

→ `checkValidity` / `reportValidity` — run built-in checks

→ `Object.fromEntries(new FormData(form))` — form → object

→ Pass `FormData` straight to `fetch` for multipart uploads

→ `setCustomValidity("")` — clear a custom error

TYPESCRIPT

- `FormData.get` returns `FormDataEntryValue | null` (string or File); narrow before use
- `form.elements.email` needs a cast to `HTMLInputElement`

VALIDATION ATTRIBUTES

ATTRIBUTE	CHECKS
`required`	not empty
`type="email" / "url"`	format
`min / max / step`	number range
`minlength / maxlength`	text length
`pattern`	regex match

READING VALUES

FIELD	READ WITH
`text / select`	.value
`checkbox`	.checked
`radio group`	form.elements.name.value
`number`	.valueAsNumber
`file`	.files

⚠ COMMON MISTAKES

- Listening for button `click` instead of form `submit`
- Doing math with field values without converting from strings
- Relying only on client-side validation; always validate on the server

✓ TIPS & BEST PRACTICES

- Style invalid fields with `:user-invalid` so errors show after interaction
- Use `valueAsNumber` and `valueAsDate` to skip manual parsing
- Disable the submit button while the request is in flight

QUICK SUMMARY

- `submit` + `preventDefault` — handle in JS
- `FormData` — read all fields
- constraint validation — built-in checks
- values are strings — convert

fetch & HTTP

OVERVIEW

`fetch(url, options)` makes HTTP requests in browsers, Node 18+, Deno, and Bun. It returns a Promise of a `Response`. The key gotcha: fetch only **rejects on network failure**. A 404 or 500 still resolves, so always check `response.ok` or `status`. Read the body once with `json()`, `text()`, or `blob()`.

NOTES

- Default method is GET
- Send JSON with `JSON.stringify` plus a `Content-Type` header
- Cookies are sent only for same-origin requests unless `credentials: "include"`
- Cross-origin requests need CORS headers from the server

GET, POST, ERROR HANDLING

```
const res = await fetch("https://api.example.com/users?limit=10");
res.status;                                  // 200
res.ok;                                      // true for 200-299
res.headers.get("content-type");             // "application/json"
const users = await res.json();

async function postJSON(url, data) {
  const res = await fetch(url, {
    method: "POST",
    headers: { "Content-Type": "application/json",
               Authorization: `Bearer ${token}` },
    body: JSON.stringify(data),
    signal: AbortSignal.timeout(8000),
  });
  if (!res.ok) throw new Error(`HTTP ${res.status}: ${await res.text()}`);
  return res.status === 204 ? null : res.json();
}
const created = await postJSON("/api/users", { name: "Ana" });

const url = new URL("/search", location.origin);
url.searchParams.set("q", "js & ts");        // encoded for you
await fetch(url);
```

CODE EXPLAINED

- → `res.ok` — check before reading the body
- → Headers object — case-insensitive keys
- → `204 No Content` — no body to parse
- → Build query strings with `URL` to get encoding right

TYPESCRIPT

- `res.json()` returns `Promise<any>`; validate it or type it: `(await res.json()) as User[]`
- Better: parse with a Zod schema (sheet 085) so the type is checked at runtime

REQUEST OPTIONS

OPTION	EXAMPLE
`method`	"POST", "PUT", "DELETE"
`headers`	{ "Content-Type": … }
`body`	string, FormData, Blob
`signal`	AbortSignal
`credentials`	"include" for cookies
`cache`	"no-store"

READING THE BODY

METHOD	RETURNS
`json()`	parsed JSON
`text()`	string
`blob()`	Blob (files, images)
`arrayBuffer()`	raw bytes
`formData()`	FormData

⚠ COMMON MISTAKES

- Assuming fetch throws on 404/500; it doesn't
- Reading the body twice; use `res.clone()` if you need it again
- Sending an object as `body` without `JSON.stringify`

✓ TIPS & BEST PRACTICES

- Wrap fetch in a small helper that checks `ok` and parses JSON
- Always set a timeout signal on requests to external services
- Use `FormData` as the body for file uploads; don't set Content-Type yourself

QUICK SUMMARY

- `fetch(url, opts)` — HTTP request
- check `res.ok` — 4xx/5xx resolve
- `json() / text()` — read once
- `signal` — timeouts and cancel

Storage & Cookies

OVERVIEW

Web Storage gives each origin a simple string key–value store. `localStorage` persists across sessions; `sessionStorage` lasts for one tab. Both are synchronous and limited to about 5 MB, so store JSON and keep it small. Cookies are sent to the server with every request. **IndexedDB** handles large or structured data.

NOTES

- Values are always strings; store objects with `JSON.stringify`
- Storage can be unavailable or throw (private mode, quota exceeded)
- The `storage` event fires in **other** tabs when a value changes
- Never store secrets or auth tokens in localStorage (XSS can read it)

STORAGE APIS

```
localStorage.setItem("theme", "dark");
localStorage.getItem("theme");              // "dark"
localStorage.getItem("missing");            // null
localStorage.removeItem("theme");

const prefs = { fontSize: 14, lang: "en" };
localStorage.setItem("prefs", JSON.stringify(prefs));

function load(key, fallback) {
  try { return JSON.parse(localStorage.getItem(key)) ?? fallback; }
  catch { return fallback; }
}
load("prefs", {}).fontSize;                 // 14

sessionStorage.setItem("step", "2");        // this tab only

window.addEventListener("storage", e => {  // sync across tabs
  if (e.key === "theme") applyTheme(e.newValue);
});

document.cookie = "seen=1; max-age=86400; path=/; SameSite=Lax";
document.cookie;                            // "seen=1; other=x"
await cookieStore?.get("seen");             // async Cookie Store API
```

CODE EXPLAINED

→ `getItem` returns `null` for missing keys

→ Safe `load` helper — handles missing and corrupt data

→ `storage` event — keeps multiple tabs in sync

→ `document.cookie` — assignment adds one cookie, read returns all

TYPESCRIPT

- `getItem` returns `string | null`; `JSON.parse` returns `any`, so validate or type the result
- Wrap storage in a typed helper: `load<T>(key: string, fallback: T): T`

STORAGE OPTIONS

STORE	LIFETIME	SIZE
`localStorage`	until cleared	~5 MB
`sessionStorage`	tab session	~5 MB
`cookies`	set by expiry	~4 KB each
`IndexedDB`	until cleared	large (GBs)
`Cache API`	until cleared	large

WEB STORAGE API

MEMBER	DOES
`setItem(k, v)`	store string
`getItem(k)`	read or null
`removeItem(k)`	delete
`clear()`	delete all
`length / key(i)`	iterate

⚠ COMMON MISTAKES

- Storing objects directly, which saves `"[object Object]"`
- Putting JWTs or passwords in localStorage
- Calling storage in Node or SSR code where `localStorage` doesn't exist

✓ TIPS & BEST PRACTICES

- Version your stored data (`prefs:v2`) so format changes don't break old users
- Use the `idb-keyval` library for an easy IndexedDB key–value API
- Use `HttpOnly` cookies (set by the server) for sessions

QUICK SUMMARY

- `localStorage` — persistent strings
- `sessionStorage` — per tab
- JSON — store objects
- IndexedDB — big/structured data

URL & History

OVERVIEW

The URL class parses and builds URLs safely, and URLSearchParams handles query strings with correct encoding. In the browser, `location` describes the current page, and the **History API** (`pushState`, `replaceState`, `popstate`) lets single-page apps change the address bar without reloading.

NOTES

- `new URL(path, base)` resolves relative URLs
- `URL.canParse(str)` checks validity without try/catch
- `pushState` doesn't fire `popstate`; only back/forward do
- `location.href = …` navigates; `location.replace` skips history

URLS, PARAMS, HISTORY

```
const u = new URL("https://shop.example.com:8080/items?id=7&sort=asc#top");
u.hostname;          // "shop.example.com"
u.port;              // "8080"
u.pathname;          // "/items"
u.hash;              // "#top"
u.searchParams.get("id");              // "7"
u.searchParams.set("page", "2");
u.searchParams.delete("sort");
u.toString();        // "https://shop.example.com:8080/items?id=7&page=2#top"

new URL("../img/a.png", "https://x.com/docs/page/").href;
// "https://x.com/docs/img/a.png"

const params = new URLSearchParams({ q: "café & tea", n: 5 });
params.toString();   // "q=caf%C3%A9+%26+tea&n=5"
Object.fromEntries(new URLSearchParams(location.search));

history.pushState({ tab: "reviews" }, "", "?tab=reviews");
window.addEventListener("popstate", e => showTab(e.state?.tab));
location.reload();
```

CODE EXPLAINED

- → URL parts are properties you can read or set
- → `searchParams` — live view of the query string
- → `URLSearchParams` — encodes spaces, &, accents
- → `pushState(state, "", url)` — change URL, keep page

TYPESCRIPT

- `searchParams.get` returns `string | null`
- `history.state` is `any`; type your state objects when reading them

URL PROPERTIES

PROPERTY	EXAMPLE
`href`	full URL
`origin`	https://x.com:8080
`protocol`	https:
`host / hostname`	x.com:8080 / x.com
`pathname`	/items
`search / hash`	?id=7 / #top

HISTORY API

CALL	DOES
`history.pushState(s, "", url)`	new entry
`history.replaceState(…)`	edit current entry
`history.back() / forward()`	navigate
`popstate event`	back/forward pressed

⚠ COMMON MISTAKES

- Building query strings by concatenation and missing encoding
- Expecting `popstate` after `pushState`
- Parsing URLs with regex instead of the `URL` class

✓ TIPS & BEST PRACTICES

- Keep UI state like filters and tabs in the URL so pages are shareable
- Use `replaceState` for state changes that shouldn't add history entries
- Check out the Navigation API for richer SPA routing where supported

QUICK SUMMARY

- `URL` — parse and build
- `URLSearchParams` — query strings
- `location` — current page
- `pushState / popstate` — SPA history

Observers & Animation Frames

OVERVIEW

Observers let the browser tell you when something changes instead of you polling for it. `IntersectionObserver` reports when elements enter or leave the viewport (lazy loading, infinite scroll, scroll animations). `ResizeObserver` reports element size changes, and `MutationObserver` reports DOM changes. `requestAnimationFrame` schedules work right before the next paint.

NOTES

- Observers batch notifications into an array of entries
- Call `unobserve(el)` or `disconnect()` when done
- Observers are far cheaper than `scroll` / `resize` listeners
- `rAF` callbacks pause in background tabs

OBSERVERS AND ANIMATION FRAMES

```
const io = new IntersectionObserver((entries) => {
  for (const e of entries) {
    if (!e.isIntersecting) continue;
    e.target.src = e.target.dataset.src;       // lazy-load image
    io.unobserve(e.target);
  }
}, { rootMargin: "200px", threshold: 0.1 });
document.querySelectorAll("img[data-src]").forEach(img => io.observe(img));

const ro = new ResizeObserver(([entry]) => {
  const { width } = entry.contentRect;
  entry.target.classList.toggle("compact", width < 400);
});
ro.observe(sidebar);

const mo = new MutationObserver(muts => console.log(muts.length, "changes"));
mo.observe(list, { childList: true, subtree: true, attributes: true });
mo.disconnect();

function frame(time) {
  ball.style.transform = `translateX(${(time / 10) % 300}px)`;
  requestAnimationFrame(frame);
}
requestAnimationFrame(frame);
```

CODE EXPLAINED

- → `isIntersecting` — element is in view
- → `rootMargin` — start loading before it's visible
- → `contentRect` — the element's new size
- → `rAF` — one call per frame, synced with the display

TYPESCRIPT

- Entry types are provided: `IntersectionObserverEntry`, `ResizeObserverEntry`, `MutationRecord`
- `e.target` is `Element`; cast to `HTMLImageElement` to set `src`

OBSERVERS

OBSERVER	WATCHES
`IntersectionObserver`	visibility in viewport
`ResizeObserver`	element size
`MutationObserver`	DOM tree/attributes
`PerformanceObserver`	timing entries

INTERSECTIONOBSERVER OPTIONS

OPTION	MEANS
`root`	scroll container (default viewport)
`rootMargin`	grow/shrink the root box
`threshold`	0–1 visible fraction

⚠ COMMON MISTAKES

- Never disconnecting observers on removed elements
- Changing an element's size inside its own ResizeObserver callback (loop warnings)
- Animating with `setInterval` instead of `requestAnimationFrame`

✓ TIPS & BEST PRACTICES

- Use `loading="lazy"` on images first; use IntersectionObserver for custom cases
- Prefer CSS container queries over ResizeObserver for pure styling
- Use the `time` argument in rAF for frame-rate-independent animation

QUICK SUMMARY

- `IntersectionObserver` — visibility
- `ResizeObserver` — size
- `MutationObserver` — DOM changes
- `requestAnimationFrame` — per-frame work

process & Environment

OVERVIEW

The global `process` object connects a Node program to its environment: command-line arguments (`argv`), environment variables (`env`), standard streams, the working directory, and the exit code. Node can load `.env` files natively with `--env-file`, and `util.parseArgs` parses flags without a library.

NOTES

- `process.env` values are always strings (or undefined)
- Set `process.exitCode` instead of calling `exit()` so pending output flushes
- Exit code 0 means success; anything else is failure
- `SIGINT` is Ctrl+C; handle it for graceful shutdown

ARGUMENTS, ENV, EXIT

```
// $ node --env-file=.env app.js build --port 8080 -v
process.argv.slice(2);   // ["build", "--port", "8080", "-v"]

import { parseArgs } from "node:util";
const { values, positionals } = parseArgs({
  options: { port: { type: "string", default: "3000" },
             verbose: { type: "boolean", short: "v" } },
  allowPositionals: true,
});
// values = { port: "8080", verbose: true }, positionals = ["build"]

const port = Number(process.env.PORT ?? 3000);
const isProd = process.env.NODE_ENV === "production";
if (!process.env.API_KEY) {
  console.error("API_KEY is required");
  process.exitCode = 1;
}

process.stdout.write("no newline");
process.cwd();  process.platform;     // "/proj", "linux"
process.on("SIGINT", async () => { await server.close(); process.exit(0); });
```

CODE EXPLAINED

→ `--env-file` — loads `.env` into `process.env` (Node 20.6+)

→ `parseArgs` — typed flags, defaults, short aliases

→ `exitCode = 1` — fail without cutting off output

→ `SIGINT` handler — clean shutdown on Ctrl+C

TYPESCRIPT

- `process.env.X` is `string | undefined`; validate env at startup (Zod works well)
- `parseArgs` infers value types from the `options` config

PROCESS MEMBERS

MEMBER	GIVES
`argv`	command-line array
`env`	environment variables
`cwd()`	working directory
`exit(code)`	exit now
`exitCode`	exit status at end
`platform / arch`	"win32", "x64"…
`memoryUsage()`	heap and RSS

STANDARD STREAMS

STREAM	USE
`process.stdin`	input
`process.stdout`	normal output
`process.stderr`	errors

⚠ COMMON MISTAKES

- Using `process.env.PORT + 1` and getting `"30001"`
- Calling `process.exit()` right after writing output, truncating it
- Committing `.env` files with secrets

✓ TIPS & BEST PRACTICES

- Validate all required env vars once at startup and fail fast
- Commit a `.env.example` listing the variables without values
- Return non-zero exit codes from CLIs so scripts can detect failure

QUICK SUMMARY

- `process.argv` — arguments
- `process.env` — config (strings)
- `parseArgs` — flag parsing
- `exitCode` — signal success/failure

EventEmitter & Streams

OVERVIEW

Much of Node is built on **EventEmitter**: objects emit named events and listeners react with `on`. **Streams** are emitters that move data in chunks, letting you process files and network data larger than memory. Connect them with `pipeline`, which handles errors and **backpressure** (slowing a fast producer for a slow consumer).

NOTES

- Four stream types: Readable, Writable, Duplex, Transform
- An `error` event with no listener crashes the process
- Readable streams are async iterable: `for await (const chunk of r)`
- Use `stream/promises` for the Promise-based `pipeline`

EMITTERS AND STREAM PIPELINES

```
import { EventEmitter, once } from "node:events";
import { pipeline } from "node:stream/promises";
import { createReadStream, createWriteStream } from "node:fs";
import { createGzip } from "node:zlib";
import { Transform } from "node:stream";

class Job extends EventEmitter {
  run() { this.emit("progress", 50); this.emit("done", { ok: true }); }
}
const job = new Job();
job.on("progress", pct => console.log(`${pct}%`));
job.once("done", result => console.log(result));
job.on("error", err => console.error(err));     // always handle
job.run();
const [result] = await once(job, "done");       // await an event

const upper = new Transform({
  transform(chunk, enc, cb) { cb(null, chunk.toString().toUpperCase()); },
});
await pipeline(
  createReadStream("in.txt"), upper, createGzip(),
  createWriteStream("out.txt.gz"),
);                                              // errors reject here
```

CODE EXPLAINED

- → `emit(name, ...args)` — call every listener
- → `once` — listen one time; `events.once` returns a Promise
- → Transform — modify chunks as they pass
- → `pipeline` — wires streams, handles errors and backpressure

TYPESCRIPT

- Type events with a generic map (Node 20.11+ `@types/node`): `class Job extends EventEmitter<{ done: [Result] }>`
- Streams are typed in `@types/node`; chunks are `Buffer` unless an encoding is set

EVENTEMITTER API

METHOD	DOES
`on(e, fn)`	add listener
`once(e, fn)`	listen once
`off(e, fn)`	remove listener
`emit(e, ...a)`	fire event
`listenerCount(e)`	count

STREAM TYPES

TYPE	EXAMPLE
`Readable`	fs.createReadStream
`Writable`	fs.createWriteStream
`Duplex`	net.Socket
`Transform`	zlib.createGzip

⚠ COMMON MISTAKES

- Not listening for `error` events, so one failure crashes everything
- Using `.pipe()` chains, which don't forward errors; use `pipeline`
- Adding listeners in a loop and triggering MaxListenersExceeded warnings

✓ TIPS & BEST PRACTICES

- Use `for await` over readable streams for simple line or chunk processing
- Prefer `pipeline` from `node:stream/promises` in async code
- Remove listeners (or use `once`) for short-lived subscriptions

QUICK SUMMARY

- EventEmitter — on / emit
- streams — chunked data
- `pipeline` — connect safely
- handle `error` — always

HTTP Server (node:http)

OVERVIEW

Node's built-in `node:http` module creates web servers with no dependencies. `createServer` takes a handler that receives a **request** (`IncomingMessage`) and a **response** (`ServerResponse`). You route on `req.method` and the URL, read bodies from the request stream, and set status and headers before ending the response. Frameworks like Express and Fastify build on this.

NOTES

- The request body arrives as a stream; collect it before parsing
- Call `res.end()` exactly once per request
- `req.url` is only path and query; combine with a base to parse
- Node 22+ can also run Web-standard handlers via frameworks like Hono

A TINY JSON API

```ts
import { createServer } from "node:http";

const todos = [{ id: 1, title: "Learn Node" }];

const server = createServer(async (req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);
  const send = (status, data) => {
    res.writeHead(status, { "Content-Type": "application/json" });
    res.end(JSON.stringify(data));
  };

  if (req.method === "GET" && url.pathname === "/todos") {
    return send(200, todos);
  }
  if (req.method === "POST" && url.pathname === "/todos") {
    let body = "";
    for await (const chunk of req) body += chunk;
    const todo = { id: todos.length + 1, ...JSON.parse(body) };
    todos.push(todo);
    return send(201, todo);
  }
  send(404, { error: "Not found" });
});

server.listen(3000, () => console.log("http://localhost:3000"));
```

CODE EXPLAINED

- → `new URL(req.url, base)` — easy path and query parsing
- → `writeHead` — status plus headers in one call
- → `for await` over `req` — read the body
- → Return after sending so only one response goes out

TYPESCRIPT

- Handler params are `IncomingMessage` and `ServerResponse` from `node:http`
- Parsed bodies are `any`; validate them before use

REQUEST (REQ)

MEMBER	GIVES
`req.method`	"GET", "POST"…
`req.url`	"/todos?x=1"
`req.headers`	lowercase header object
`for await (chunk of req)`	body chunks

RESPONSE (RES)

MEMBER	DOES
`res.statusCode = 200`	set status
`res.setHeader(k, v)`	set header
`res.writeHead(s, h)`	status + headers
`res.write(data)`	send chunk
`res.end(data)`	finish response

⚠ COMMON MISTAKES

- Forgetting `res.end()`, leaving requests hanging until timeout
- Calling `JSON.parse` on bodies without try/catch
- Sending a response twice ("headers already sent" error)

✓ TIPS & BEST PRACTICES

- Use a framework (Express, Fastify, Hono) for anything beyond a few routes
- Limit body size to protect against huge uploads
- Use `node --watch server.js` during development

QUICK SUMMARY

- `createServer(handler)` — server
- `req` — method, url, headers, body
- `res.writeHead / end` — respond
- `listen(port)` — start

Node Utility Modules

OVERVIEW

Node ships many small utility modules that replace common npm packages. `node:os` reports system info, `node:util` has helpers like `promisify`, `inspect`, `parseArgs`, and `styleText`. `node:crypto` covers hashing, random values, and UUIDs, and `node:child_process` runs other programs.

NOTES

- Always import built-ins with the `node:` prefix
- `util.promisify` converts callback APIs to Promises
- `execFile` is safer than `exec` (no shell interpretation)
- `util.styleText` adds terminal colors without chalk

UTILITY MODULES

```
import os from "node:os";
import { promisify, inspect, styleText, isDeepStrictEqual } from "node:util";
import { createHash, randomUUID, randomBytes } from "node:crypto";
import { execFile } from "node:child_process";

os.platform();  os.cpus().length;   // "linux", 8
os.totalmem() / 1e9;                // 16 (GB)
os.homedir();  os.EOL;              // "/home/me", "\n"

console.log(styleText("green", "✔ passed"));
console.log(inspect(deepObj, { depth: null, colors: true }));
isDeepStrictEqual({ a: [1] }, { a: [1] });   // true

createHash("sha256").update("hello").digest("hex");
// "2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824"
randomUUID();                       // "9b1d…"
randomBytes(16).toString("base64url");       // URL-safe token

const run = promisify(execFile);
const { stdout } = await run("git", ["rev-parse", "--short", "HEAD"]);
stdout.trim();                      // "a1b2c3d"
```

CODE EXPLAINED

- → `os` — CPU, memory, platform facts
- → `styleText` — colors that respect `NO_COLOR`
- → `createHash` — checksums and fingerprints
- → `execFile` + `promisify` — run commands, await output

TYPESCRIPT

- All typed through `@types/node`
- `promisify` keeps types for standard callback signatures; custom ones may need annotations

HANDY BUILT-INS

MODULE	REPLACES
`node:test`	basic test runners
`node:util parseArgs`	yargs/commander (simple CLIs)
`node:util styleText`	chalk
`node:crypto randomUUID`	uuid
`fetch (global)`	axios/node-fetch
`--env-file`	dotenv
`--watch`	nodemon

CHILD_PROCESS

FUNCTION	USE
`execFile(cmd, args)`	run program, get output
`spawn(cmd, args)`	stream output
`exec(cmdString)`	via shell (careful)
`fork(module)`	Node child with IPC

⚠ COMMON MISTAKES

- Passing user input to `exec`, which allows shell injection
- Using `crypto.createHash("md5")` for passwords; use `scrypt` or argon2
- Installing packages for features Node already has

✓ TIPS & BEST PRACTICES

- Check the built-ins table before adding a dependency
- Hash passwords with `crypto.scrypt` and a random salt
- Use `spawn` with `stdio: "inherit"` to show a child's output live

QUICK SUMMARY

- `os` — system info
- `util` — promisify, inspect, styleText
- `crypto` — hashes, UUIDs, tokens
- `child_process` — run programs

TypeScript: Introduction & Compiler

OVERVIEW

TypeScript adds a static type system to JavaScript. You write `.ts` files with type annotations; the compiler `tsc` **checks** them and **emits** plain JavaScript with the types erased. Types never exist at runtime, so they can't validate data from outside. Many runtimes now **strip** types and run `.ts` directly, but stripping doesn't check anything; only `tsc` does.

NOTES

- Every valid JS program is valid TS (possibly with type errors)
- Types cost nothing at runtime; they're gone after compilation
- Type stripping only works for "erasable" syntax (no enums, namespaces, parameter properties)
- A native Go-based compiler (TypeScript 7) is in progress; check its status

FROM .TS TO .JS

```ts
// greet.ts
interface User { name: string; age?: number }

function greet(user: User): string {
  return `Hi ${user.name}`;
}
greet({ name: "Ana" });          // OK
// greet({ nam: "Ana" });        // error: 'nam' does not exist in type 'User'

// $ npx tsc greet.ts            → greet.js (types erased):
function greet(user) {
  return `Hi ${user.name}`;
}

// $ npx tsc --noEmit            // type-check only, no output
// $ node greet.ts               // Node strips types and runs (no checking)
// $ npx tsx greet.ts            // run in dev with esbuild
// $ deno run greet.ts  /  bun greet.ts
```

CODE EXPLAINED

- → `interface` / annotations — compile-time only
- → Errors appear in the editor and in `tsc` output
- → Emitted JS has no types
- → `--noEmit` — use tsc purely as a checker
- → Runtimes that strip types skip checking entirely

FILE TYPES

EXTENSION	CONTAINS
`.ts`	TypeScript
`.tsx`	TS + JSX
`.d.ts`	types only (declarations)
`.mts / .cts`	ESM / CommonJS TS

WAYS TO RUN

TOOL	CHECKS TYPES?
`tsc`	yes (and emits JS)
`tsc --noEmit`	yes, no output
`node file.ts`	no (strips)
`tsx / esbuild / Vite`	no (strips)
`IDE (tsserver)`	yes, live

⚠ COMMON MISTAKES

- Assuming types validate API responses at runtime; they don't
- Running with tsx or Vite only and never running `tsc`, so type errors ship
- Using enums or namespaces with Node type stripping, which can't erase them

✓ TIPS & BEST PRACTICES

- Add `tsc --noEmit` to CI and a pre-commit hook
- Turn on `erasableSyntaxOnly` if you run TS through type stripping
- Start new projects with `strict: true` from day one

QUICK SUMMARY

- TS = JS + static types
- `tsc` — check and emit
- types erased — nothing at runtime
- stripping — runs, doesn't check

tsconfig.json

OVERVIEW

`tsconfig.json` controls how TypeScript checks and compiles your project. The most important setting is `"strict": true`, which turns on a family of safety checks. `target` sets the JS version to emit, `module`/`moduleResolution` set how imports work, and `lib` decides which built-in APIs (DOM, ES2025) are known.

NOTES

- Create one with `npx tsc --init`
- `strict` enables `strictNullChecks`, `noImplicitAny`, and more
- `extends` lets you share a base config
- Use `@tsconfig/*` packages for good presets (node22, vite-react…)

A MODERN TSCONFIG.JSON

```jsonc
{
  "compilerOptions": {
    "target": "ES2023",
    "module": "nodenext",                // Node ESM/CJS rules
    "moduleResolution": "nodenext",
    "lib": ["ES2024", "DOM"],
    "strict": true,
    "noUncheckedIndexedAccess": true,  // arr[i] may be undefined
    "exactOptionalPropertyTypes": true,
    "verbatimModuleSyntax": true,      // require `import type`
    "erasableSyntaxOnly": true,        // safe for type stripping
    "skipLibCheck": true,              // skip .d.ts in node_modules
    "outDir": "dist",
    "rootDir": "src",
    "declaration": true,               // emit .d.ts (libraries)
    "sourceMap": true
  },
  "include": ["src"],
  "exclude": ["dist", "node_modules"]
}
```

CODE EXPLAINED

- → `target` — syntax level of the output
- → `module: nodenext` — matches how Node really resolves imports
- → `noUncheckedIndexedAccess` — catches out-of-range reads
- → `skipLibCheck` — faster builds, fewer third-party errors
- → `include` — which files belong to the project

WHAT STRICT ENABLES

FLAG	CATCHES
`noImplicitAny`	untyped params
`strictNullChecks`	null/undefined misuse
`strictFunctionTypes`	unsafe callbacks
`strictPropertyInitialization`	unset class fields
`useUnknownInCatchVariables`	catch (e) as unknown
`noImplicitThis`	untyped this

MODULE SETTINGS

PROJECT	USE
`Node app/library`	nodenext
`Vite/bundler app`	esnext + moduleResolution: bundler
`Legacy CommonJS`	commonjs

⚠ COMMON MISTAKES

- Leaving `strict` off and inheriting a pile of hidden `any`s
- Mismatched `module` settings that make imports work in the editor but fail at runtime
- Forgetting `DOM` in `lib` for browser code (or including it in Node-only code)

✓ TIPS & BEST PRACTICES

- Turn strictness flags on early; adding them later means fixing hundreds of errors
- Use `tsc --showConfig` to see the final merged settings
- Separate `tsconfig.build.json` for emitting, base config for editor/checking

QUICK SUMMARY

- `strict: true` — essential
- `target` — output JS level
- `module` — import rules
- `lib` — known built-in APIs

Basic Types

OVERVIEW

Basic annotations use a colon: `let x: number`. The primitive types are lowercase: `string`, `number`, `boolean`, `bigint`, `symbol`, `null`, `undefined`. Arrays are `T[]`, and **tuples** fix both length and position types. Special types: `any` turns checking off, `unknown` is a safe "could be anything", `void` means no return value, and `never` means "can't happen".

NOTES

- Prefer `unknown` over `any` for values you haven't checked
- `any` spreads silently through everything it touches
- `object` means any non-primitive
- Lowercase types only; `String` / `Number` are wrapper objects

BASIC TYPES

```ts
let title: string = "Report";
let count: number = 3;
let done: boolean = false;
let big: bigint = 10n;

let tags: string[] = ["a", "b"];
let ids: Array<number> = [1, 2];
let point: [number, number] = [3, 4];              // tuple
let entry: [name: string, age?: number] = ["Ana"]; // labeled, optional
const fixed: readonly string[] = ["x"];

let anything: any = 4;
anything.foo.bar();                // no error (and a runtime crash)
let input: unknown = JSON.parse(text);
// input.name;                     // error: 'input' is of type 'unknown'
if (typeof input === "object" && input !== null && "name" in input) {
  input.name;                      // OK after narrowing
}

function log(msg: string): void { console.log(msg); }
function fail(msg: string): never { throw new Error(msg); }
```

CODE EXPLAINED

- → `T[]` and `Array<T>` — identical
- → Tuples — exact length and per-position types
- → `any` — disables checking; avoid
- → `unknown` — must narrow before use
- → `never` — function never returns normally

TYPE REFERENCE

TYPE	VALUES
`string / number / boolean`	primitives
`null / undefined`	nullish
`T[] / readonly T[]`	arrays
`[A, B]`	tuple
`any`	anything, unchecked
`unknown`	anything, must check
`void / never`	no value / impossible

ANY VS UNKNOWN

	ANY	UNKNOWN
`assign anything to it`	yes	yes
`use without checks`	yes	no
`assign it anywhere`	yes	only to unknown/any
`safe`	no	yes

⚠ COMMON MISTAKES

- Using `any` to silence errors instead of fixing types
- Writing `String` or `Object` (wrapper types) instead of `string` or `object`
- Annotating everything; let inference handle obvious locals

✓ TIPS & BEST PRACTICES

- Type function parameters and exports; let TS infer locals
- Use `unknown` for parsed JSON and caught errors
- Enable typescript-eslint's `no-explicit-any` as a warning

QUICK SUMMARY

- `: Type` — annotation
- `T[]` / tuples — arrays
- `unknown` over `any`
- `void` / `never` — returns

Sheet 114

Inference, Widening & satisfies

inference · widening · literal types · as const · satisfies · contextual typing

OVERVIEW

TypeScript **infers** most types from values, so you rarely annotate locals. `const` keeps **literal** types (`"GET"`), while `let` **widens** to the general type (`string`). `as const` freezes an expression into its narrowest readonly form. `satisfies` checks a value against a type **without** widening it, so you keep precise inferred types.

NOTES

- Return types are inferred from `return` statements
- Callback parameters get types from context (contextual typing)
- Empty arrays infer as `any[]` or `never[]`; annotate them
- Hover in your editor to see inferred types

INFERENCE, AS CONST, SATISFIES

```ts
let a = 5;                      // number
const b = 5;                    // 5 (literal)
let method = "GET";             // string (widened)
const m2 = "GET";               // "GET"
const arr = [1, "x"];           // (string | number)[]

[1, 2].map(n => n * 2);         // n inferred as number from context

const routes = ["home", "about"] as const;
// readonly ["home", "about"]
type Route = (typeof routes)[number];        // "home" | "about"

const config = { mode: "dark", size: 12 } as const;
// { readonly mode: "dark"; readonly size: 12 }

type Colors = Record<string, string | [number, number, number]>;
const palette = {
  red: [255, 0, 0],
  blue: "#00f",
} satisfies Colors;
palette.red[0];                 // OK: still known as a tuple
palette.blue.toUpperCase();     // OK: still known as string
// with ": Colors" both would be string | tuple
```

CODE EXPLAINED

- → `const` — literal type; `let` — widened type
- → `as const` — readonly, literal everything
- → `(typeof x)[number]` — union of array element types
- → `satisfies` — validates shape, keeps precise types
- → Annotation (`: Colors`) — widens to the declared type

ANNOTATION VS SATISFIES VS AS CONST

FORM	CHECKS?	KEEPS NARROW TYPE?
`const x: T = v`	yes	no (becomes T)
`const x = v satisfies T`	yes	yes
`const x = v as const`	no	yes, readonly
`const x = v as T`	barely	no (assertion)

WHEN TO ANNOTATE

PLACE	ANNOTATE?
`local variables`	rarely
`function params`	always
`exported function returns`	recommended
`empty arrays / objects`	yes

⚠ COMMON MISTAKES

- Annotating with a broad type and losing useful literal information
- Using `as` to "fix" a type error; that's an assertion, not a check
- `const list = []` then pushing values; annotate `const list: string[] = []`

✓ TIPS & BEST PRACTICES

- Use `as const` for lookup tables and option lists
- Use `satisfies` for config objects
- Derive types from values (`typeof`) so they never drift

QUICK SUMMARY

- inference — types from values
- `const` — literal types
- `as const` — narrow + readonly
- `satisfies` — check without widening

Type Aliases & Interfaces

OVERVIEW

Name your types with **type aliases** (`type`) or **interfaces** (`interface`). Both describe object shapes and can be extended. Interfaces support **declaration merging** and `extends`; type aliases can name anything: unions, tuples, primitives, mapped and conditional types. Use whichever your team prefers for objects and `type` for everything else.

NOTES

- Interfaces with the same name merge automatically
- `extends` on interfaces reports conflicts clearly
- Intersections (`&`) combine type aliases
- Both are erased at runtime

TYPE VS INTERFACE

```ts
interface User {
  id: number;
  name: string;
}
interface Admin extends User {          // inheritance
  permissions: string[];
}

type Point = { x: number; y: number };
type Point3D = Point & { z: number };   // intersection
type ID = string | number;              // union: type only
type Pair = [string, number];           // tuple: type only
type Handler = (e: Event) => void;      // function type

interface Window { appVersion: string } // merges with built-in Window
window.appVersion = "1.2.0";            // now allowed

const a: Admin = { id: 1, name: "Bo", permissions: ["write"] };
function move(p: Point): Point3D { return { ...p, z: 0 }; }

interface Repo<T> { find(id: ID): T | undefined }  // generic
```

CODE EXPLAINED

- → `interface X extends Y` — add fields to a base
- → `type A = B & C` — combine with intersection
- → Unions, tuples, functions — need `type`
- → Merging — extend global or library interfaces
- → Both can be generic

COMPARISON

FEATURE	INTERFACE	TYPE
object shapes	✔	✔
extends	extends	&
unions / tuples	✖	✔
declaration merging	✔	✖
mapped / conditional	✖	✔
implements in class	✔	✔ (object types)

RULE OF THUMB

DESCRIBING	USE
public object API	interface
union of options	type
function signature	type
extending a library type	interface (merge)

⚠ COMMON MISTAKES

- Accidentally merging two interfaces with the same name in one scope
- Using huge `&` intersections whose conflicts produce `never` fields
- Mixing styles randomly across a codebase

✓ TIPS & BEST PRACTICES

- Pick a convention and enforce it with typescript-eslint `consistent-type-definitions`
- Use interfaces for objects that classes implement
- Name types as nouns; avoid `I` prefixes like `IUser`

QUICK SUMMARY

- `type` — name any type
- `interface` — object shapes, mergeable
- `extends` / `&` — combine
- both erased at runtime

Object Types

OVERVIEW

Object types describe properties and their types. Mark properties **optional** with ? and **read-only** with `readonly`. **Index signatures** describe dictionaries with unknown keys, and `Record<K, V>` is the shorthand. Fresh object literals get **excess property checks**, which catch typos in property names.

NOTES

- `readonly` is compile-time only
- Optional `?:` allows the property to be missing
- Index signatures make every key's value potentially `undefined` with `noUncheckedIndexedAccess`
- Excess property checks apply only to object literals assigned directly

OBJECT TYPE FEATURES

```ts
interface Product {
  readonly id: number;
  name: string;
  price: number;
  description?: string;                 // optional
  tags: readonly string[];
}

const p: Product = { id: 1, name: "Pen", price: 2, tags: [] };
// p.id = 2;                            // error: read-only
p.description?.toUpperCase();           // may be undefined

type Scores = { [player: string]: number };   // index signature
type Inventory = Record<string, number>;      // same idea
const stock: Inventory = { pens: 40, pads: 12 };
stock.erasers;                          // number | undefined (strict index)

type Flags = Record<"darkMode" | "beta", boolean>;  // fixed keys
const f: Flags = { darkMode: true, beta: false };

// const q: Product = { id: 2, name: "Pad", price: 4, tags: [], colour: "red" };
// error: Object literal may only specify known properties
const extra = { id: 2, name: "Pad", price: 4, tags: [], colour: "red" };
const q: Product = extra;               // OK: not a fresh literal
```

CODE EXPLAINED

- → `readonly` — no reassignment after creation
- → `?:` — property may be absent
- → `Record<string, V>` — open dictionary
- → `Record<"a" | "b", V>` — required fixed keys
- → Excess checks catch typos in literals

PROPERTY SYNTAX

SYNTAX	MEANS
`name: T`	required
`name?: T`	optional
`readonly name: T`	can't reassign
`[key: string]: T`	any string key
`[key: on${string}]: T`	pattern keys

DICTIONARY TYPES

TYPE	USE
`Record<string, T>`	open map
`Record<K, T>`	every K required
`Partial<Record<K, T>>`	some K
`Map<K, T>`	runtime map

⚠ COMMON MISTAKES

- Assuming `readonly` protects data at runtime; use `Object.freeze` for that
- Using index signatures when the keys are known; list them explicitly
- Confusing optional (`a?: T`) with nullable (`a: T | null`)

✓ TIPS & BEST PRACTICES

- Make properties `readonly` by default in data objects
- Enable `exactOptionalPropertyTypes` to forbid assigning `undefined` to optional props
- Use `Map` when keys are dynamic and frequently changing

QUICK SUMMARY

- `?` — optional
- `readonly` — no reassign
- index signature / `Record` — dictionaries
- excess checks — catch typos

Unions & Intersections

OVERVIEW

A **union** (`A | B`) is a value that can be one of several types; you must narrow it before using type-specific members. An **intersection** (`A & B`) has all members of both. **Discriminated unions** give each variant a shared literal field (like `kind`), which makes narrowing automatic and lets TypeScript check that every case is handled.

NOTES

- Only members common to all union variants are usable without narrowing
- Literal unions replace most enums: `"sm" | "md" | "lg"`
- Intersections of conflicting primitives become `never`
- Assigning to `never` in `default` forces exhaustive handling

UNIONS AND DISCRIMINATED UNIONS

```ts
type Size = "sm" | "md" | "lg";
function pad(value: string | number): string {
  return typeof value === "number" ? " ".repeat(value) : value;
}

type Shape =
  | { kind: "circle"; r: number }
  | { kind: "rect"; w: number; h: number }
  | { kind: "tri"; base: number; h: number };

function area(s: Shape): number {
  switch (s.kind) {
    case "circle": return Math.PI * s.r ** 2;     // s is circle here
    case "rect":   return s.w * s.h;
    case "tri":    return (s.base * s.h) / 2;
    default: return s satisfies never;           // errors if a case is missing
  }
}

type Timestamped = { createdAt: Date };
type Post = { title: string } & Timestamped;      // intersection
type Result<T> = { ok: true; value: T } | { ok: false; error: string };
```

CODE EXPLAINED

- → `A | B` — one of; narrow before using specifics
- → `kind` — the discriminant field
- → Each `case` narrows to one variant
- → `never` check — compile error when a new variant is unhandled
- → `A & B` — both at once

UNION VS INTERSECTION

	A | B	A & B
`value is`	A or B	A and B
`usable members`	common ones	all of them
`typical use`	variants, options	mixing in fields

COMMON UNIONS

TYPE	USE
`"a" | "b"`	string options
`T | null`	maybe-missing value
`T | T[]`	one or many
`Success | Failure`	results
`Loading | Loaded | Error`	UI states

⚠ COMMON MISTAKES

- Accessing a property that exists on only one variant without narrowing
- Modeling states with several optional booleans (`isLoading`, `isError`) instead of a union
- Forgetting a `never` check, so new variants silently fall through

✓ TIPS & BEST PRACTICES

- Model state machines and API results as discriminated unions
- Use a consistent discriminant name (`kind`, `type`, or `status`)
- Prefer literal unions to enums for simple string options

QUICK SUMMARY

- `|` — one of several
- `&` — all combined
- discriminant field — auto narrowing
- `never` — exhaustiveness

Narrowing & Type Guards

typeof · instanceof · in · equality · type predicates is · asserts · control-flow analysis

OVERVIEW

Narrowing is how TypeScript refines a broad type to a specific one inside a branch. It follows your runtime checks: `typeof`, `instanceof`, `in`, equality, truthiness, and assignments. For checks it can't understand on its own, write a **type predicate** (`x is T`) or an **assertion function** (`asserts x is T`).

NOTES

- Narrowing follows control flow, including early returns
- Narrowing resets in callbacks if the variable can be reassigned
- Type predicates are trusted, so make sure they're correct
- TS 5.5+ infers predicates from simple `filter` callbacks

NARROWING TECHNIQUES

```ts
function format(x: string | number | Date | null) {
  if (x === null) return "—";                  // equality
  if (typeof x === "string") return x.trim();  // typeof
  if (x instanceof Date) return x.toISOString(); // instanceof
  return x.toFixed(2);                         // only number left
}

type Fish = { swim(): void };
type Bird = { fly(): void };
function move(a: Fish | Bird) {
  if ("swim" in a) a.swim(); else a.fly();      // in
}

function isString(v: unknown): v is string {    // type predicate
  return typeof v === "string";
}
const values: unknown[] = ["a", 1, "b"];
const strs = values.filter(isString);           // string[]

function assertUser(v: unknown): asserts v is { id: number } {
  if (!v || typeof v !== "object" || !("id" in v)) throw new Error("bad");
}
assertUser(data);  data.id;                     // narrowed after the call
```

CODE EXPLAINED

- → Each check removes possibilities from the union
- → `in` — narrow by property presence
- → `v is T` — reusable custom check
- → Predicates make `filter` return the narrowed type
- → `asserts` — narrows everything after the call

NARROWING GUARDS

GUARD	NARROWS BY
`typeof x === "string"`	primitive type
`x instanceof Date`	class
`"key" in x`	property present
`x.kind === "a"`	discriminant
`isFoo(x) (x is Foo)`	custom predicate

PREDICATE VS ASSERTION

FORM	RETURNS
`(x): x is T`	boolean, narrows in if
`(x): asserts x is T`	void or throws
`(x): asserts x`	truthy check

⚠ COMMON MISTAKES

- Writing a type predicate whose logic doesn't match its claim
- Expecting narrowing to persist inside a callback for a `let` variable
- Using `as` instead of a real check

✓ TIPS & BEST PRACTICES

- Prefer discriminated unions; their narrowing is automatic
- Use assertion functions at the top of handlers for input validation
- Combine with Zod: `schema.parse` returns already-narrowed data

QUICK SUMMARY

- control flow — checks narrow types
- `typeof / instanceof / in` — built-in guards
- `x is T` — custom predicate
- `asserts x is T` — throw or narrow

Function Types & Overloads

signatures · optional / default params · overloads · this parameter · void vs undefined · call signatures

OVERVIEW

Function types describe parameters and return values: `(a: number, b: number) => number`. Parameters can be **optional** (`?`), have **defaults**, or be **rest** arrays. **Overloads** declare several call signatures for one implementation. You can also type `this` explicitly and describe objects that are callable or constructable.

NOTES

- A `void` return type allows callbacks that return values (they're ignored)
- Parameter names in function types are just labels
- Overload signatures are visible; the implementation signature isn't
- Optional params must come after required ones

TYPING FUNCTIONS

```
function add(a: number, b: number): number { return a + b; }
const mul = (a: number, b: number): number => a * b;
type BinaryOp = (a: number, b: number) => number;
const sub: BinaryOp = (a, b) => a - b;          // params inferred

function greet(name: string, greeting = "Hi", punct?: string) {
  return `${greeting}, ${name}${punct ?? "!"}`;
}
function sum(...nums: number[]): number { return nums.reduce((t, n) => t + n, 0); }

function parse(input: string): number;          // overloads
function parse(input: string[]): number[];
function parse(input: string | string[]) {
  return Array.isArray(input) ? input.map(Number) : Number(input);
}
parse("4");            // number
parse(["1", "2"]);     // number[]

function onClick(this: HTMLButtonElement, e: MouseEvent) { this.disabled = true; }

type Logger = { (msg: string): void; level: "info" | "debug" };  // callable
type Ctor<T> = new (...args: any[]) => T;       // constructable
```

CODE EXPLAINED

- → Function type alias — reuse a signature
- → Contextual typing — `sub`'s params need no annotations
- → Overloads — callers see precise return types
- → `this` parameter — erased in output, checked in TS
- → Call and construct signatures — callable objects and classes

PARAMETER FORMS

SYNTAX	MEANS
`a: T`	required
`a?: T`	optional (T | undefined)
`a: T = v`	default (optional)
`...a: T[]`	rest
`{ x, y }: P`	destructured

RETURN TYPES

TYPE	USE
`void`	no meaningful return
`never`	always throws / loops
`Promise<T>`	async functions
`x is T`	type predicate
`asserts x`	assertion function

⚠ COMMON MISTAKES

- Writing overloads when a union parameter (or generic) would do
- Marking a callback `() => void` and then relying on its return value
- Putting optional parameters before required ones

✓ TIPS & BEST PRACTICES

- Prefer generics over overloads when the output type follows the input type
- Use an options object type for functions with several optional params
- Annotate return types on exported functions for clearer errors

QUICK SUMMARY

- `(a: T) => R` — function type
- `?` / defaults / rest — params
- overloads — multiple signatures
- `this:` — typed context

Enums & Literal Unions

OVERVIEW

An `enum` names a set of constants and, unlike most TS features, **generates runtime code**. Numeric enums auto-increment; string enums need explicit values. Many teams now prefer **literal unions** or an `as const` object, which give the same safety with no generated code and work with type stripping (`erasableSyntaxOnly`).

NOTES

- Numeric enums create a reverse mapping (value → name)
- `const enum` inlines values but breaks with isolated-module tools
- Enums are nominal: a plain string isn't assignable to a string enum
- Enums aren't allowed under `erasableSyntaxOnly`

ENUMS AND THEIR ALTERNATIVES

```ts
enum Direction { Up, Down, Left, Right }       // 0, 1, 2, 3
Direction.Up;              // 0
Direction[0];              // "Up" (reverse mapping)

enum Status {
  Active = "ACTIVE",
  Banned = "BANNED",
}
function setStatus(s: Status) {}
setStatus(Status.Active);
// setStatus("ACTIVE");    // error: string isn't Status

type StatusU = "ACTIVE" | "BANNED";            // literal union
function setStatus2(s: StatusU) {}
setStatus2("ACTIVE");      // OK, autocompletes

const Role = {                                 // as const object
  Admin: "admin",
  User: "user",
} as const;
type Role = (typeof Role)[keyof typeof Role];  // "admin" | "user"
Role.Admin;                                    // usable at runtime
Object.values(Role);       // ["admin", "user"]
```

CODE EXPLAINED

→ Numeric enum — values start at 0

→ String enum — explicit values, no reverse map

→ Literal union — zero runtime cost

→ `as const` object — runtime values plus a derived union type

→ Same name for value and type is allowed

OPTIONS COMPARED

	ENUM	LITERAL UNION	AS CONST OBJ
runtime code	yes	no	yes (plain object)
iterate values	yes	no	yes
accepts raw strings	no	yes	yes (via type)
type stripping safe	no	yes	yes

WHICH TO USE

NEED	PICK
just a type	literal union
values + type	as const object
bit flags	numeric enum
existing enum code	keep enum

⚠ COMMON MISTAKES

- Using numeric enums and passing any number without error in older TS versions
- Using `const enum` with Babel/esbuild/Vite, which compile files in isolation
- Comparing a string enum to raw strings from an API without mapping

✓ TIPS & BEST PRACTICES

- Default to literal unions; reach for `as const` objects when you need runtime values
- Turn on `erasableSyntaxOnly` to keep code compatible with type stripping
- Validate external strings into your union type with Zod `z.enum`

QUICK SUMMARY

- `enum` — named constants, runtime code
- string enums — explicit values
- literal unions — zero cost
- `as const` object — values + type

Classes in TypeScript

public / private / protected · readonly · parameter properties · abstract · implements · override

OVERVIEW

TypeScript adds compile-time features to classes: **access modifiers** (`public`, `private`, `protected`), `readonly` fields, **parameter properties** that declare and assign in one step, `abstract` classes and methods, `implements` for interfaces, and `override` to mark replaced methods. Remember that TS `private` is not enforced at runtime; JS `#private` is.

NOTES

- Members are `public` by default
- `protected` — visible in subclasses, hidden outside
- `implements` checks the shape; it adds nothing to the class
- Parameter properties aren't erasable syntax

CLASS FEATURES IN TS

```ts
interface Shape { area(): number; name: string }

abstract class Base implements Shape {
  abstract readonly name: string;          // subclasses must define
  protected createdAt = new Date();
  abstract area(): number;
  describe(): string { return `${this.name}: ${this.area().toFixed(1)}`; }
}

class Circle extends Base {
  readonly name = "circle";
  constructor(private readonly r: number) { // parameter property
    super();
  }
  override area(): number { return Math.PI * this.r ** 2; }
}

const c = new Circle(2);
c.describe();              // "circle: 12.6"
// c.r;                    // error: private
// new Base();             // error: abstract class

class Account { #pin = "";  private note = ""; }  // runtime vs TS-only
```

CODE EXPLAINED

- → `abstract` — can't instantiate; forces implementations
- → `implements Shape` — compile error if members are missing
- → `private readonly r` in constructor — declares and assigns
- → `override` — errors if the parent has no such method
- → `#pin` vs `private` — runtime vs compile-time privacy

MODIFIERS

MODIFIER	VISIBLE TO
`public`	everyone (default)
`protected`	class + subclasses
`private`	class only (TS check)
`#name`	class only (runtime)
`readonly`	set once

CLASS KEYWORDS

KEYWORD	DOES
`abstract class`	base only, no new
`abstract m()`	must override
`implements I`	check against interface
`override m()`	mark replacement
`declare x: T`	type-only field

⚠ COMMON MISTAKES

- Relying on TS `private` for security; it's visible in the JS output
- Using parameter properties in projects that run TS via type stripping
- Forgetting `override` after renaming a parent method, leaving a dead method

✓ TIPS & BEST PRACTICES

- Prefer `#private` when privacy must hold at runtime
- Turn on `noImplicitOverride`
- Favor interfaces + composition over deep abstract hierarchies

QUICK SUMMARY

- `public / protected / private` — access
- `abstract` — must implement
- `implements` — shape check
- `override` — explicit replacement

Generics

OVERVIEW

Generics let a function, class, or type work with many types while keeping them connected. A type parameter like <T> is a placeholder filled in at each use, usually by **inference** from the arguments. **Constraints** (`T extends …`) limit what T can be so you can safely use its members, and **defaults** fill in T when it isn't specified.

NOTES

- Name parameters meaningfully when there are several: `<TKey, TValue>`
- If `T` appears only once in a signature, you probably don't need a generic
- Explicit type arguments: `fn<string>(x)`
- In `.tsx`, write `<T,>` for generic arrows

GENERIC FUNCTIONS, CONSTRAINTS, CLASSES

```ts
function first<T>(items: T[]): T | undefined { return items[0]; }
first([1, 2]);              // number | undefined (T inferred)
first(["a"]);               // string | undefined

function longest<T extends { length: number }>(a: T, b: T): T {
  return a.length >= b.length ? a : b;            // .length is safe
}
longest("hi", "hello");     // "hello"
longest([1], [1, 2]);       // number[]
// longest(1, 2);           // error: number has no length

function pluck<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}
pluck({ id: 7, name: "Ana" }, "name");            // string

class Box<T = string> {                           // default type
  constructor(public value: T) {}
  map<U>(fn: (v: T) => U): Box<U> { return new Box(fn(this.value)); }
}
new Box(3).map(n => n.toFixed(1)).value;          // string
type ApiResponse<T> = { data: T; error?: string };
```

CODE EXPLAINED

- → `<T>` — placeholder, inferred from arguments
- → `extends { length: number }` — constraint
- → `K extends keyof T` — key must belong to T
- → `T[K]` — the property's type
- → `<T = string>` — default type argument

GENERIC SYNTAX

SYNTAX	MEANS
`function f<T>(x: T)`	generic function
`<T extends U>`	constraint
`<T = Default>`	default
`class C<T>`	generic class
`type R<T> = …`	generic type
`f<string>(x)`	explicit argument

BUILT-IN GENERICS

TYPE	EXAMPLE
`Array<T>`	Array<number>
`Promise<T>`	Promise<User>
`Map<K, V>`	Map<string, number>
`Record<K, V>`	Record<string, T>

⚠ COMMON MISTAKES

- Adding generics that never relate two things (`<T>(x: T): void` is just `unknown`)
- Over-constraining with `extends any` or `extends object` when no constraint is needed
- Returning `T` from a function that actually builds a new object of a different shape

✓ TIPS & BEST PRACTICES

- Let inference work; supply type arguments only when it can't
- Use `const` type parameters (`<const T>`) to infer literal types
- Start concrete, then generalize once you see repetition

QUICK SUMMARY

- `<T>` — type placeholder
- `extends` — constraint
- `keyof` + `T[K]` — typed property access
- defaults — `<T = X>`

keyof, typeof & Indexed Access

OVERVIEW

Three operators let you build types from other types. `keyof T` gives a union of T's keys. Type-level `typeof x` captures the type of a **value**. **Indexed access** `T[K]` looks up the type of a property, and `T[number]` gets the element type of an array or tuple. Together they let types follow your values automatically.

NOTES

- Type-level `typeof` only works on values (variables, properties)
- `keyof` of an index signature gives `string | number`
- `(typeof arr)[number]` is the classic "union from array" trick
- These operators exist only in type positions

DERIVING TYPES

```ts
type User = { id: number; name: string; tags: string[] };

type UserKey = keyof User;              // "id" | "name" | "tags"
type NameType = User["name"];           // string
type Tag = User["tags"][number];        // string
type IdOrName = User["id" | "name"];    // number | string

const defaults = { theme: "dark", fontSize: 14, beta: false };
type Settings = typeof defaults;
// { theme: string; fontSize: number; beta: boolean }

const ROLES = ["admin", "editor", "viewer"] as const;
type Role = (typeof ROLES)[number];     // "admin" | "editor" | "viewer"

function getSetting<K extends keyof Settings>(key: K): Settings[K] {
  return defaults[key];
}
getSetting("fontSize");                 // number
// getSetting("color");                 // error: not a key

type Handler = typeof window.addEventListener;    // function type
type Ret = ReturnType<typeof getSetting>;         // combine with utilities
```

CODE EXPLAINED

- → `keyof` — union of property names
- → `T["k"]` — type of one property
- → `T[number]` — element type
- → `typeof value` — type of a runtime value
- → `as const` + `[number]` — union from a constant list

TYPE OPERATORS

OPERATOR	PRODUCES
`keyof T`	union of keys
`typeof v`	type of value v
`T[K]`	property type
`T[number]`	array element type
`T[keyof T]`	union of value types

VALUE SPACE VS TYPE SPACE

VALUE	TYPE
`const x = 1`	typeof x → 1
`const o = {…}`	typeof o
`function f()`	typeof f
`class C`	C (instance), typeof C (class)

⚠ COMMON MISTAKES

- Using `typeof` on a type (`typeof User`) instead of a value
- Forgetting `as const`, so `(typeof arr)[number]` is just `string`
- Duplicating a type by hand that could be derived from a value

✓ TIPS & BEST PRACTICES

- Make one source of truth (a const object or schema) and derive types from it
- Use `T[keyof T]` to get the union of an object's value types
- Hover derived types in the editor to confirm what you got

QUICK SUMMARY

- `keyof` — key union
- `typeof` — type from value
- `T[K]` — indexed access
- `[number]` — element type

Utility Types — Objects

OVERVIEW

Utility types are built-in generics that transform object types. They save you from writing near-duplicate interfaces. `Partial` makes everything optional (great for updates), `Required` does the opposite, `Readonly` locks properties, `Pick` and `Omit` select or drop keys, and `Record` builds an object type from keys and a value type.

NOTES

- All utility types are shallow
- `Omit` doesn't check that the keys exist on the type
- Combine them: `Partial<Omit<User, "id">>`
- They're implemented with mapped types (sheet 127)

OBJECT UTILITY TYPES

```ts
interface User {
  id: number;
  name: string;
  email: string;
  avatar?: string;
}

type UserUpdate = Partial<User>;          // every field optional
function update(id: number, patch: Partial<Omit<User, "id">>) {}
update(1, { name: "Bo" });                // OK

type FullUser = Required<User>;  type Frozen = Readonly<User>;

type UserPreview = Pick<User, "id" | "name">;
// { id: number; name: string }

type NewUser = Omit<User, "id">;          // for create forms
// { name: string; email: string; avatar?: string }

type Role = "admin" | "editor";
const perms: Record<Role, string[]> = {
  admin: ["*"],
  editor: ["posts:write"],
};                                        // both keys required
```

CODE EXPLAINED

- → `Partial` — patches and option objects
- → `Omit<T, "id">` — create-forms without server fields
- → `Pick` — lightweight views
- → `Record<Role, …>` — compile error if a role is missing
- → Nest them for precise shapes

OBJECT UTILITIES

UTILITY	RESULT
`Partial<T>`	all props optional
`Required<T>`	all props required
`Readonly<T>`	all props readonly
`Pick<T, K>`	only keys K
`Omit<T, K>`	all except K
`Record<K, V>`	keys K, values V

RECIPES

NEED	TYPE
`update payload`	Partial<Omit<T, "id">>
`create payload`	Omit<T, "id" | "createdAt">
`make one key optional`	Omit<T, K> & Partial<Pick<T, K>>
`lookup table`	Record<Key, Value>

⚠ COMMON MISTAKES

- Typo in an `Omit` key silently does nothing (use a stricter custom helper if needed)
- Expecting `Readonly` or `Partial` to apply to nested objects
- Using `Partial<T>` everywhere and losing the guarantee that fields exist

✓ TIPS & BEST PRACTICES

- Derive request/response shapes from one model type instead of rewriting them
- Name composed utility types (`type NewUser = …`) for readability
- Use type-fest for deep variants like `PartialDeep`

QUICK SUMMARY

- `Partial / Required` — optionality
- `Readonly` — immutability
- `Pick / Omit` — select keys
- `Record` — keys → values

Utility Types — Unions & Functions

OVERVIEW

A second family of utility types works on **unions** and **functions**. `Exclude` and `Extract` filter union members, and `NonNullable` strips `null` and `undefined`. For functions, `ReturnType` and `Parameters` capture signatures, `Awaited` unwraps Promises, and `InstanceType` gets the instance type of a class.

NOTES

- Use these with `typeof` on functions and classes (values)
- `Awaited` unwraps nested Promises recursively
- `NoInfer<T>` (TS 5.4) blocks inference from a position
- String helpers: `Uppercase`, `Lowercase`, `Capitalize`, `Uncapitalize`

UNION AND FUNCTION UTILITIES

```ts
type Status = "idle" | "loading" | "success" | "error";
type Busy = Extract<Status, "loading">;              // "loading"
type Settled = Exclude<Status, "idle" | "loading">;  // "success" | "error"
type Maybe = string | null | undefined;
type Sure = NonNullable<Maybe>;                      // string

async function fetchUser(id: number, opts?: { cache: boolean }) {
  return { id, name: "Ana" };
}
type FetchArgs = Parameters<typeof fetchUser>;
// [id: number, opts?: { cache: boolean }]
type FetchRet = ReturnType<typeof fetchUser>;  // Promise<{ id; name }>
type User = Awaited<FetchRet>;                 // { id: number; name: string }

class Logger { log(m: string) {} }
type L = InstanceType<typeof Logger>;          // Logger
type Args = ConstructorParameters<typeof Date>;

function wrap<F extends (...a: any[]) => any>(fn: F) {
  return (...args: Parameters<F>): ReturnType<F> => fn(...args);
}
type Event = `on${Capitalize<"click" | "focus">}`;   // "onClick" | "onFocus"
```

CODE EXPLAINED

→ `Extract` / `Exclude` — keep or drop union members

→ `Parameters` — tuple of argument types

→ `ReturnType` + `Awaited` — the resolved result type

→ `wrap` — preserves the wrapped function's signature

→ `Capitalize` — string literal manipulation

UNION UTILITIES

UTILITY	RESULT
`Exclude<U, X>`	U without X
`Extract<U, X>`	members of U in X
`NonNullable<T>`	no null/undefined
`NoInfer<T>`	don't infer from here

FUNCTION & CLASS UTILITIES

UTILITY	GIVES
`Parameters<F>`	argument tuple
`ReturnType<F>`	return type
`Awaited<T>`	unwrapped Promise
`InstanceType<C>`	instance type
`ConstructorParameters<C>`	constructor args
`ThisParameterType<F>`	this type

⚠ COMMON MISTAKES

- Writing `ReturnType<fetchUser>` instead of `ReturnType<typeof fetchUser>`
- Forgetting `Awaited` and getting `Promise<User>` where you wanted `User`
- Using `Exclude` on object types; it filters unions, not properties (use `Omit`)

✓ TIPS & BEST PRACTICES

- Derive types from third-party functions you don't control with `Parameters` / `ReturnType`
- Combine `Awaited<ReturnType<typeof fn>>` for async results
- Use `NonNullable` after filtering arrays of optional values

QUICK SUMMARY

- `Exclude / Extract` — filter unions
- `NonNullable` — remove nullish
- `Parameters / ReturnType` — function types
- `Awaited` — unwrap Promises

Conditional Types

T extends U ? X : Y · infer · distributive conditionals · filtering with never

OVERVIEW

A **conditional type** chooses a type based on a test: `T extends U ? X : Y`. Inside the true branch, `infer` declares a type variable to **extract** part of a type, like a Promise's value or a function's first argument. When T is a naked type parameter and receives a union, the condition is **distributed** over each member.

NOTES

- Conditional types are how `ReturnType`, `Awaited`, `Exclude` are built
- Wrap in brackets (`[T] extends [U]`) to stop distribution
- Returning `never` from a branch removes that union member
- Keep them small; deeply recursive types slow the compiler

CONDITIONAL TYPES AND INFER

```
type IsString<T> = T extends string ? "yes" : "no";
type A = IsString<"hi">;                  // "yes"
type B = IsString<42>;                    // "no"

type ElementOf<T> = T extends (infer E)[] ? E : T;
type C = ElementOf<number[]>;             // number
type D = ElementOf<string>;               // string

type UnwrapPromise<T> = T extends Promise<infer V> ? V : T;
type E = UnwrapPromise<Promise<Date>>;    // Date

type FirstArg<F> = F extends (first: infer A, ...rest: any[]) => any ? A : never;
type F1 = FirstArg<(id: number, x: string) => void>;  // number

type MyExclude<T, U> = T extends U ? never : T;       // distributes
type G = MyExclude<"a" | "b" | "c", "a">;             // "b" | "c"

type ToArray<T> = T extends any ? T[] : never;
type H = ToArray<string | number>;        // string[] | number[]
type ToArrayND<T> = [T] extends [any] ? T[] : never;
type I = ToArrayND<string | number>;      // (string | number)[]
```

CODE EXPLAINED

- → `extends` in a type — "is assignable to"
- → `infer E` — capture a piece of the matched type
- → `never` in a branch — filters members out
- → Distribution — applied to each union member
- → `[T]` — disable distribution

SYNTAX

FORM	MEANING
`T extends U ? X : Y`	if T fits U, X else Y
`infer R`	capture type R
`T extends U ? never : T`	filter out U
`[T] extends [U]`	non-distributive

BUILT FROM CONDITIONALS

UTILITY	DEFINITION IDEA
`Exclude<T, U>`	T extends U ? never : T
`NonNullable<T>`	T & {}
`ReturnType<F>`	F extends (…) => infer R ? R : any
`Awaited<T>`	recursive Promise unwrap

⚠ COMMON MISTAKES

- Being surprised by distribution over unions; bracket the check to avoid it
- Writing complex conditional types where an overload or simple union is clearer
- Using `infer` outside an `extends` clause (not allowed)

✓ TIPS & BEST PRACTICES

- Test types with small `type T1 = …` lines and hover to verify
- Name helper types clearly; they become documentation
- Check type-fest before writing your own advanced utilities

QUICK SUMMARY

- `T extends U ? X : Y` — type if/else
- `infer` — extract part of a type
- distributive — over unions
- `never` — remove members

Mapped & Template Literal Types

[K in keyof T] · +/- modifiers · as remapping · template literal types · Capitalize / Uppercase

OVERVIEW

A **mapped type** builds a new object type by looping over keys: `{ [K in keyof T]: … }`. Modifiers +/− add or remove `readonly` and ?, and an `as` clause **remaps** keys, including renaming or filtering them. **Template literal types** build string types from other strings, which pairs perfectly with key remapping.

NOTES

- `Partial`, `Readonly`, `Record`, `Pick` are all mapped types
- `-?` removes optionality; `-readonly` removes readonly
- Remapping a key to `never` drops it
- Template literal types distribute over unions

MAPPED AND TEMPLATE LITERAL TYPES

```ts
type User = { id: number; name: string; email?: string };

type Nullable<T> = { [K in keyof T]: T[K] | null };
type Mutable<T> = { -readonly [K in keyof T]: T[K] };
type Concrete<T> = { [K in keyof T]-?: T[K] };      // like Required

type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K];
};
type UserGetters = Getters<User>;
// { getId: () => number; getName: () => string; getEmail?: … }

type OnlyStrings<T> = {
  [K in keyof T as T[K] extends string ? K : never]: T[K];
};
type S = OnlyStrings<User>;              // { name: string }

type Method = "get" | "post";
type Path = "/users" | "/posts";
type Route = `${Uppercase<Method>} ${Path}`;
// "GET /users" | "GET /posts" | "POST /users" | "POST /posts"

type CssVar = `--${string}`;
const v: CssVar = "--accent";            // OK
type EventName<T extends string> = `${T}Changed`;
```

CODE EXPLAINED

→ `[K in keyof T]` — loop over keys

→ `-readonly` / `-?` — strip modifiers

→ `as` + template literal — rename keys

→ `as … ? K : never` — filter keys by value type

→ Template literal unions — every combination

MAPPED TYPE PIECES

SYNTAX	DOES
`[K in keyof T]`	iterate keys
`[K in Union]`	keys from a union
`readonly / ?`	add modifier
`-readonly / -?`	remove modifier
`as NewKey`	rename / filter

STRING HELPERS

HELPER	"HELLO WORLD" →
`Uppercase<S>`	"HELLO WORLD"
`Lowercase<S>`	"hello world"
`Capitalize<S>`	"Hello world"
`Uncapitalize<S>`	"hello world"

⚠ COMMON MISTAKES

- Forgetting `string & K` inside template literals when keys may include numbers or symbols
- Generating huge unions from template literals (they multiply quickly)
- Rebuilding utility types that already exist

✓ TIPS & BEST PRACTICES

- Use key remapping to generate event, getter, or action names from a model
- Type CSS custom properties and route strings with template literals
- Hover the result; mapped types are easier to verify than to read

QUICK SUMMARY

- `[K in keyof T]` — mapped type
- `+/- readonly ?` — modifiers
- `as` — remap or filter keys
- template literal types — string patterns

Declaration Files

OVERVIEW

Declaration files (`.d.ts`) describe the types of JavaScript code without containing any implementation. Most packages ship their own; others have community types in `@types/*`. Use `declare` to describe things that exist at runtime but TS doesn't know about, and **augmentation** to add to existing global or module types.

NOTES

- `.d.ts` files contain only types and `declare` statements
- `tsc --declaration` generates them from your `.ts` code
- A `.d.ts` with no import/export adds **global** declarations
- Use `declare module "x"` for untyped packages or file imports

DECLARATIONS AND AUGMENTATION

```ts
// types/globals.d.ts
declare const __APP_VERSION__: string;        // injected by bundler
interface Window { analytics?: { track(e: string): void } }

// types/untyped-lib.d.ts
declare module "tiny-slugify" {
  export default function slugify(s: string, sep?: string): string;
}

declare module "*.svg" { const url: string; export default url; }

// augment a module's types (file must be a module)
import "express";
declare module "express" {
  interface Request { user?: { id: number; role: string } }
}

// add globals from inside a module file
declare global { var __DEV__: boolean }

// $ npm i -D @types/node @types/express
```

CODE EXPLAINED

→ `declare const` — value exists at runtime; trust me

→ `declare module "pkg"` — types for an untyped package

→ Wildcard modules — type asset imports

→ Module augmentation — add fields to library types

→ `declare global` — extend globals from a module file

WHERE TYPES COME FROM

SOURCE	EXAMPLE
`bundled in package`	"types" in package.json
`DefinitelyTyped`	@types/express
`your .d.ts files`	types/*.d.ts
`lib setting`	DOM, ES2024
`generated`	tsc --declaration

DECLARATION KEYWORDS

FORM	DECLARES
`declare const / let`	global value
`declare function`	global function
`declare module "m"`	module's types
`declare global { }`	global additions
`export {}`	make file a module

⚠ COMMON MISTAKES

- Declaring things that don't actually exist at runtime (types can't create them)
- Writing augmentation in a non-module file, so it replaces instead of merges
- Installing `@types/x` when package `x` already bundles its types

✓ TIPS & BEST PRACTICES

- Check the npm page for the TS badge before installing `@types`
- Keep custom declarations in a `types/` folder included by tsconfig
- Write a minimal `declare module` to unblock yourself, then improve it

QUICK SUMMARY

- `.d.ts` — types only
- `@types/*` — community types
- `declare` — describe runtime values
- augmentation — extend existing types

Assertions & Escape Hatches

as · as unknown as · ! · @ts-expect-error · @ts-ignore · any vs unknown

OVERVIEW

Sometimes you know more than the compiler. **Type assertions** (`as T`) and the **non-null assertion** (`!`) override its judgment, and `@ts-expect-error`/`@ts-ignore` silence specific errors. These are escape hatches: they remove safety rather than add it, so use them sparingly, narrowly, and with a comment explaining why.

NOTES

- Assertions don't change runtime values or do conversions
- TS blocks wildly incompatible assertions unless you go through `unknown`
- `@ts-expect-error` fails if there's no error, so stale ones get noticed
- `as const` is an assertion too, but a safe one

ESCAPE HATCHES

```ts
const input = document.getElementById("email") as HTMLInputElement;
input.value;                                // OK, but no runtime check

const el = document.querySelector(".total")!;   // asserts non-null
el.textContent = "42";                      // crashes if it's missing

const n = "42" as unknown as number;        // double assertion: a lie
n.toFixed(2);                               // runtime TypeError

// @ts-expect-error: legacy API takes a string here, fixed in v3
legacyFormat(123);

// @ts-ignore                               // silences anything (avoid)
riskyCall();

// safer alternatives
const maybe = document.querySelector<HTMLInputElement>("#email");
if (maybe) maybe.value;                     // real check
function assertEl<T extends Element>(e: T | null): T {
  if (!e) throw new Error("element missing");
  return e;
}
const total = assertEl(document.querySelector(".total"));
```

CODE EXPLAINED

- → `as T` — tell TS the type; unchecked
- → `!` — remove `null` / `undefined` from the type
- → `as unknown as T` — forces any assertion
- → `@ts-expect-error` — preferred over `@ts-ignore`
- → Assertion helpers — keep the check, keep the type

ESCAPE HATCHES

TOOL	RISK
`as T`	medium: unchecked cast
`x!`	medium: null crash
`as unknown as T`	high: anything goes
`any`	high: spreads silently
`@ts-expect-error`	low-medium: documented
`@ts-ignore`	high: hides future errors

USE INSTEAD

INSTEAD OF	USE
`x as Foo`	type guard / Zod parse
`el!`	if (el) or assert helper
`any`	unknown + narrowing
`@ts-ignore`	@ts-expect-error + reason

⚠ COMMON MISTAKES

- Asserting API data with `as User` instead of validating it
- Sprinkling `!` after every DOM query
- Leaving `@ts-ignore` comments that hide new, unrelated errors later

✓ TIPS & BEST PRACTICES

- Require a reason after `@ts-expect-error` (typescript-eslint `ban-ts-comment`)
- Search the codebase for `as unknown as` periodically
- Enable `no-non-null-assertion` as a warning

QUICK SUMMARY

- `as T` — unchecked cast
- `!` — assume non-null
- `@ts-expect-error` — documented suppression
- prefer guards and validation

Decorators

OVERVIEW

Decorators are functions that wrap or annotate classes and their members using `@name` syntax. TypeScript 5.0+ implements the **standard** (TC39 stage 3) decorators: each receives the original value and a **context** object, and can return a replacement. Older code and some frameworks (Angular, NestJS) still use the legacy `experimentalDecorators` version, which works differently.

NOTES

- Standard decorators need no tsconfig flag
- The two versions aren't compatible; check what your framework uses
- Parameter decorators exist only in the legacy version
- The `accessor` keyword creates an auto getter/setter that decorators can wrap

STANDARD DECORATORS

```ts
function logged<T extends (...a: any[]) => any>(
  target: T, ctx: ClassMethodDecoratorContext) {
  return function (this: unknown, ...args: Parameters<T>): ReturnType<T> {
    console.log(`→ ${String(ctx.name)}(${args.join(", ")})`);
    return target.apply(this, args);
  };
}

function clamp(min: number, max: number) {          // decorator factory
  return (t: ClassAccessorDecoratorTarget<unknown, number>,
          _ctx: ClassAccessorDecoratorContext) => ({
    set(this: unknown, v: number) {
      t.set.call(this, Math.min(max, Math.max(min, v)));
    },
  });
}

class Calculator {
  @clamp(0, 100) accessor percent = 50;
  @logged add(a: number, b: number) { return a + b; }
}
new Calculator().add(2, 3);       // logs "→ add(2, 3)", returns 5
```

CODE EXPLAINED

- → Method decorator — returns a replacement function
- → `ctx.addInitializer` — run setup per instance (e.g. auto-bind)
- → Factory — a function that returns a decorator
- → `accessor` — decoratable auto-property
- → `ctx.name` — the decorated member's name

DECORATOR TARGETS

TARGET	CONTEXT TYPE
`class`	ClassDecoratorContext
`method`	ClassMethodDecoratorContext
`getter / setter`	ClassGetterDecoratorContext …
`field`	ClassFieldDecoratorContext
`accessor`	ClassAccessorDecoratorContext

STANDARD VS LEGACY

	STANDARD	LEGACY
`tsconfig`	none	experimentalDecorators
`signature`	(value, context)	(target, key, desc)
`parameter decorators`	no	yes
`metadata`	Symbol.metadata	reflect-metadata

⚠ COMMON MISTAKES

- Mixing standard decorator code with a framework that needs `experimentalDecorators`
- Assuming decorators work on plain functions; they're for classes only
- Putting heavy logic in decorators, making behavior hard to trace

✓ TIPS & BEST PRACTICES

- Use decorators for cross-cutting concerns: logging, binding, validation, timing
- Keep decorators small and well named
- Check runtime support: TS downlevels them, but native JS support is still arriving

QUICK SUMMARY

- `@dec` — wrap class members
- `(value, context)` — standard signature
- factories — configurable decorators
- legacy — `experimentalDecorators`

Adopting TypeScript

OVERVIEW

You can adopt TypeScript **gradually**. Start by type-checking existing JavaScript with `checkJs` and JSDoc, rename files to `.ts` one at a time with `allowJs`, then raise strictness flag by flag. For large codebases, **project references** split the code into separately built pieces, and `tsc -b` builds them incrementally.

NOTES

- `allowJs` lets `.ts` and `.js` files import each other
- `// @ts-check` opts a single JS file into checking
- `// @ts-nocheck` opts a file out during migration
- `incremental` caches results in a `.tsbuildinfo` file

MIGRATION PATH AND PROJECT REFERENCES

```
// Step 1: check JS without renaming (tsconfig.json)
{ "compilerOptions": { "allowJs": true, "checkJs": true, "noEmit": true } }

// Step 2: JSDoc types in .js files
/** @param {string} name @returns {string} */
export function greet(name) { return `Hi ${name}`; }

// Step 3: rename leaf modules to .ts, add real annotations
// Step 4: turn on strict checks one at a time
"noImplicitAny": true  →  "strictNullChecks": true  →  "strict": true

// Monorepo: tsconfig.json at the root
{ "files": [],
  "references": [{ "path": "./packages/core" },
                 { "path": "./packages/app" }] }

// packages/core/tsconfig.json
{ "compilerOptions": { "composite": true, "outDir": "dist" } }

$ npx tsc -b              // build all references in order
$ npx tsc -b --watch      // incremental rebuilds
```

CODE EXPLAINED

- → `checkJs` — find bugs before converting anything
- → JSDoc — types without changing file extensions
- → Convert leaf modules first (fewest dependents)
- → `composite` — required for referenced projects
- → `tsc -b` — builds only what changed

MIGRATION FLAGS

FLAG	PURPOSE
`allowJs`	mix JS and TS
`checkJs`	type-check JS
`noEmit`	check only
`incremental`	cache results
`composite`	enable references
`noImplicitAny`	first strict step

SUGGESTED STRICTNESS ORDER

STEP	FLAG
1	noImplicitAny
2	strictNullChecks
3	strict
4	noUncheckedIndexedAccess
5	exactOptionalPropertyTypes

⚠ COMMON MISTAKES

- Converting everything in one huge PR
- Using `any` everywhere to get green builds, then never removing it
- Turning on `strict` all at once on a large codebase

✓ TIPS & BEST PRACTICES

- Track remaining errors per flag and burn them down over time
- Convert shared utilities and types first so everything benefits
- Use `ts-migrate` or similar tools for large mechanical conversions

QUICK SUMMARY

- `checkJs` — check JS first
- `allowJs` — mix while converting
- strict flags — one at a time
- project references — `tsc -b`

Sheet 132

Quick Reference — JavaScript Language

declarations · types & coercion · strings · operators · control flow · errors · functions · arrays · objects · Map & Set · classes · async

DECLARATIONS & TYPES

SYNTAX	DOES	#
`const / let`	block-scoped bindings	007
`typeof x`	type name string	014
`Number("42") / parseInt(s, 10)`	parse	013
`String(x) / x.toString()`	to text	013
`10n`	BigInt	009
`null vs undefined`	empty vs missing	011

STRINGS

SYNTAX	DOES	#
`` `Hi ${name}` ``	template literal	017
`s.includes / startsWith`	search	018
`s.slice(1, -1)`	substring	020
`s.split(",") / xs.join()`	split / join	019
`s.trim() / padStart(3, "0")`	transform	019
`s.replaceAll("a", "b")`	replace all	019

OPERATORS

OPERATOR	DOES	#
`=== / !==`	strict equality	023
`a ?? b`	nullish fallback	025
`a?.b?.()`	optional chaining	025
`x ||= / ??= / &&=`	logical assignment	026
`...xs`	spread / rest	028
`in / instanceof`	property / prototype	029

CONTROL FLOW & ERRORS

SYNTAX	DOES	#
`if / else if / else`	branch	030
`for (const x of xs)`	values	034
`for (const k in obj)`	keys (own + inherited)	034
`try / catch / finally`	handle	037
`throw new Error("x", { cause })`	throw	038
`class MyErr extends Error`	custom	039

FUNCTIONS

SYNTAX	DOES	#
`function f(a, b = 1)`	declaration + default	041
`const f = (x) => x * 2`	arrow	046
`(...args) => args.length`	rest params	043
`closures`	inner fn keeps outer vars	045
`function* gen() { yield 1 }`	generator	062

ARRAYS

METHOD	DOES	#
`map / filter / reduce`	transform	052
`find / findLast / some / every`	search & test	053
`push / pop / shift / splice`	mutate	051
`toSorted / toReversed / with`	copy-returning	052
`flat / flatMap`	flatten	052
`sort((a, b) => a - b)`	numeric sort	060

OBJECTS

SYNTAX	DOES	#
`{ a, b, [key]: v }`	shorthand / computed	054
`const { a, b: c = 1 } = obj`	destructure	056
`Object.keys / entries`	iterate	055
`structuredClone(obj)`	deep copy	057
`Object.groupBy(xs, fn)`	group	055

MAP, SET & CLASSES

SYNTAX	DOES	#
`new Map([[k, v]])`	any-key map	058
`new Set(xs)`	unique values	059
`a.union(b) / intersection`	set methods	059
`class A extends B { }`	class	068
`#private / static x`	private / static	066

ASYNC

SYNTAX	DOES	#
`async function / await`	async code	094
`Promise.all([…])`	all, fail fast	095
`Promise.allSettled`	all results	095
`Promise.race / any`	first settled / fulfilled	095
`AbortController`	cancel fetch	096

⚠ COMMON MISTAKES

- `==` coercion surprises — use `===`
- `forEach` with `async` callbacks doesn't wait; use `for…of` or `Promise.all`
- `sort()` without a compare function sorts numbers as strings

✓ TIPS & BEST PRACTICES

- Default to `const`; use `let` only when reassigning
- Prefer copy-returning array methods (`toSorted`, `with`) over mutation
- Use `??` instead of `||` when `0` or `""` are valid values

QUICK SUMMARY

- `const` / `let`, `===`, template literals
- `map` / `filter` / `reduce` and destructuring
- `?.` and `??` for missing values
- `async` / `await` + `Promise.all`

Sheet 133

Quick Reference — Modules, DOM, Node & TypeScript

ES modules · npm · DOM & events · fetch & storage · Node files & process · tooling · TypeScript types, narrowing, generics & utility types

MODULES & NPM

SYNTAX	DOES	#
`import { a } from "./a.js"`	named import	076
`export default fn`	default export	076
`await import("./x.js")`	dynamic import	076
`require / module.exports`	CommonJS	077
`npm i -D vitest`	dev dependency	078
`"type": "module"`	ESM package	078

DOM & EVENTS

API	DOES	#
`document.querySelector(".x")`	select	097
`el.textContent / classList`	content / classes	098
`document.createElement / append`	build	099
`el.addEventListener("click", fn)`	listen	100
`e.target.closest("li")`	delegation	101
`new FormData(form)`	form values	102

BROWSER APIS

API	DOES	#
`await fetch(url)`	HTTP request	103
`res.ok / res.json()`	check / parse	103
`localStorage.setItem(k, v)`	persistent storage	104
`new URL(href).searchParams`	query string	105
`history.pushState`	change URL	105
`IntersectionObserver`	visibility	106

NODE & TOOLING

API	DOES	#
`fs.promises.readFile(p, "utf8")`	read file	072
`path.join(a, b)`	build path	074
`process.env / argv`	env / args	107
`import.meta.dirname`	current dir (ESM)	074
`vitest / eslint / prettier`	test / lint / format	082
`z.object({ … }).parse(x)`	Zod validation	085

TS BASICS

SYNTAX	DOES	#
`let n: number`	annotation	113
`type User = { id: string }`	type alias	115
`interface User { id: string }`	interface	115
`name?: string`	optional	116
`A | B / A & B`	union / intersection	117
`"a" | "b"`	literal union	120
`x satisfies T`	check without widening	114

NARROWING & FUNCTIONS

SYNTAX	DOES	#
`typeof x === "string"`	typeof guard	118
`"key" in obj`	in guard	118
`x is Fish`	type predicate	118
`switch (s.kind)`	discriminated union	118
`(a: number) => string`	function type	119
`never`	exhaustiveness check	118

GENERICS & OPERATORS

SYNTAX	DOES	#
`function f<T>(x: T): T`	generic	122
`<T extends { id: string }>`	constraint	122
`keyof T / T[K]`	keys / indexed access	123
`typeof obj`	type from value	123
`T extends U ? X : Y`	conditional type	126
`{ [K in keyof T]: … }`	mapped type	127

UTILITY TYPES & CONFIG

ITEM	DOES	#
`Partial / Required / Readonly`	modify props	124
`Pick<T, K> / Omit<T, K>`	select props	124
`Record<K, V>`	object map	124
`ReturnType / Awaited`	infer from functions	125
`"strict": true`	tsconfig must-have	112
`tsc --noEmit`	type-check only	111

⚠ COMMON MISTAKES

- `as` casts to silence errors — they hide real bugs; narrow instead
- `any` spreading through a codebase — use `unknown` and narrow
- Forgetting `res.ok` — `fetch` only rejects on network errors, not 4xx / 5xx

✓ TIPS & BEST PRACTICES

- Turn on `"strict": true` from the first commit
- Model state as discriminated unions with a `kind` field
- Use event delegation for lists instead of a listener per item

QUICK SUMMARY

- ES modules + npm scripts
- `querySelector` + `addEventListener` + `fetch` in the browser
- TS: types, unions, narrowing, generics
- `Partial`, `Pick`, `Omit`, `Record` save boilerplate